

BERT 과정

Self-Attention과 Transformer

목차 Self-Attention과 Transformer

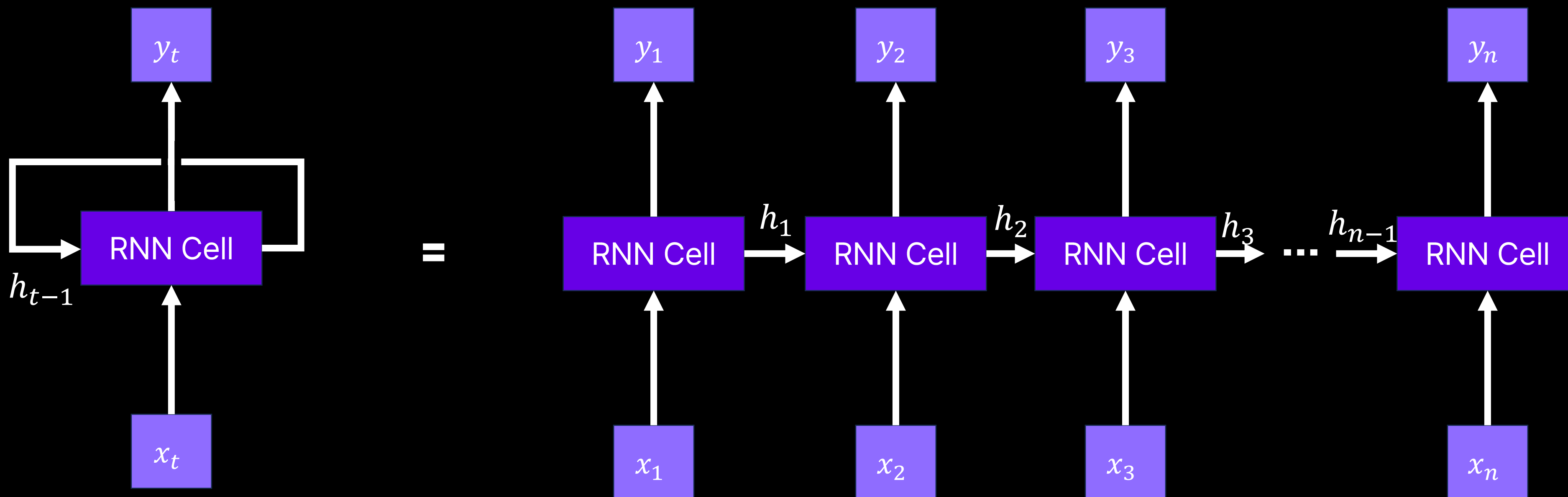
1. RNN, LSTM
2. Seq2seq과 Attention
3. Self-Attention 알고리즘
4. Transformer 아키텍처
5. PyTorch로 구현한 Transformer

1. RNN, LSTM

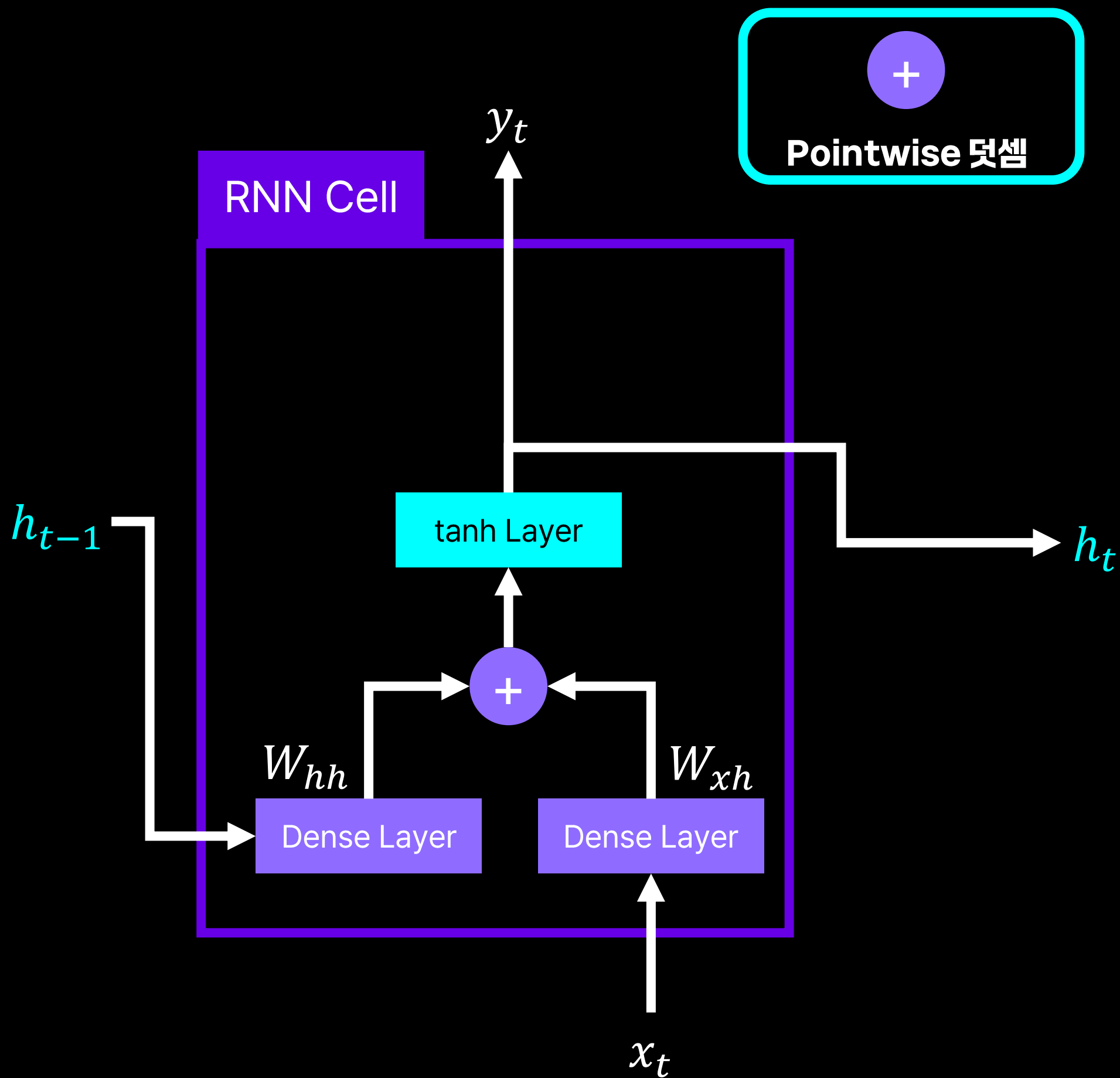


순환 신경망 (RNN)

- RNN (Recurrent Neural Network)은 **같은 구조가 반복되는** 모델을 통해 시퀀스 데이터를 처리하는 신경망이다.
- **같은 구조는 같은 가중치를 공유한다.**



RNN Cell

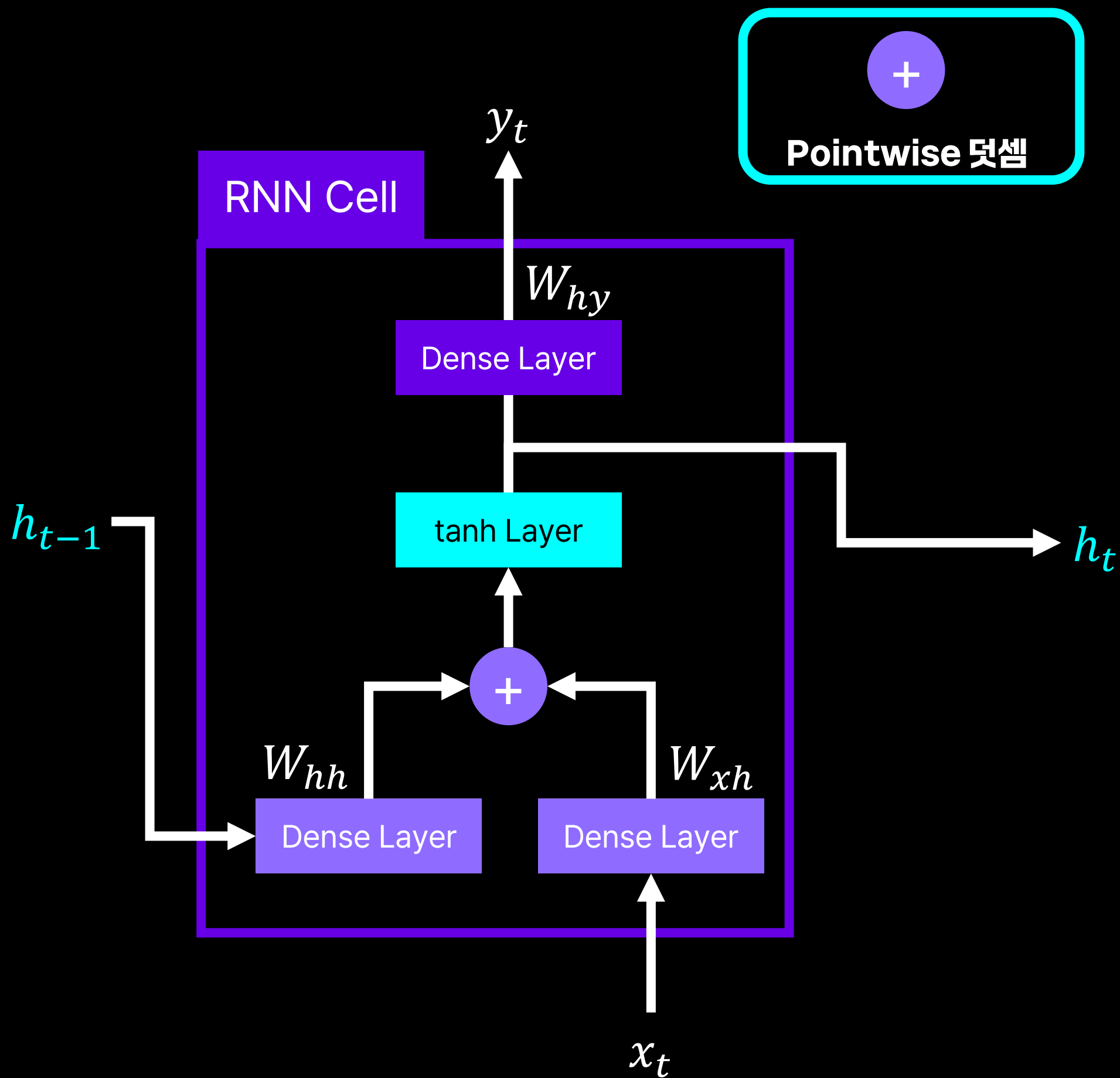


순환 신경망 (RNN)의 반복되는 단위 구조

각 RNN Cell은 입력값 x_t 와 이전의 은닉 상태 (Hidden State) h_{t-1} 을 입력으로 받아 현재 은닉 상태 h_t 와 현재 상태에 대한 출력 값 y_t 를 계산한다.

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b)$$

RNN Cell



순환 신경망 (RNN)의 반복되는 단위 구조

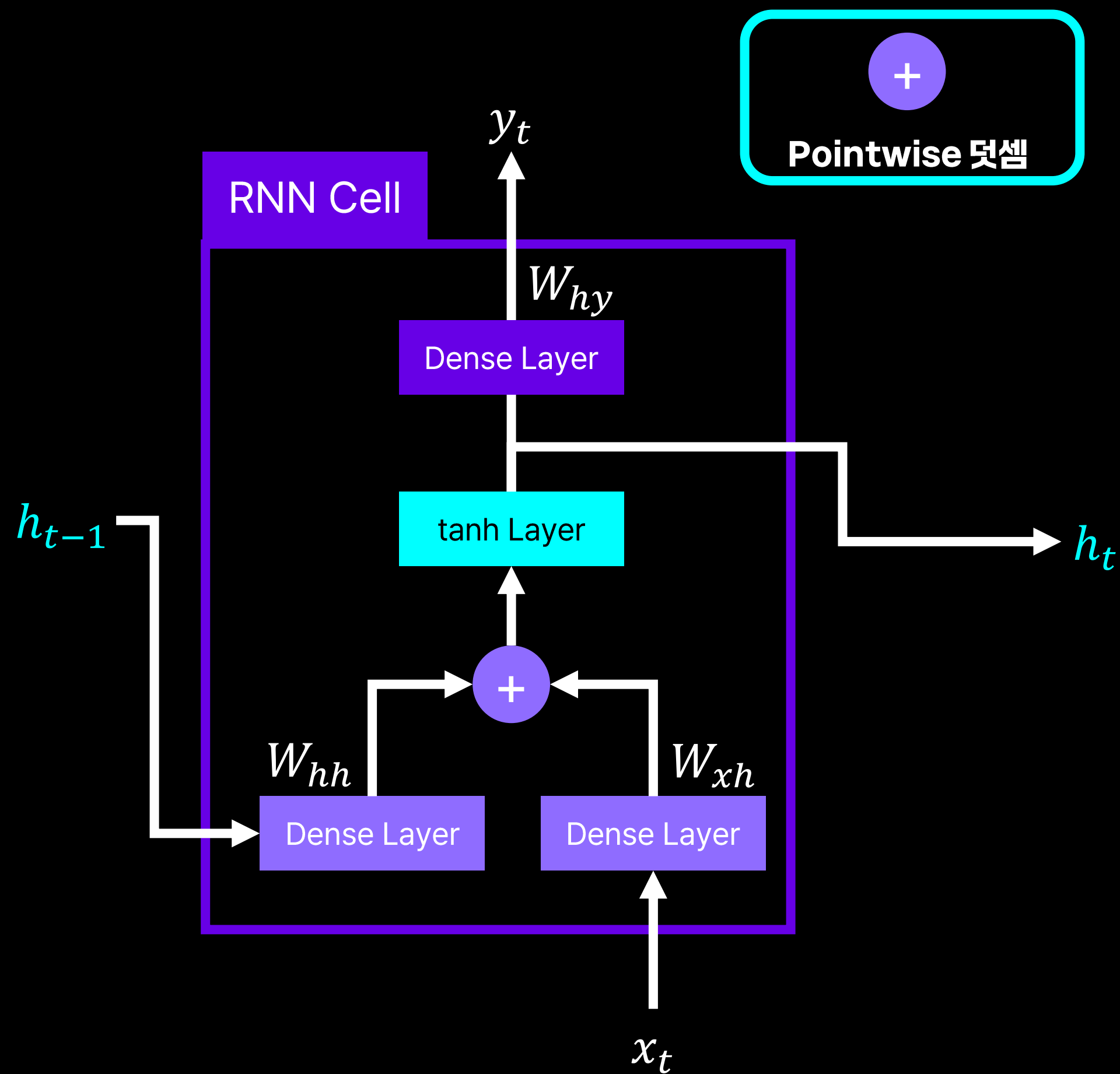
각 RNN Cell은 **입력값** x_t 와 이전의 **은닉 상태 (Hidden State)** h_{t-1} 을 입력으로 받아 현재 **은닉 상태** h_t 와 **현재 상태에 대한 출력 값** y_t 를 계산한다.

현재 상태에 대한 출력 값을 계산할 때 다른 Dense 레이어를 추가할 수 있다.

$$y_t = W_{hy}h_t + b$$



RNN Cell의 은닉 상태



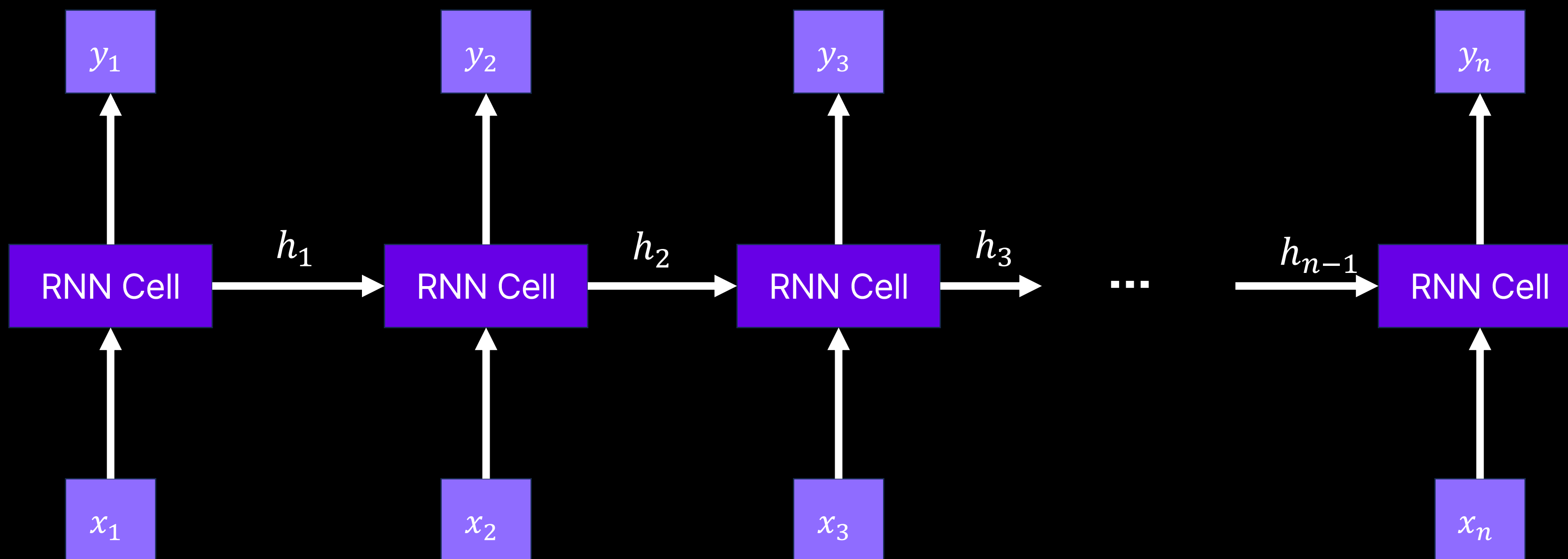
은닉 상태(Hidden State) h_t

- 특정 시점 t 까지 들어온 입력 값($x_1 \sim x_t$) 까지의 정보를 압축해서 저장한 값으로, 일종의 메모리 역할을 한다.
- 은닉 상태를 구성할 때는 이전의 은닉 상태의 값과 현재 입력 값을 단순 합산하여 생성한다.
 - 초기 은닉 상태 h_0 는 영벡터 혹은 x_1 을 사용할 수 있다.
- 이후, 상태가 끝없이 커지는 것을 방지하기 위해 tanh Layer를 통해 $[0, 1]$ 범위로 정규화 한다.



단순 RNN의 문제점

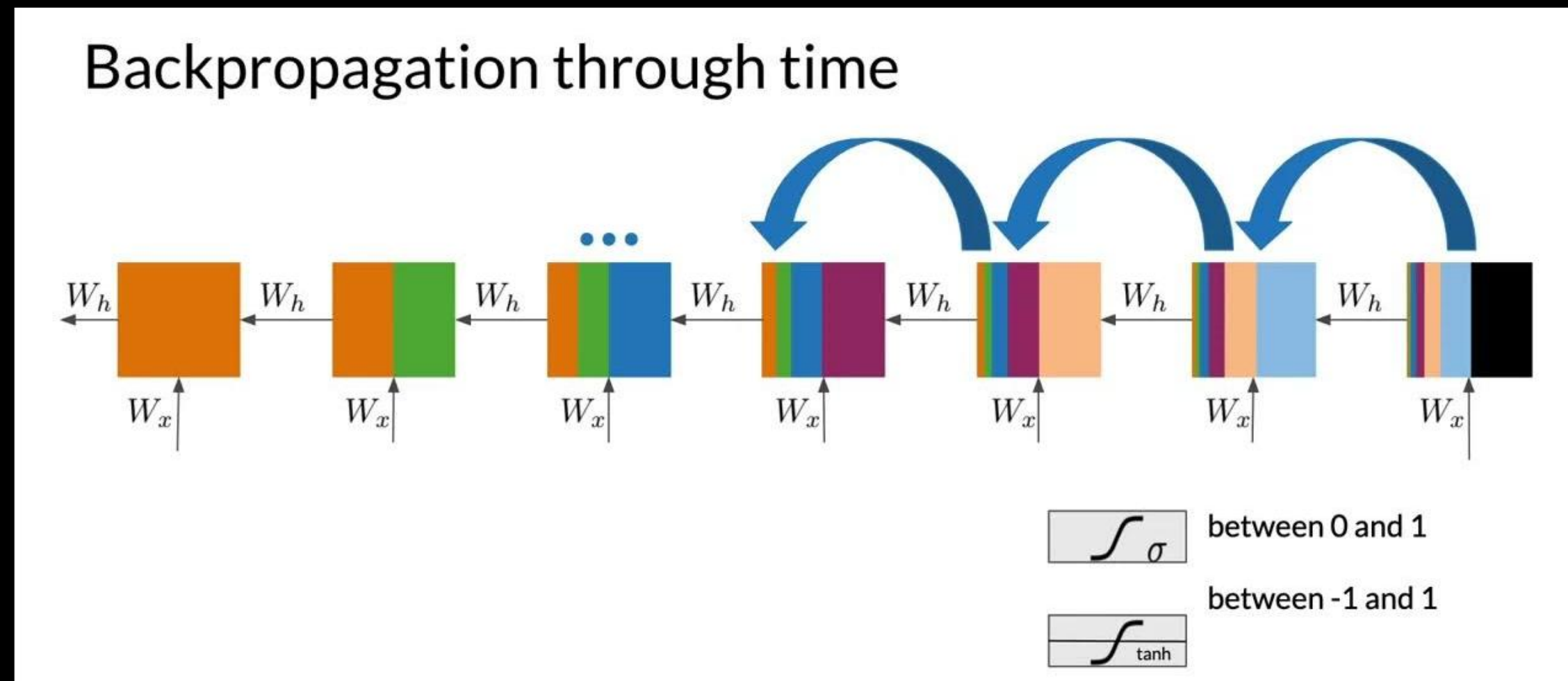
- 단순 RNN은 출력값이 시간 순서에 따라 생성된다.
- 이로 인해 **각 시점의** 출력값과 실제 값을 비교하여 **손실을 계산한다**.
- 모델을 학습시킬 때 **시간 순서에 따라** 최적화가 이루어진다
 - **Back Propagation Through Time (BPTT)**





단순 RNN의 문제점

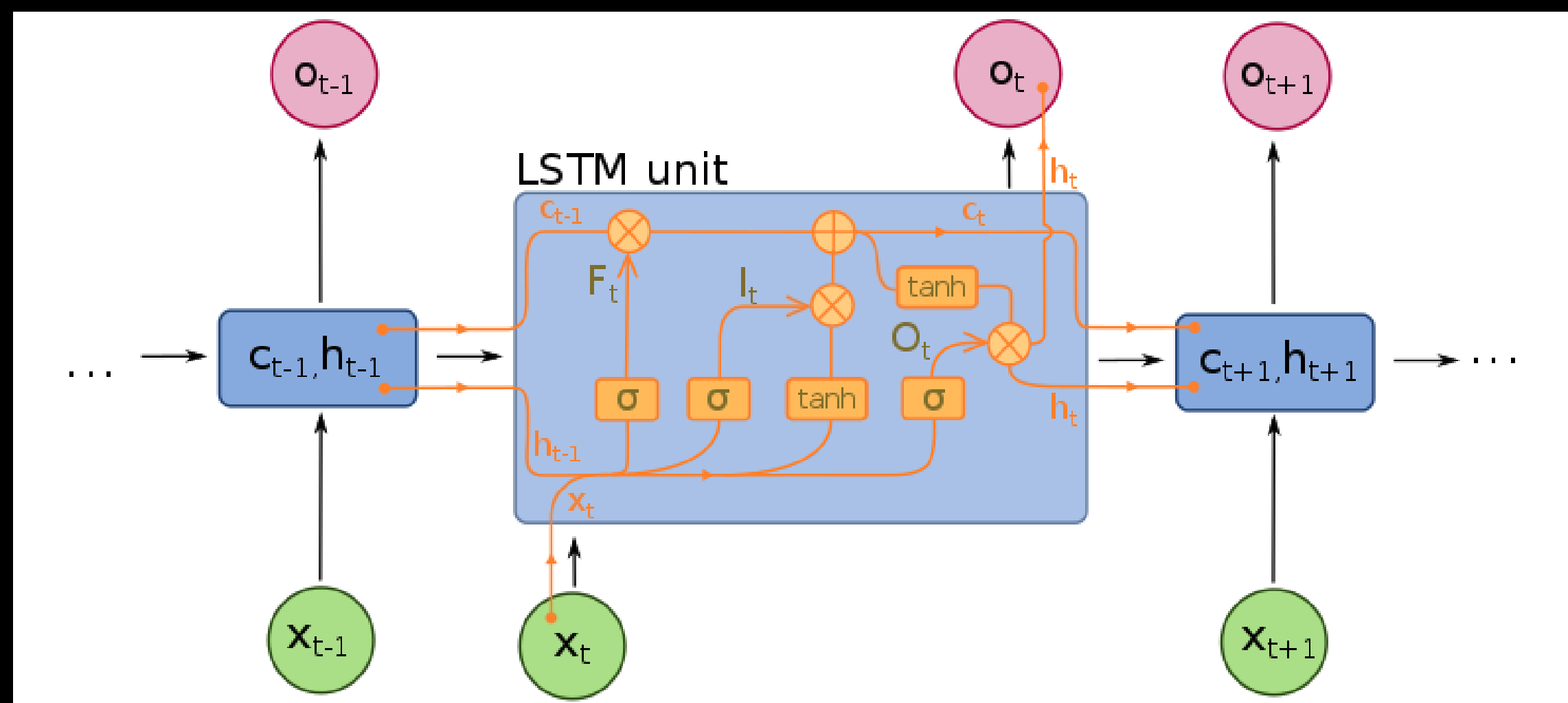
- 입력 시퀀스가 매우 길어질 경우, 나중 출력값의 오차가 초기 출력값의 최적화에 매우 적은 영향을 미칠 수 있다.
 - 기울기 소실 문제
- 또한, 자연어 같이 시퀀스 내 거리가 먼 요소 간의 의존성을 학습하기 어려워진다.
 - 장기 의존성 문제





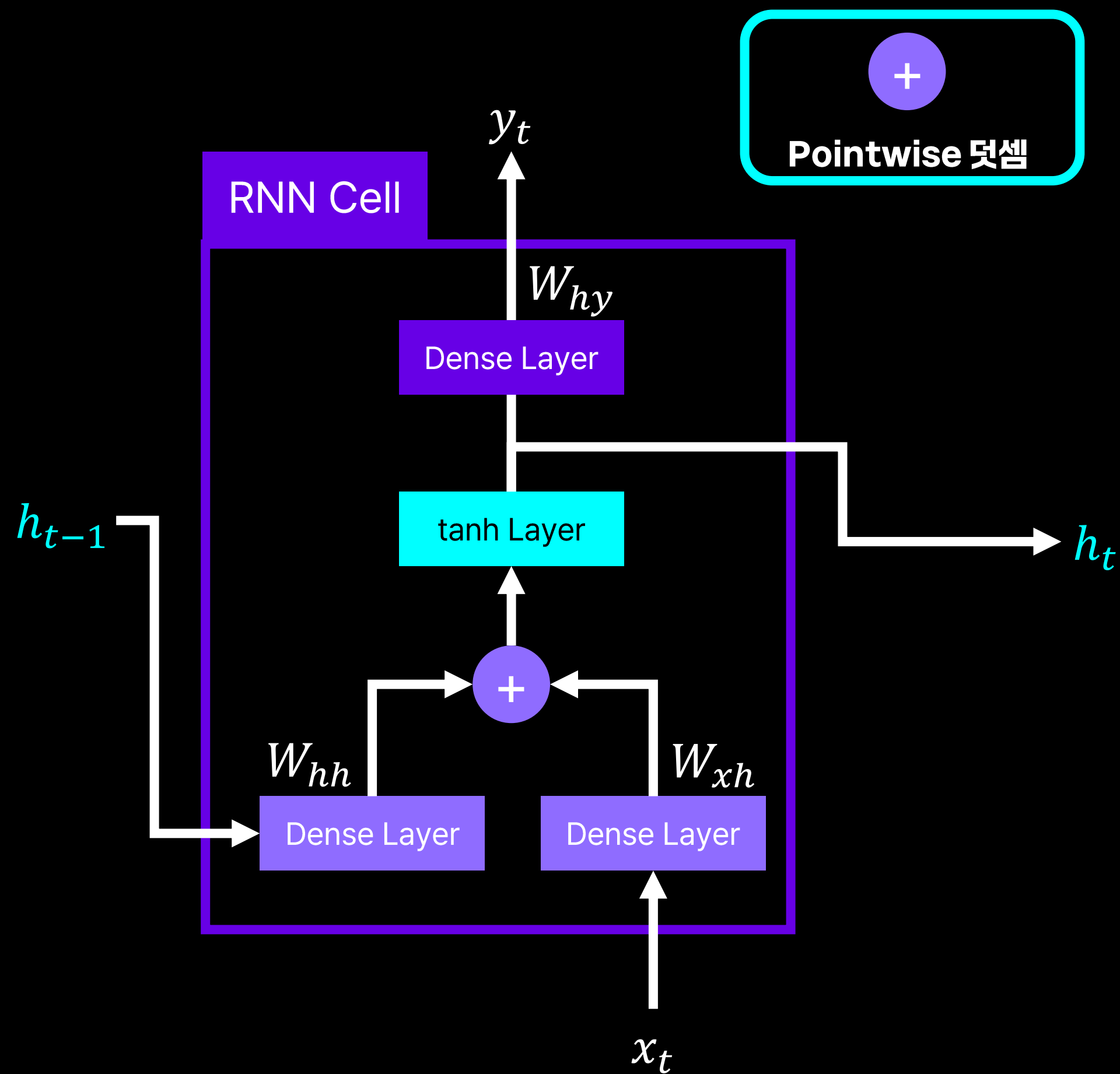
LSTM의 등장

- LSTM(Long Short-Term Memory)
 - 기존의 RNN이 가진 **장기 의존성 문제**를 해결하기 위해 고안된 신경망
 - RNN에 장기 기억력을 부여한 Memory network
- **Cell state**를 통한 장기 기억 보관
 - 세 가지 Gate를 통한 장기 기억 관리
 - Cell state에 포함시키지 않을 정보를 Gate로 차단





RNN Cell의 은닉 상태

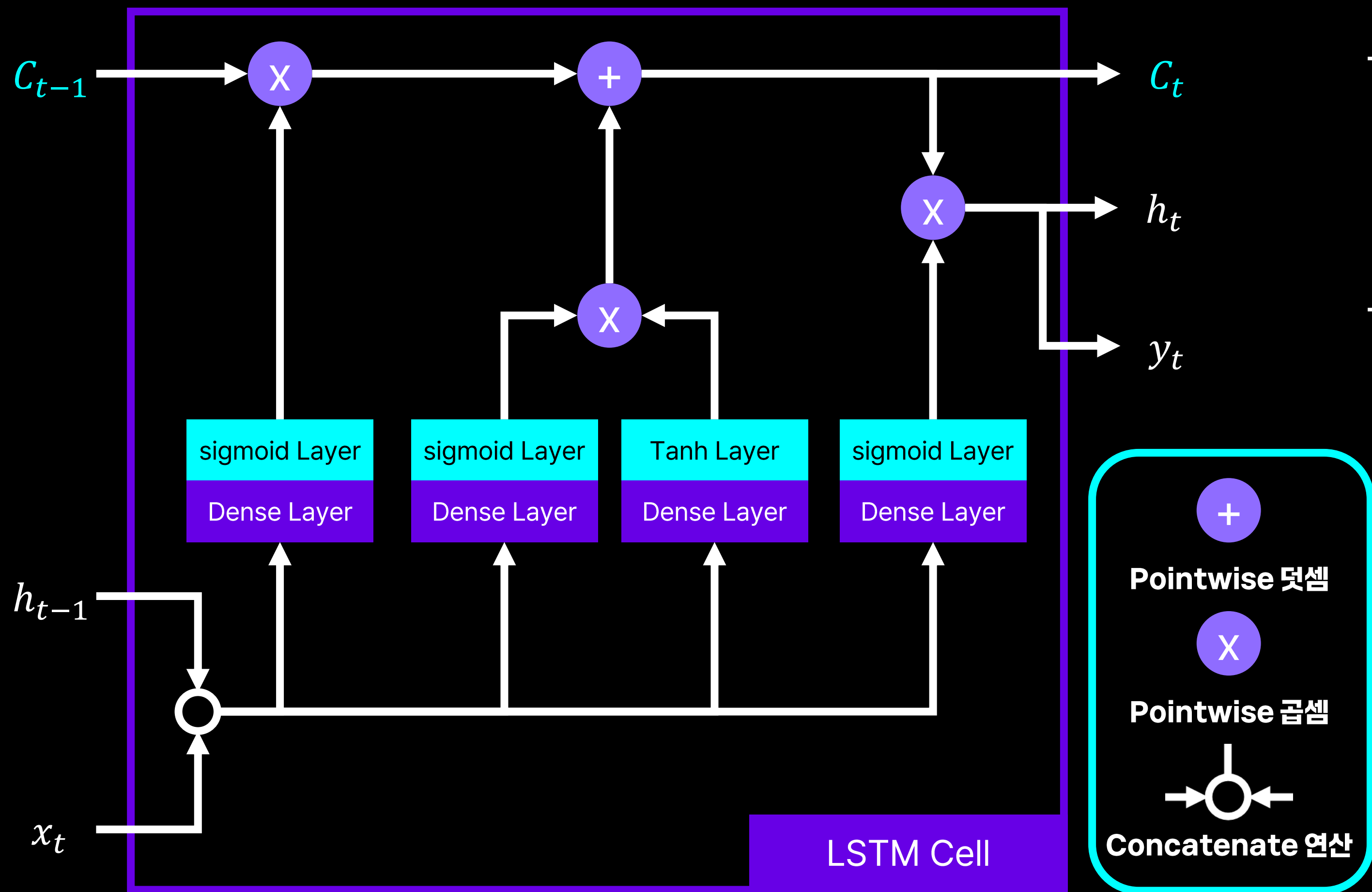


은닉 상태(Hidden State)

- 은닉 상태를 구성할 때는 이전의 은닉 상태의 값과 현재 입력 값을 단순 합산한 후 tanh Layer를 통해 $[0, 1]$ 범위로 정규화 한다.
- 이로 인해 Cell을 반복할 수록 초기 데이터가 희석된다.
- 장기 의존성 정보를 저장할 방법이 필요하다.



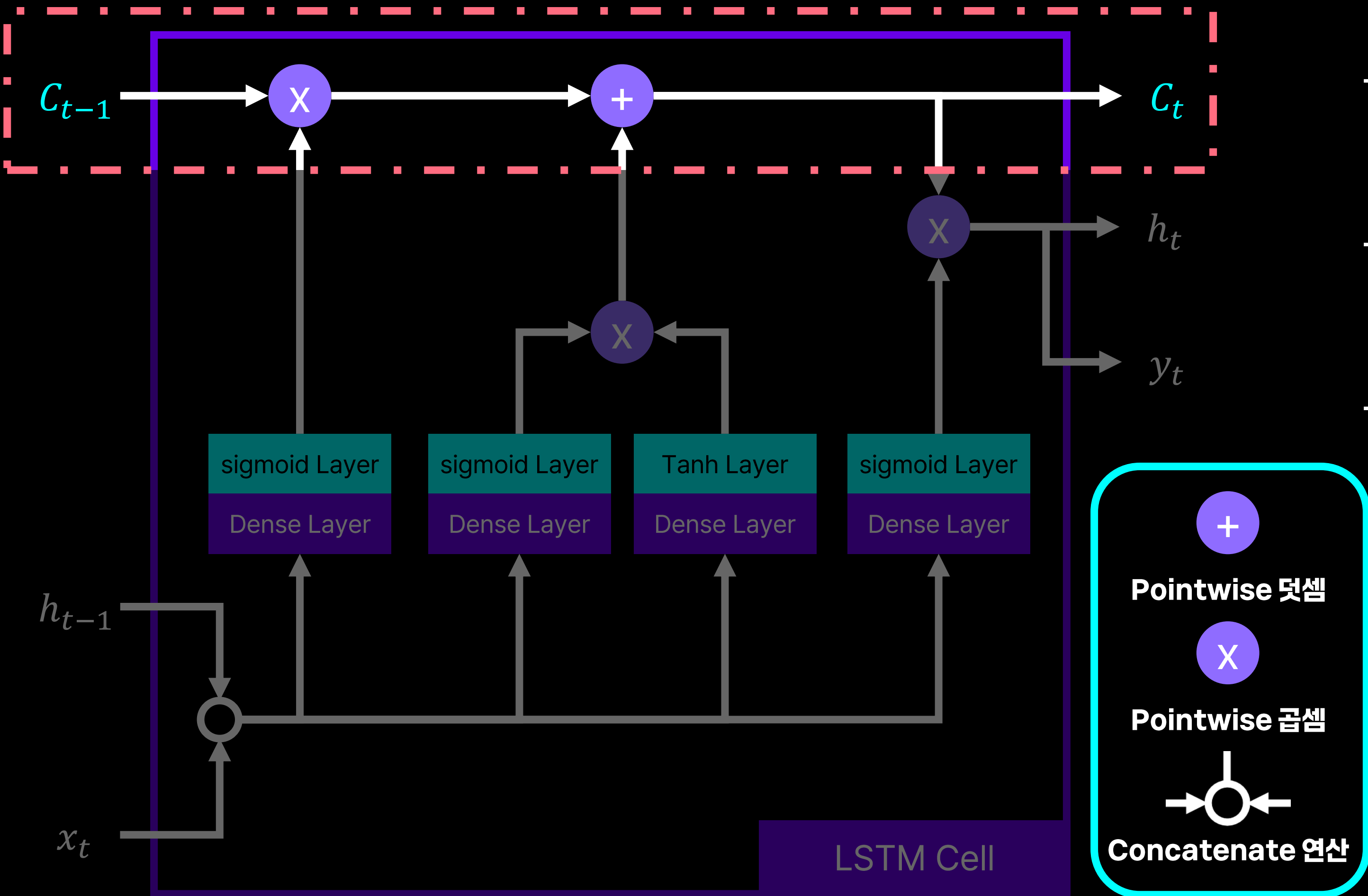
LSTM Cell



- 장기 의존성 문제와 기울기 소실 문제를 해결하기 위한 아키텍처
- Long Short-Term Memory(장단기 메모리)의 약자

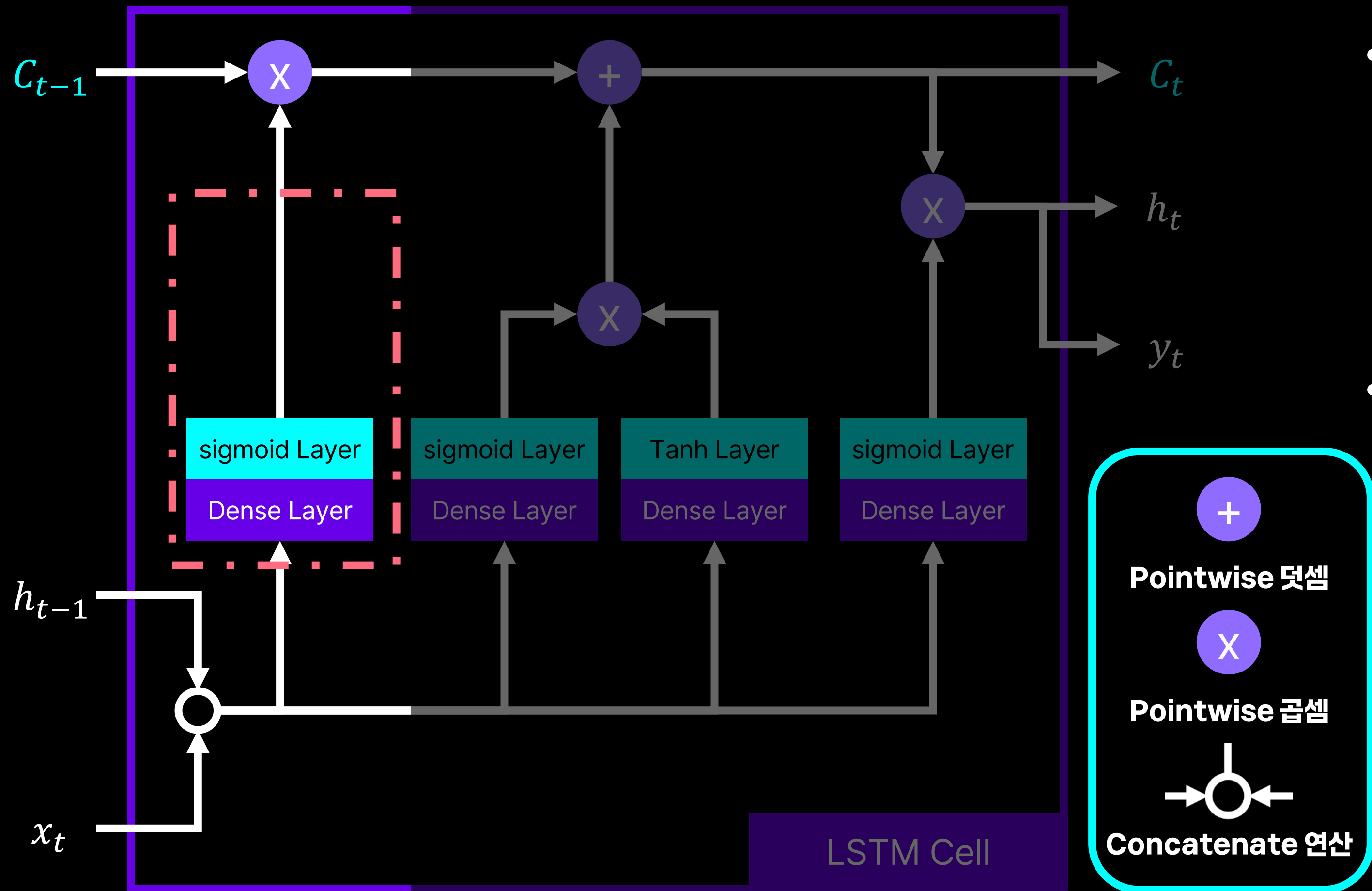


Cell State



- 기울기 소실 문제를 해결하기 위한 요소
- 장기적으로 기억할 정보를 **선택적으로** 저장
- 3개의 **Gate 구조**를 통해 정보를 **선택적으로 갱신**
 1. 망각 게이트
 2. 입력 게이트
 3. 출력 게이트

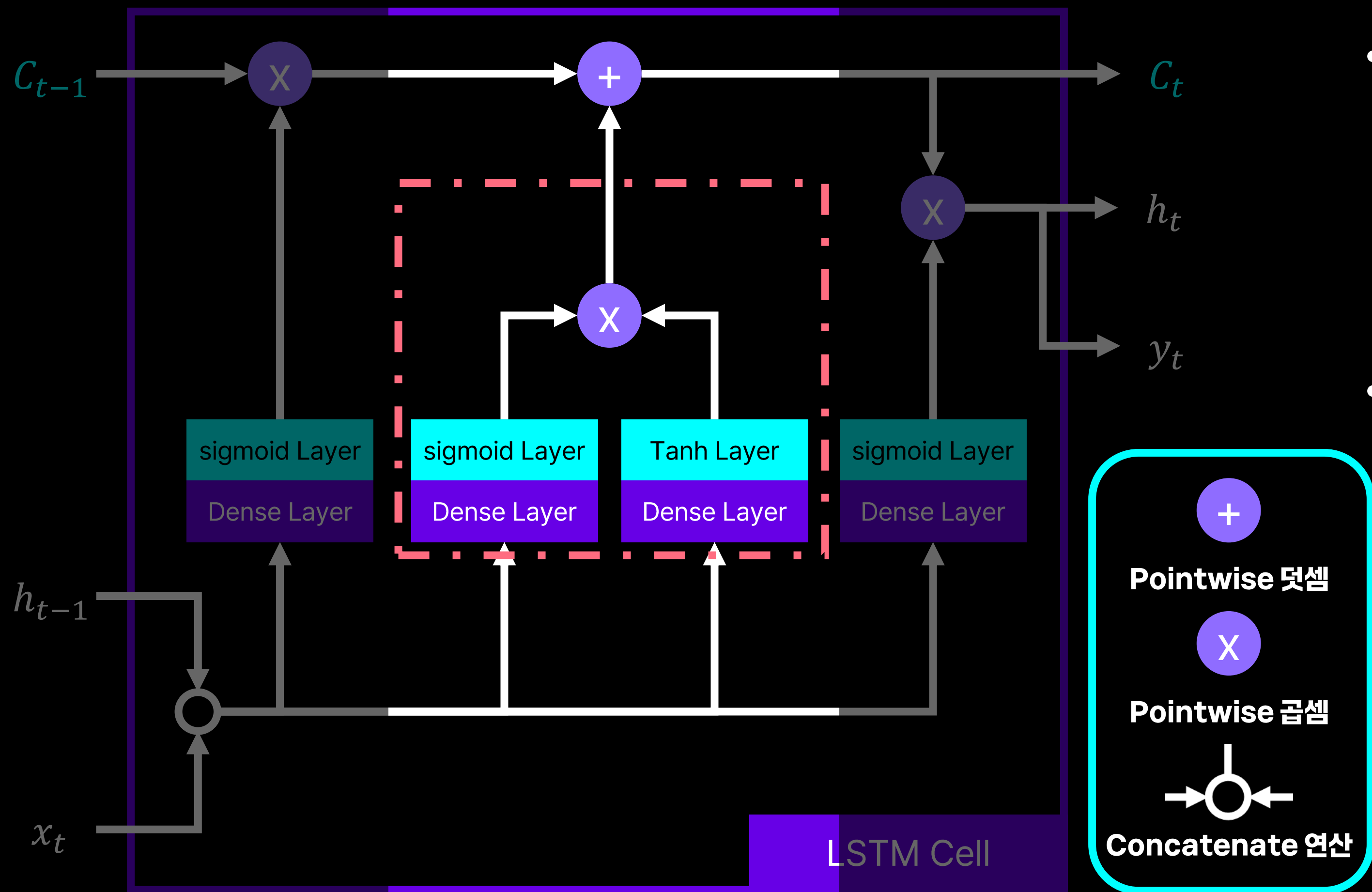
망각 게이트 (Forget Gate)



- 기존 Cell State C_{t-1} 에서 어떤 정보를 잊을 지 결정하는 게이트
- 은닉 상태 h_{t-1} 과 입력 값 x_t 를 활용해서 Cell State의 각 원소를 어느 정도로 잊을 지 계산한다.

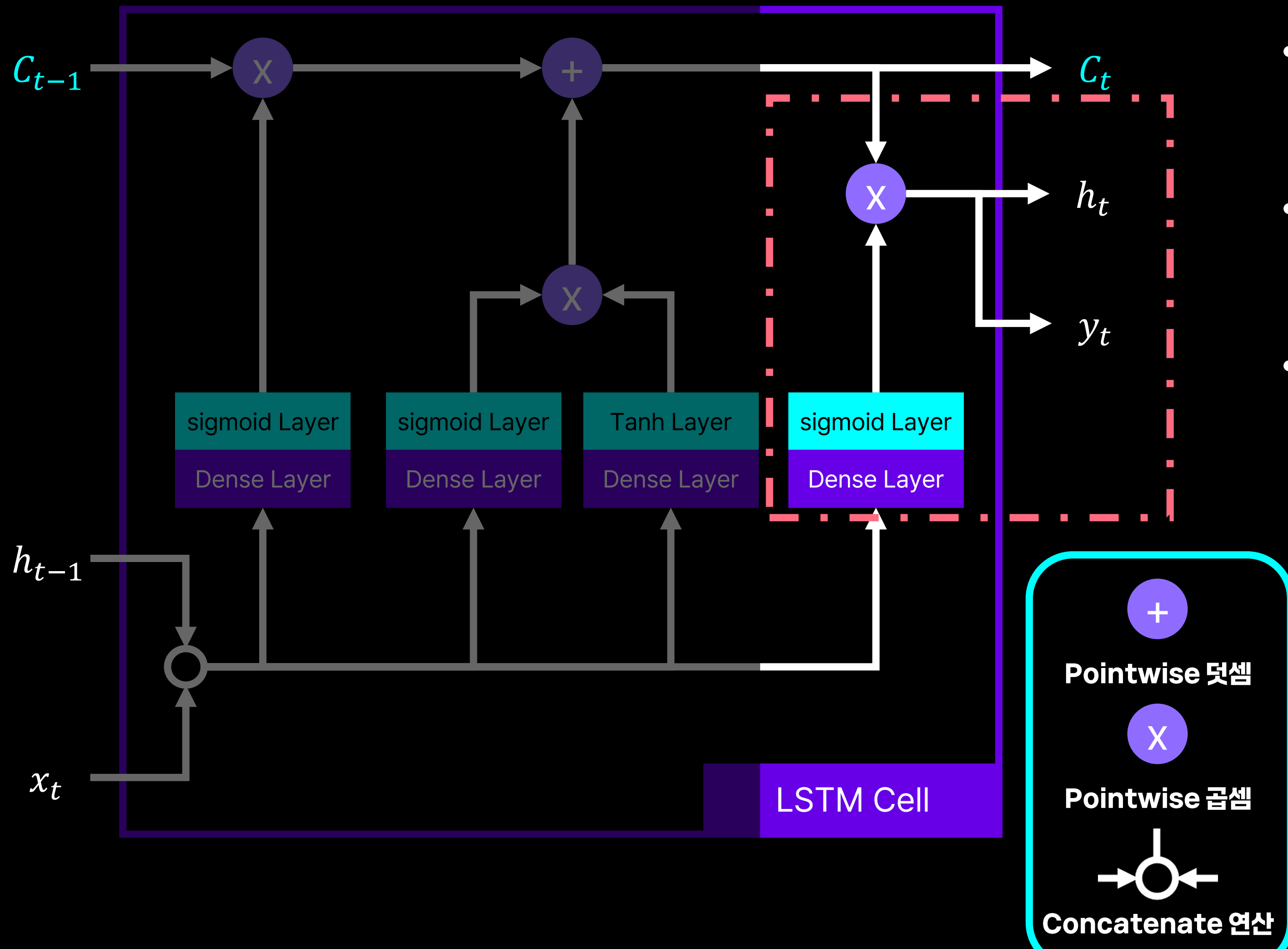


입력 게이트 (Input Gate)



- 현재 입력 받은 정보에서 어떤 정보를 Cell State C_t 에 저장할지 결정하는 게이트
- 은닉 상태 h_{t-1} 과 입력 값 x_t 를 바탕으로 기억할 정보를 Cell State에 추가한다.

출력 게이트 (Output Gate)



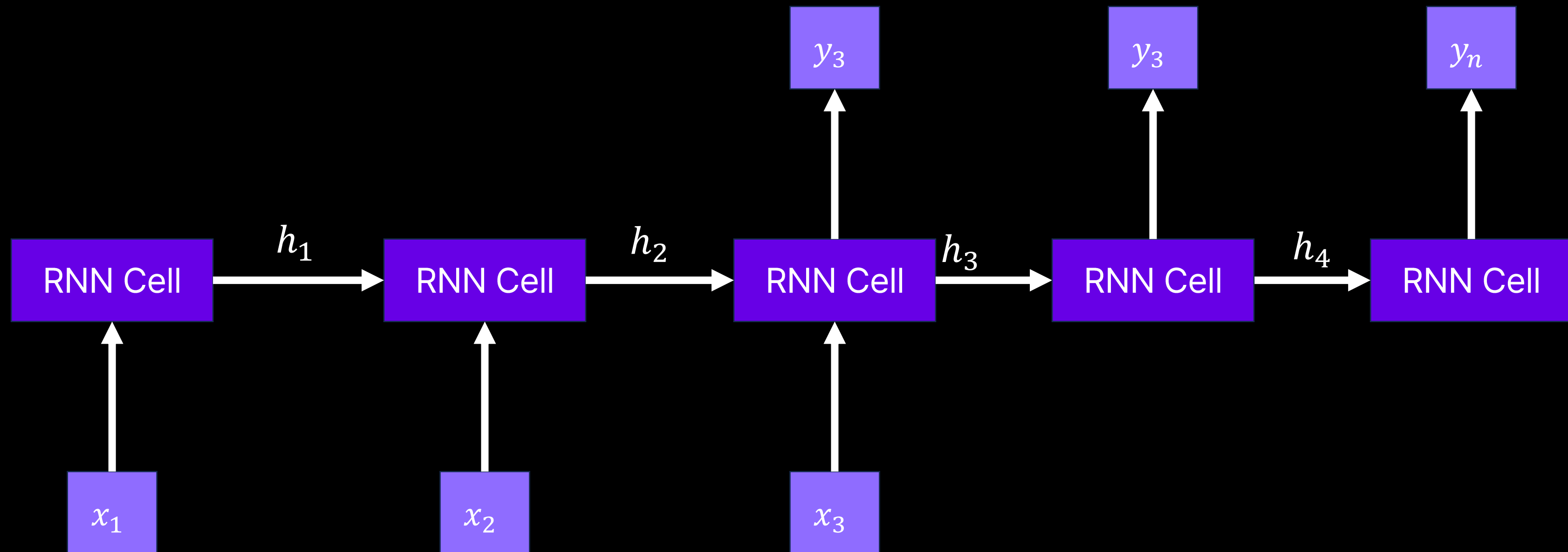
- 다음 은닉 상태 h_t 와 출력 값 y_t 을 계산하는 게이트.
- ($h_t = y_t$)
- 갱신된 Cell State c_t 를 활용한다.

2. Seq2seq와 Attention



RNN 계열 알고리즘의 한계

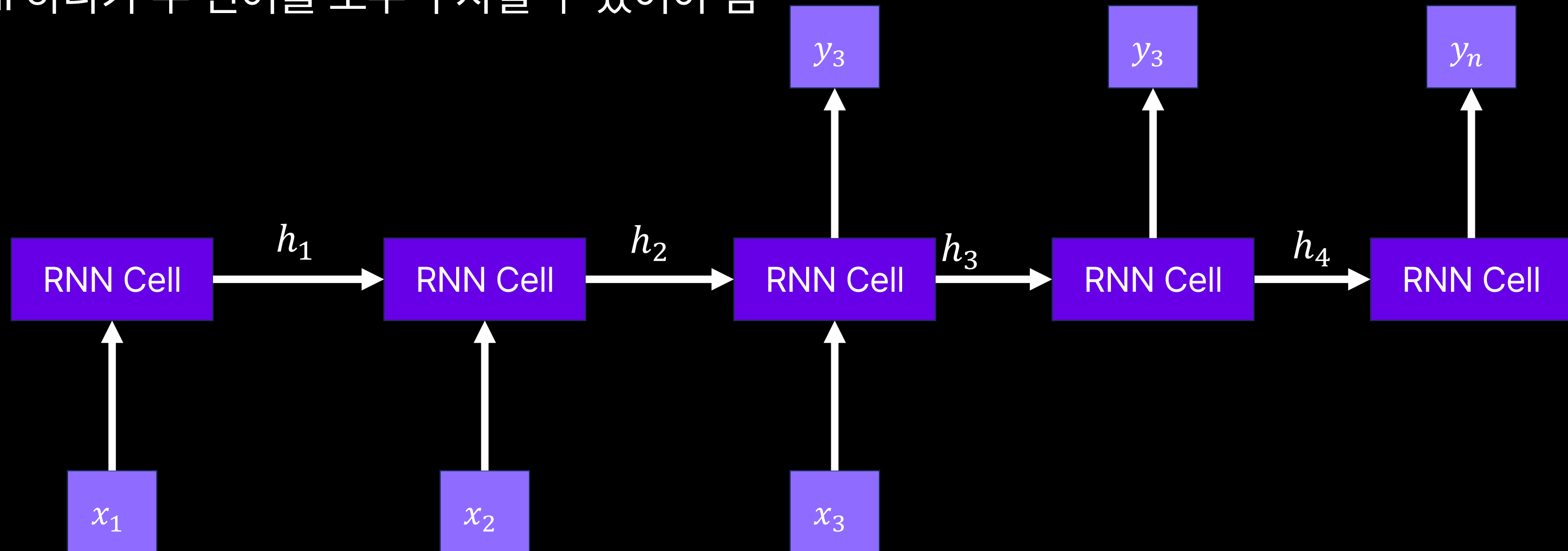
- 기계번역(NMT, Neural Machine Translation)
 - 신경망을 활용하여 대상 문장의 언어(Source language)를 목적 언어(Target language)로 바꾸는 작업



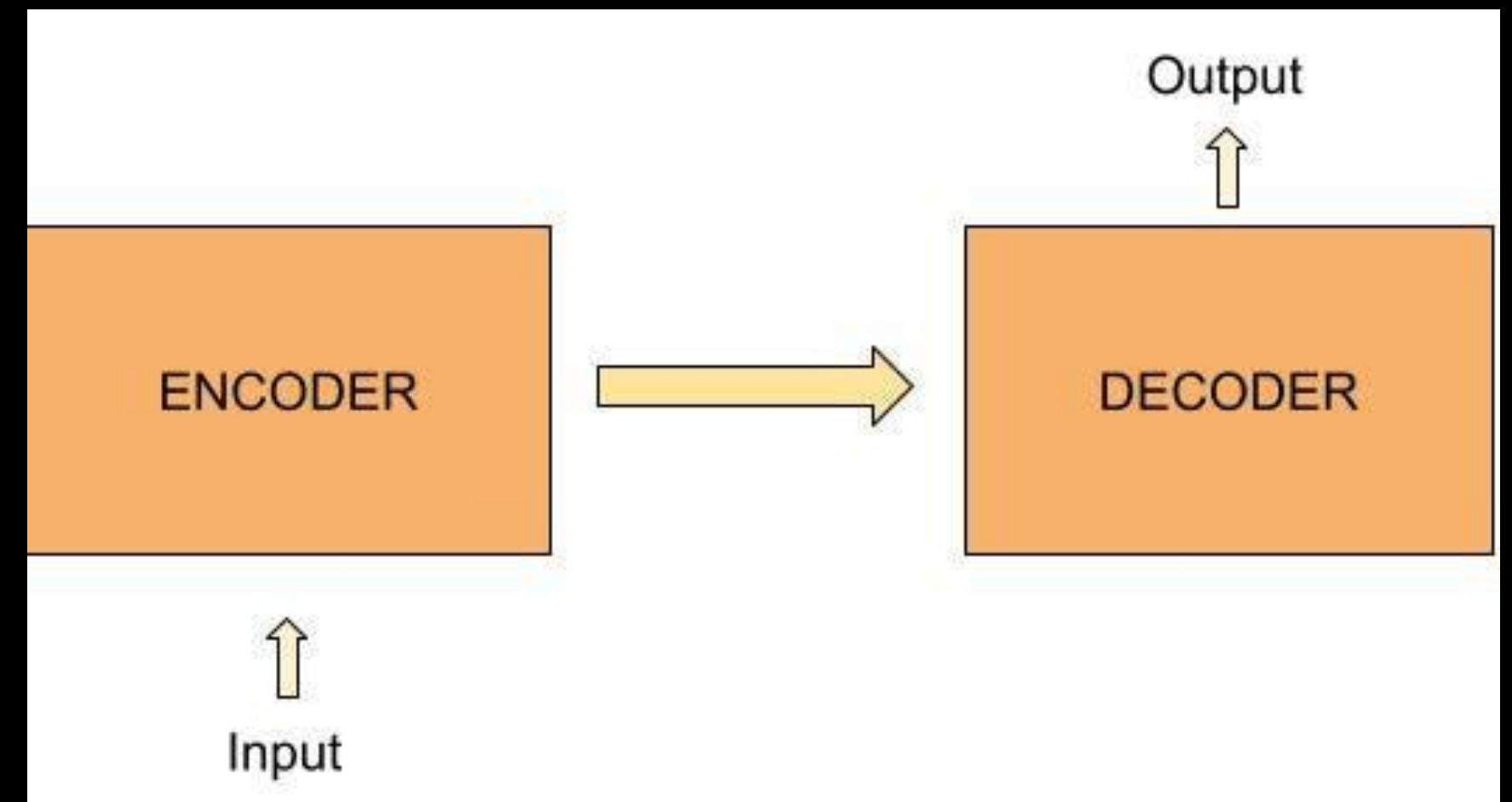


RNN 계열 알고리즘의 한계

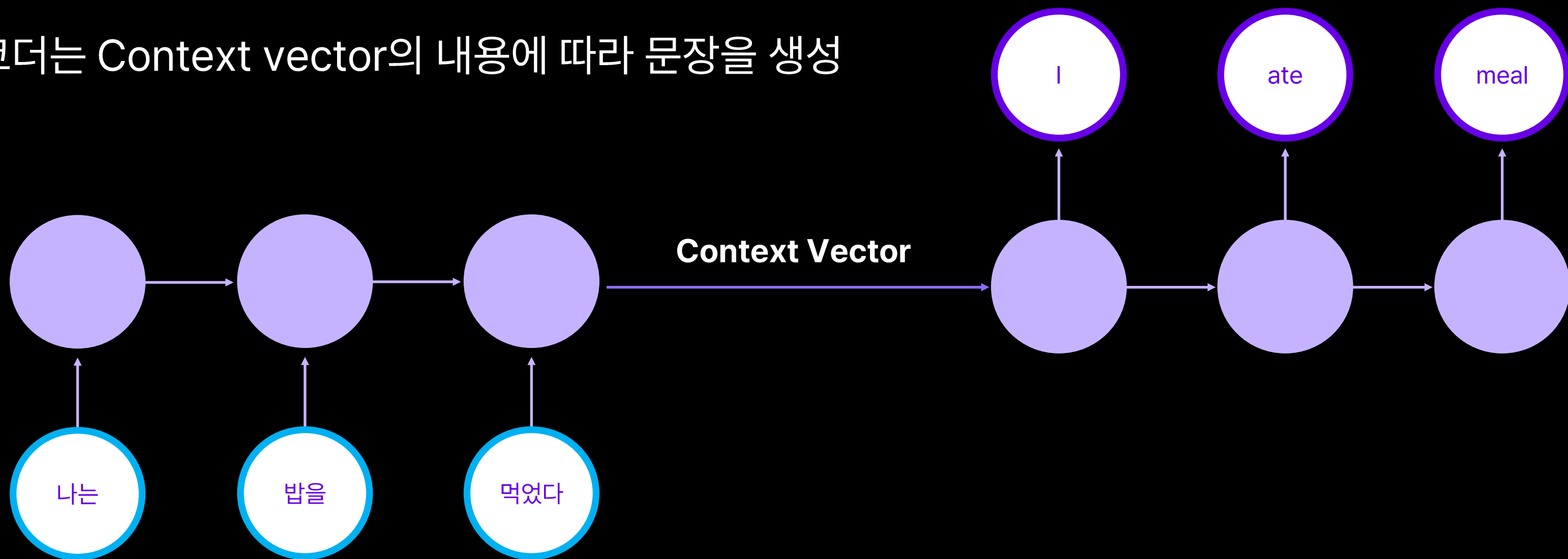
- RNN 계열 알고리즘의 문제
 - 마지막 단어 의존도가 지나치게 높음
 - 기억 손실로 인하여 디테일한 정보를 번역하지 못함
 - Cell 하나가 두 언어를 모두 구사할 수 있어야 함



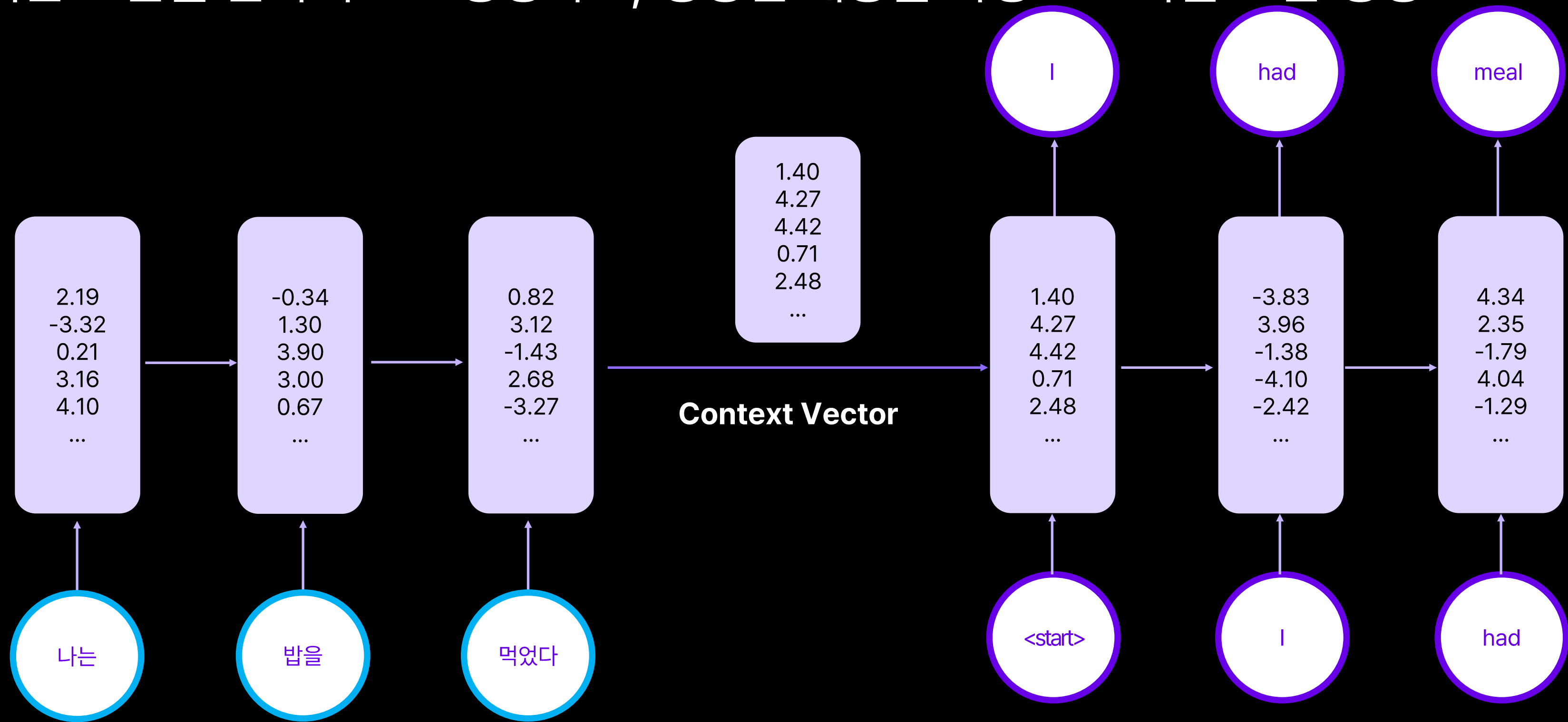
- Seq2Seq(Sequence to Sequence)의 기본 아이디어
 - 두 가지 언어를 모두 잘 구사하기 위한 모델 필요
 - 입력과 출력의 기능을 **구조적으로 분리**
- 두 개의 RNN 구조 사용: 인코더(Encoder)와 디코더(Decoder)
 - 인코더에서는 Source sentence 이해만을 담당
 - 디코더에서는 Target sentence 생성만을 담당



- Seq2Seq(Sequence to Sequence)의 기본 아이디어
 - 입력 시퀀스를 고정된 크기의 벡터로 변환
 - Context vector
 - 인코더가 이해한 내용을 숫자로 요약하여 만든 행렬
 - 디코더는 Context vector의 내용에 따라 문장을 생성



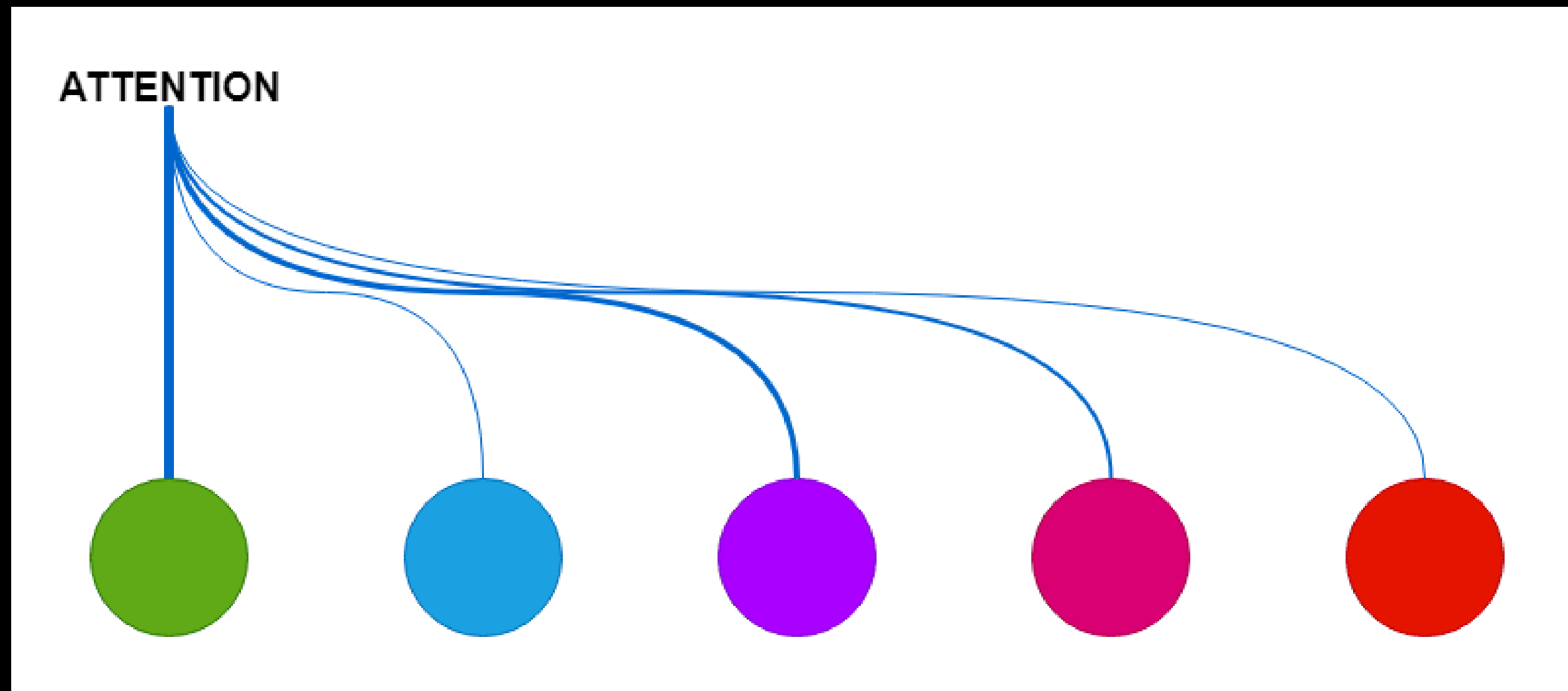
- 인코더는 토큰을 순차적으로 입력받고, 의미를 종합
- 디코더는 토큰을 순차적으로 생성하고, 생성된 내용을 바탕으로 다음 토큰 생성



- Context Vector의 한계
 - Encoder에서 단어 간 영향력의 차이를 구분하여 전달하기 힘들
 - 마지막 단어의 영향력이 강하고 앞 단어 중 디테일한 내용들이 반영될 수는 없음
 - 가령, "마트에 가서 오렌지 두 개를 2000원에 사와"라는 문장을 번역할 시, "**사와**"에 포커스를 맞추고
 - "**두 개**", "**2000원**" 등 구체적인 내용이 잘 번역되지 않을 수 있음
 - 하나의 벡터로 문맥을 요약할 수는 없는 문장도 존재
 - 길이가 길거나, 흐름이 자주 바뀌는 문장
 - 문학적인 표현
 - 생성된 문장이 모두 동일한 Context를 참조한다면 복잡한 문장을 만들 수 없음

☰ Attention Mechanism의 기본 아이디어

- 의미 관점에서 입력 시퀀스의 모든 토큰이 동일하게 중요하지 않음
- 디코더가 출력을 생성할 때 입력 시퀀스의 특정 부분에 "주목"한다면 디테일한 변환 가능
- 가중치(Attention weight)를 통해 중요한 부분에 더 많은 주의를 기울임



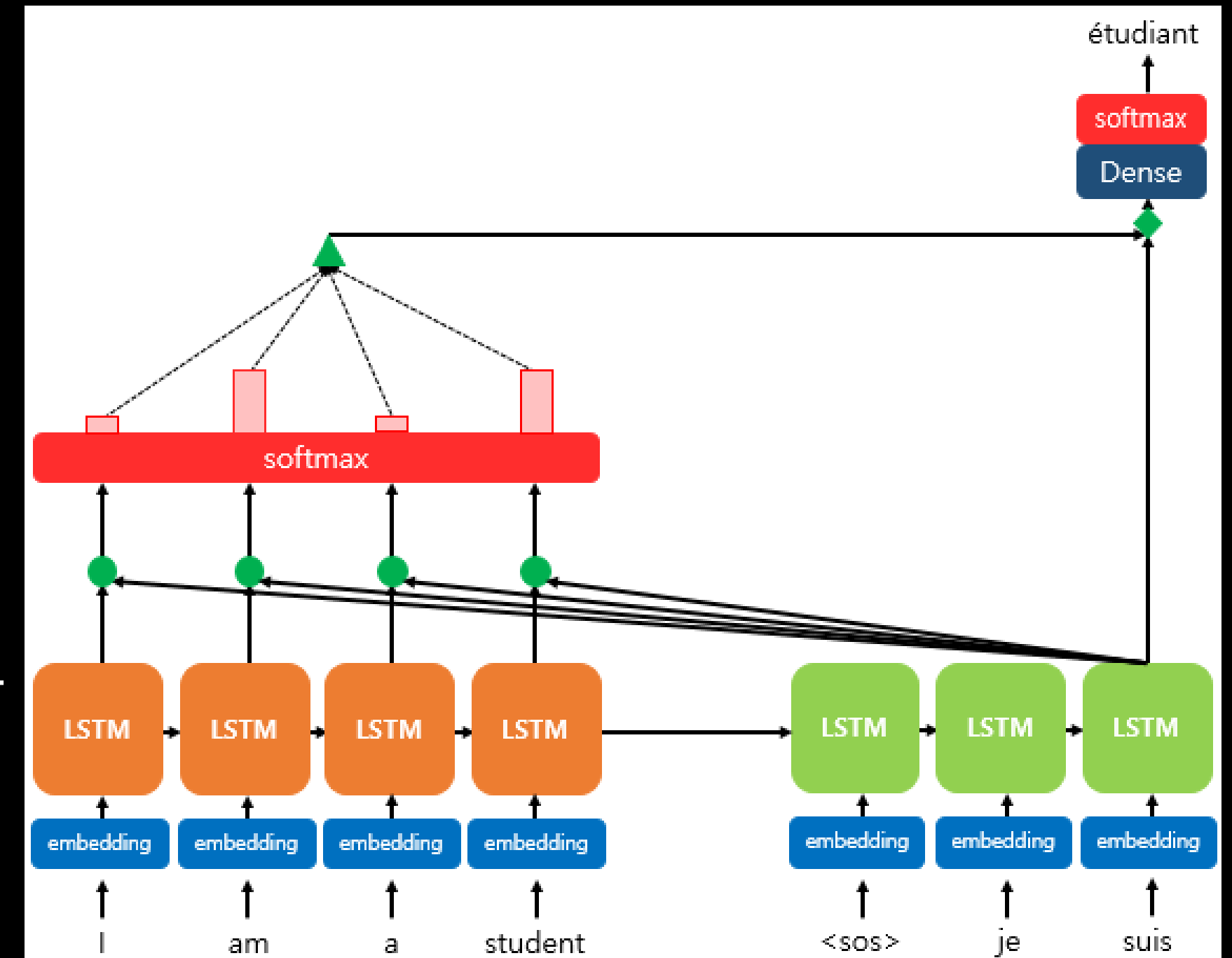
☰ Attention Mechanism의 기본 아이디어

- Python 자료형 중 Dictionary에서는 데이터를 짝지어 저장
 - Key - Value
 - **Key**: 데이터를 호출하는 인덱스
 - **Value**: 호출한 인덱스의 실제 값
- SQL에서는 DB에 필요한 정보를 요청
 - **Query**: 정보나 결과를 얻기 위해 시스템에 보내는 질문, 요청

Keys	Values
'name'	'Bob'
'age'	25
'job'	'Dev'
'city'	'New York'
'email'	'bob@web.com'

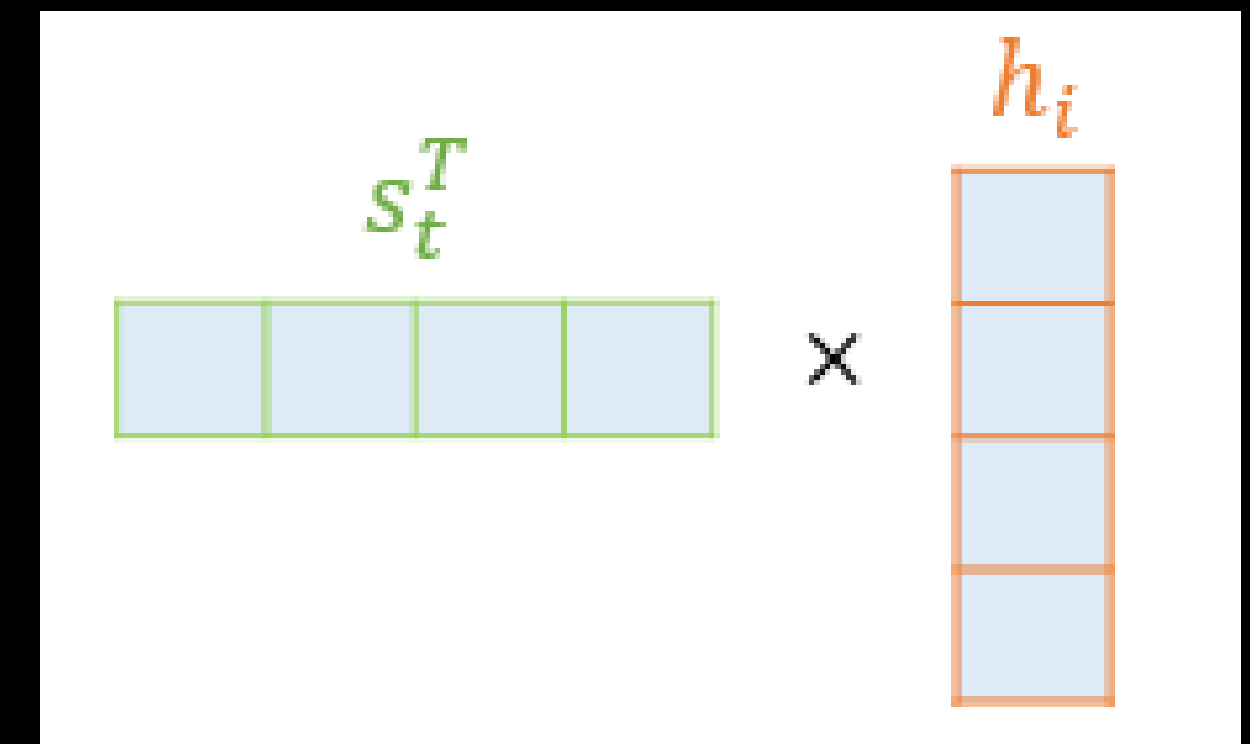
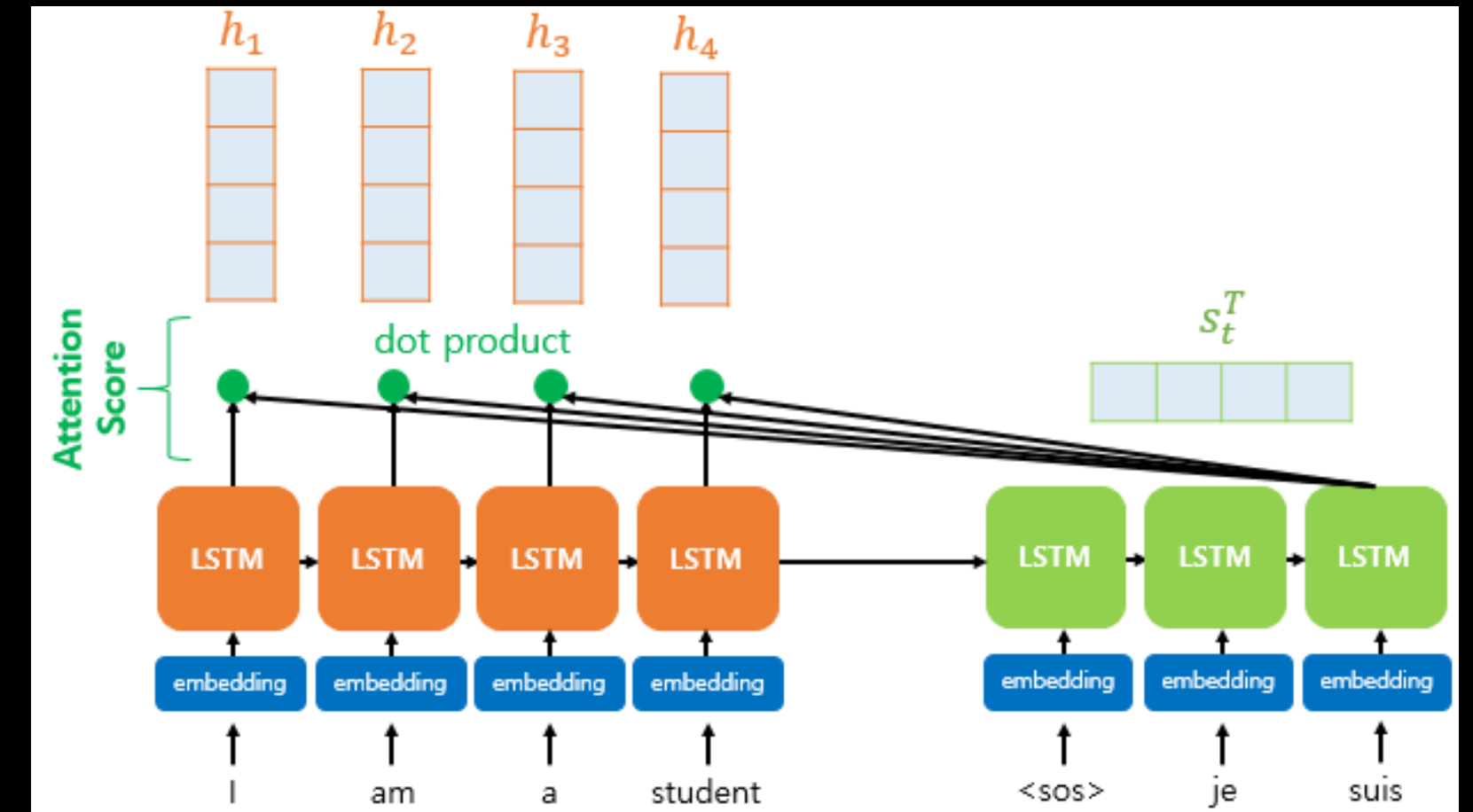
☰ Attention Mechanism의 작동 원리

- 인코더는 입력 문장의 단어를 읽고 내용을 계산
- 각 입력 단어에 대한 Hidden state(**Key**)가 생성됨
- 디코더는 문장을 구성하는 단어(**Query**) 생성 시,
입력된 여러 단어 Hidden state의 중요도(**Value**)를
참고함



≡ Attention Mechanism의 작동 원리

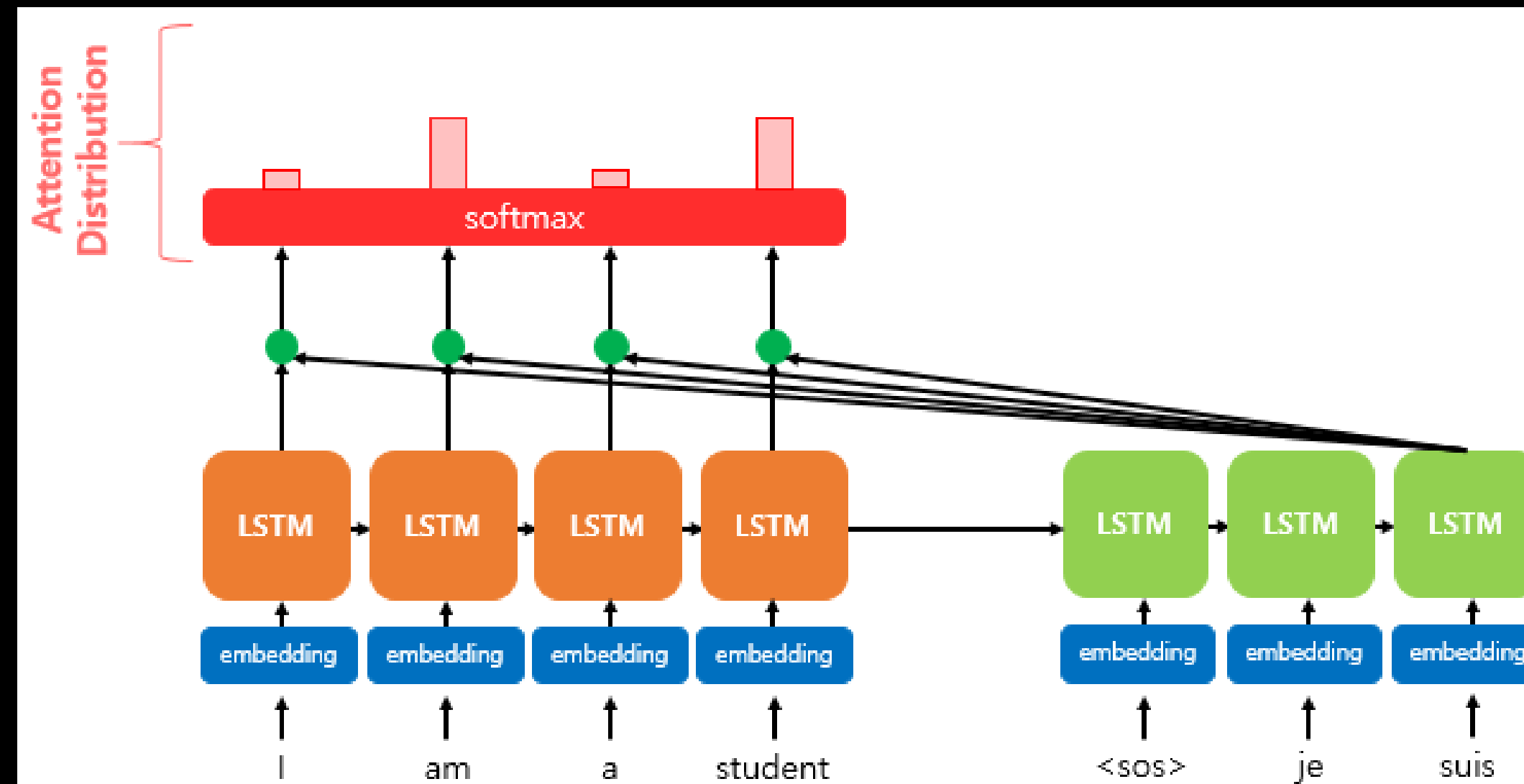
- 디코더에서 단어를 생성할 때, 인코더의 모든 Hidden state에 대해 각 요소가 디코더의 현재 Hidden state와 얼마나 유사한지를 측정한 점수 값
- 주로 벡터의 외적을 통해 계산하며, 코사인 유사도 등 다른 연산을 사용하기도 함
- Query와 Key의 내적 값





Attention Distribution

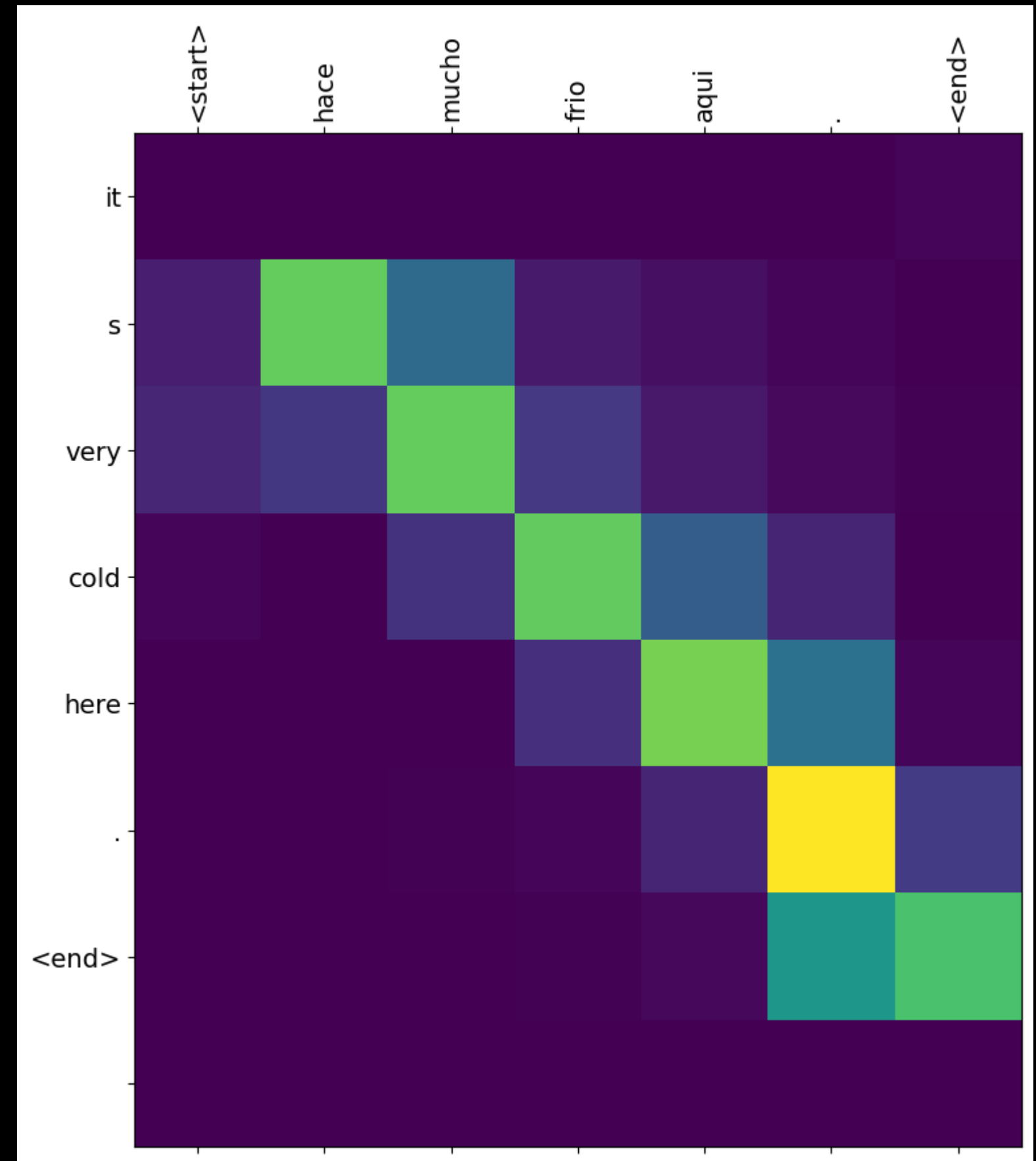
- 앞에서 구한 Attention score를 **확률의 분포**로 변환
- Softmax 활용
- 모든 인코더 Hidden state의 기여도가 **확률 값**으로 반환[0, 1]
- 각 기여도를 **Attention weight**라 함





Attention의 장점

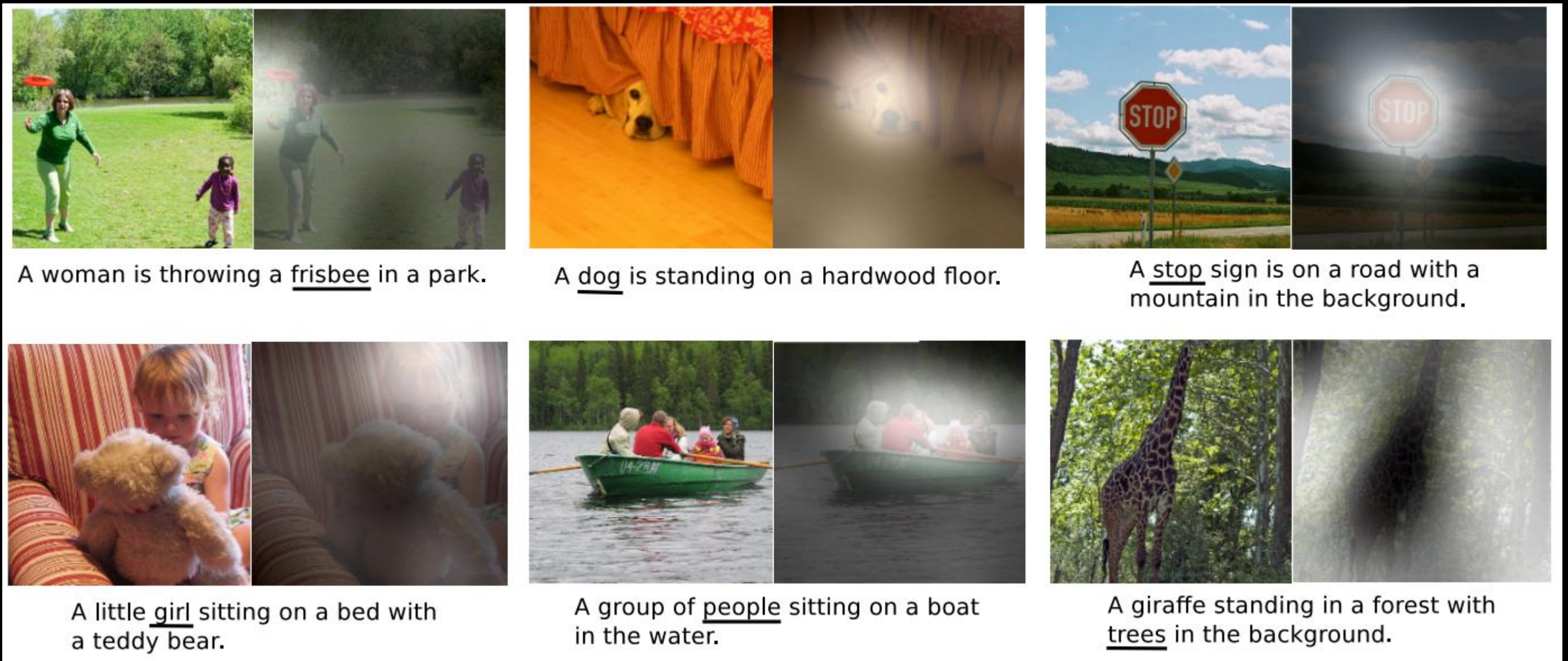
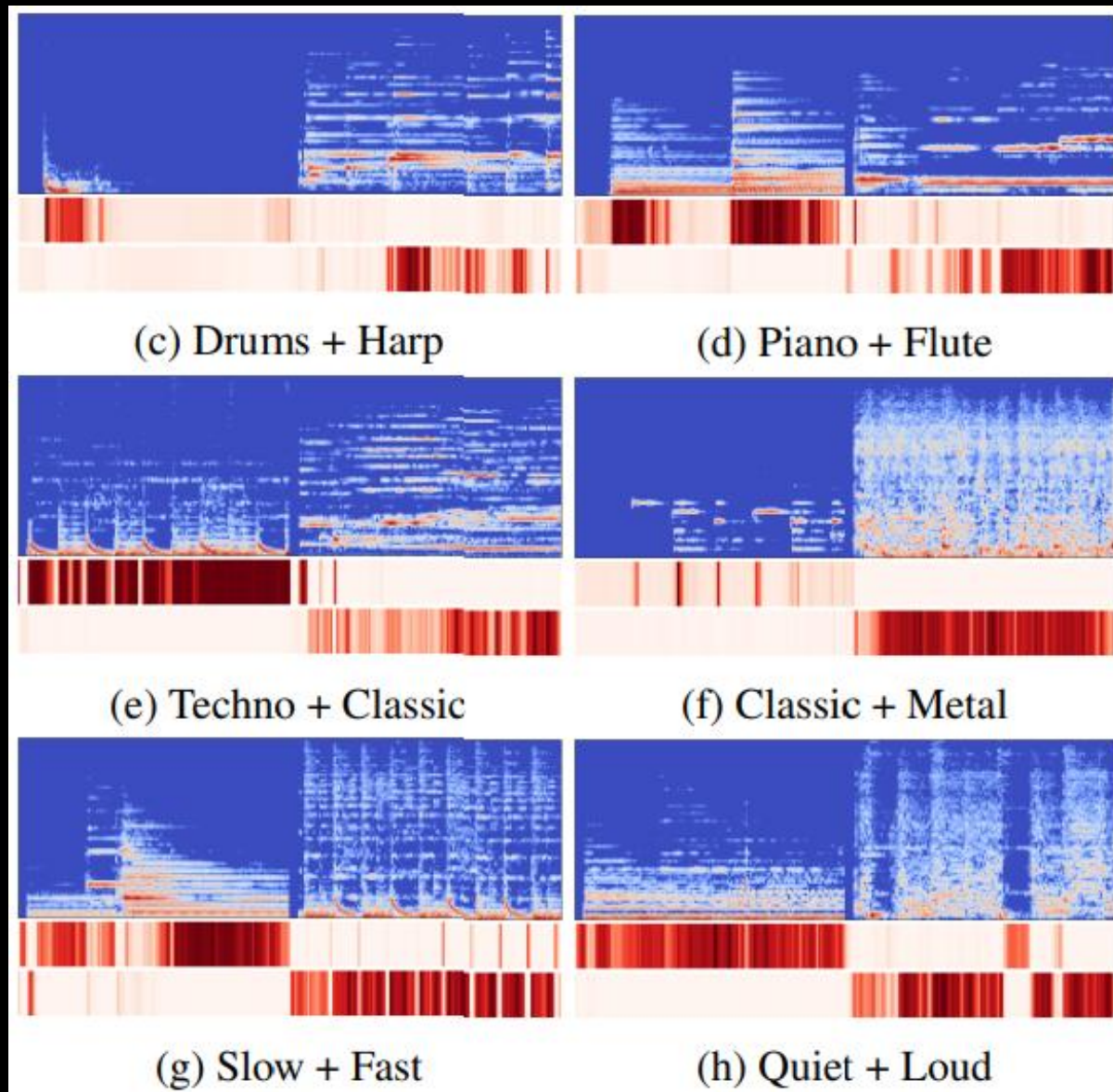
- 번역 성능 향상
 - 정보 손실 최소화: 긴 입력 시퀀스에서도 중요한 정보 유지
 - 디코딩 시 입력의 특정 부분에 집중 가능
- 시각화를 통해 모델의 주목 포인트 확인 가능(xAI)





Attention의 장점

- Attention Mechanism 응용 사례
 - Attention을 이용한 Music tagging & Image captioning



3. Self-Attention 알고리즘

RNN 모델 구조의 선천적 결함

- 입/출력을 순차적으로 처리하므로 병렬처리를 하더라도 **속도가 매우 느림**
- Long-term dependency 문제: 긴 토큰 처리 불가
- Scaling 불가 : 데이터를 많이 사용하고, 모델 크기를 늘려도 성능 개선 없음

하나의 모델은 한 가지 문제만을 해결

- 분류/번역/감성 분석/텍스트 생성
- 번역의 경우 **단방향 번역 모델**만 훈련 가능
- 하나의 모델 사용을 위하여 지나치게 많은 데이터/리소스 요구

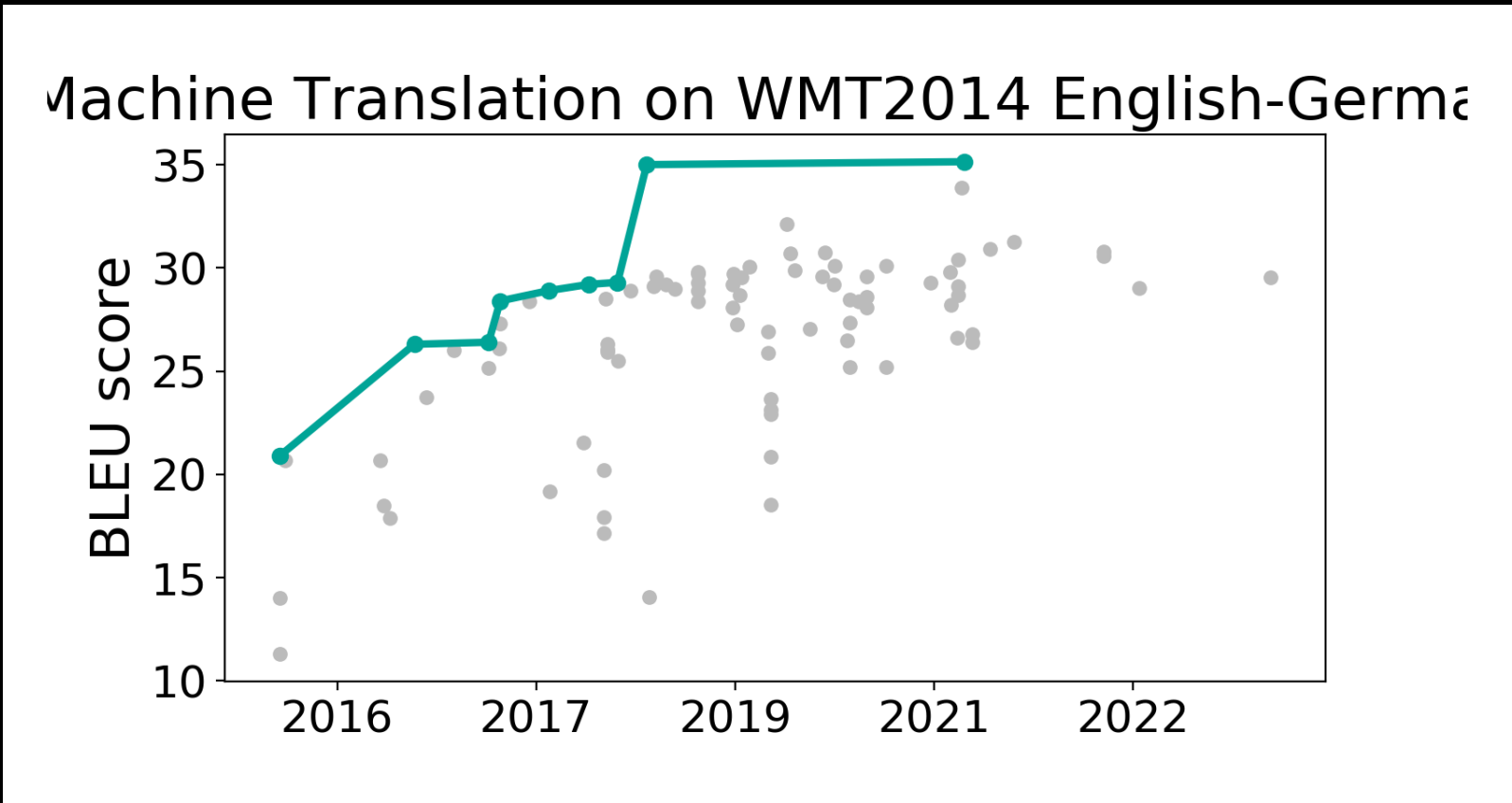


Attention is all you need

Attention is all you need(NIPS, 2017)

...
We propose a new simple network architecture, the **Transformer**, based **solely on attention mechanisms**, dispensing with recurrence and convolutions entirely.

이 논문에서는 RNN과 CNN을 요구하지 않고 **Attention**만을 사용하는, 새롭고 간결한 모델인 **Transformer**을 제시한다.



Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	



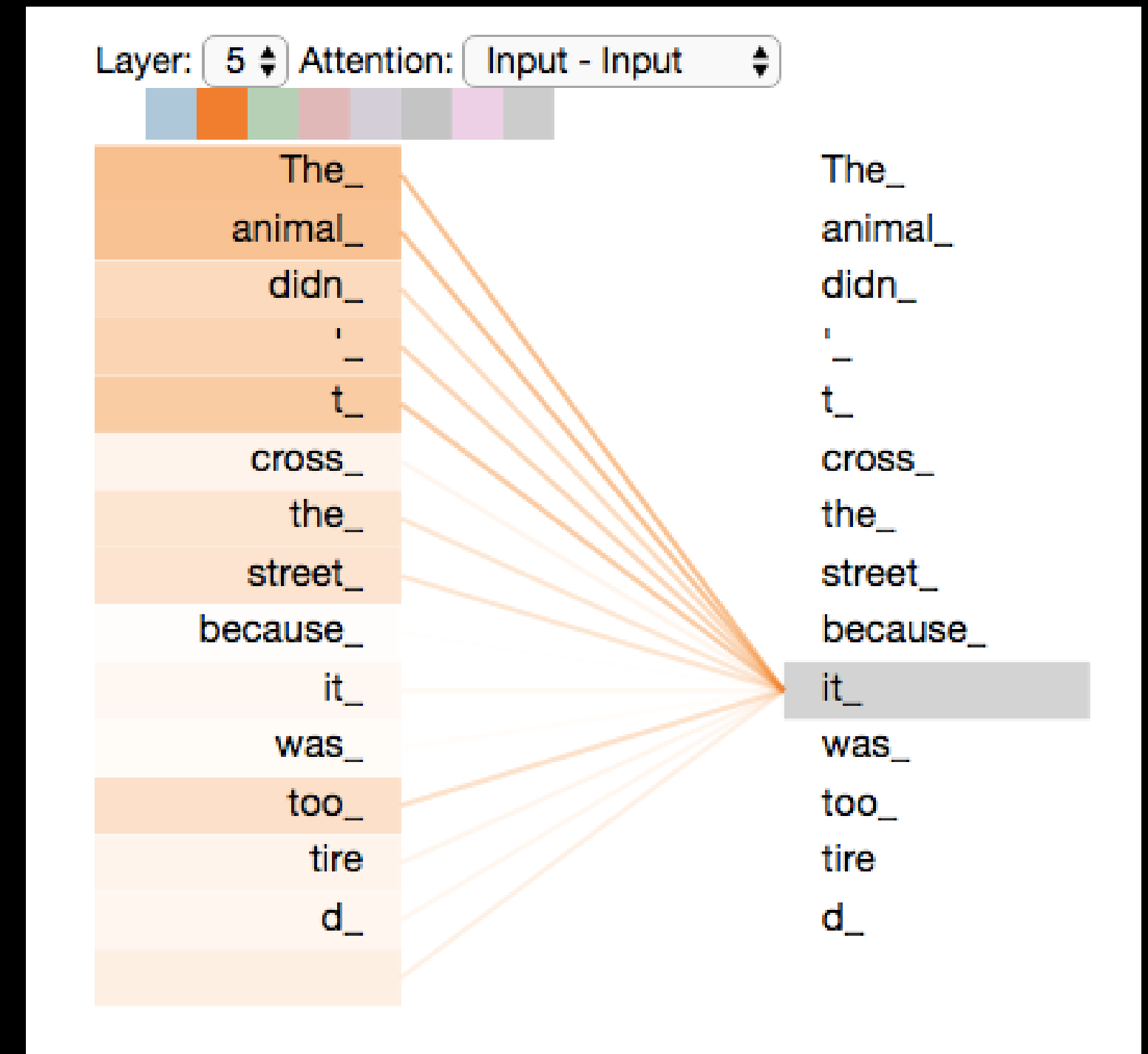
Self-Attention

모델의 발전을 가로막는 요인은 RNN으로 가정

- RNN이 Text에서 Feature을 추출하는 기능 전담
- 그러나 데이터를 읽는 근본적인 과정 개선 필요
- Attention의 연산 방식은 효과를 입증 받음
 - RNN 대신 Attention을 통해 Text feature extraction 시도

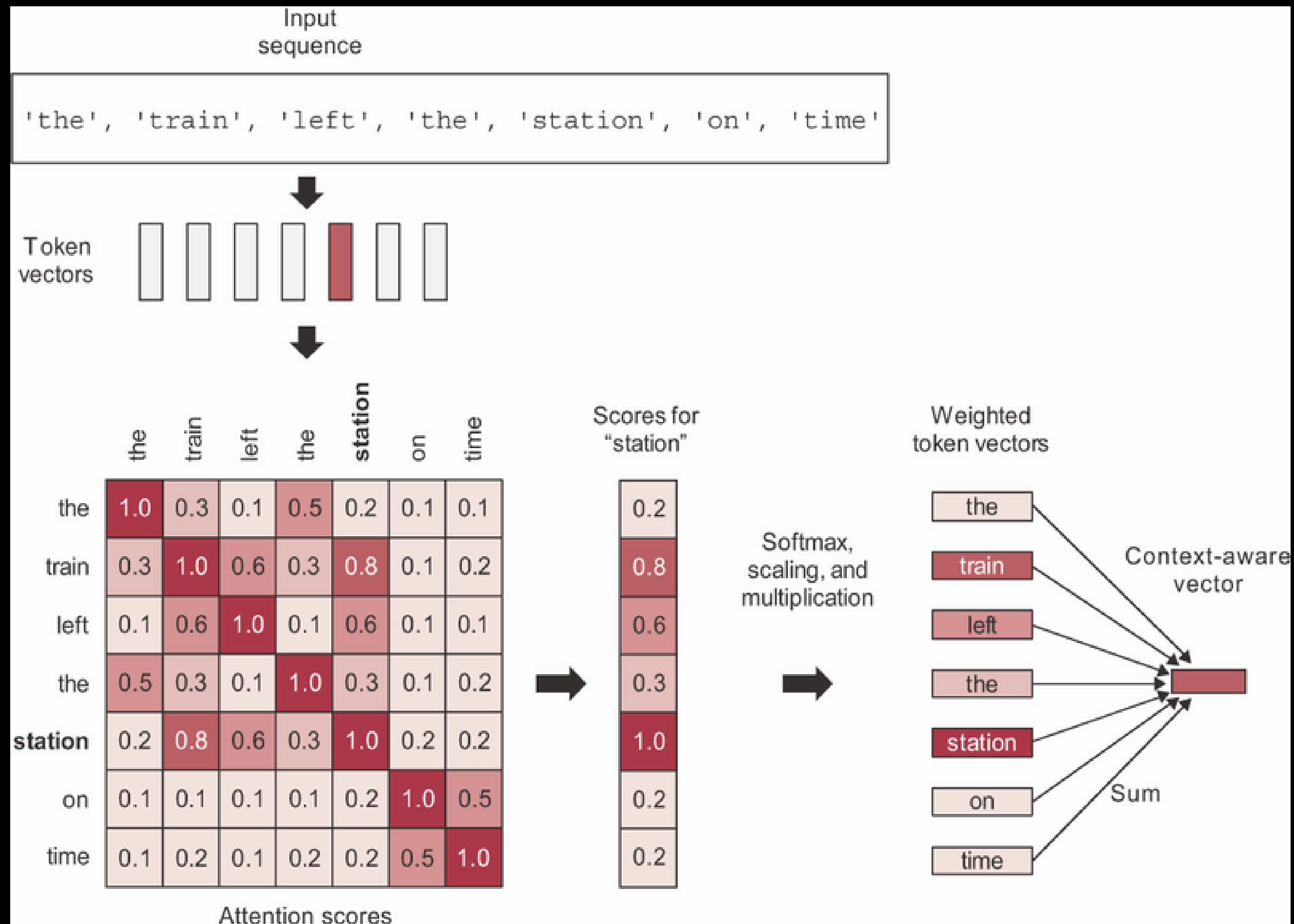
Self-Attention

- Transformer의 핵심 원리
 - 시퀀스 내 모든 토큰 간의 관계 파악
 - Query, Key, Value의 개념을 사용하여 Attention 점수 계산
 - 앞서 배운 Attention 개념과 유사
 - Attention 점수를 병렬적으로 계산
- 각 토큰에 대한 가중치를 계산하여 새로운 표현 생성





Self-Attention



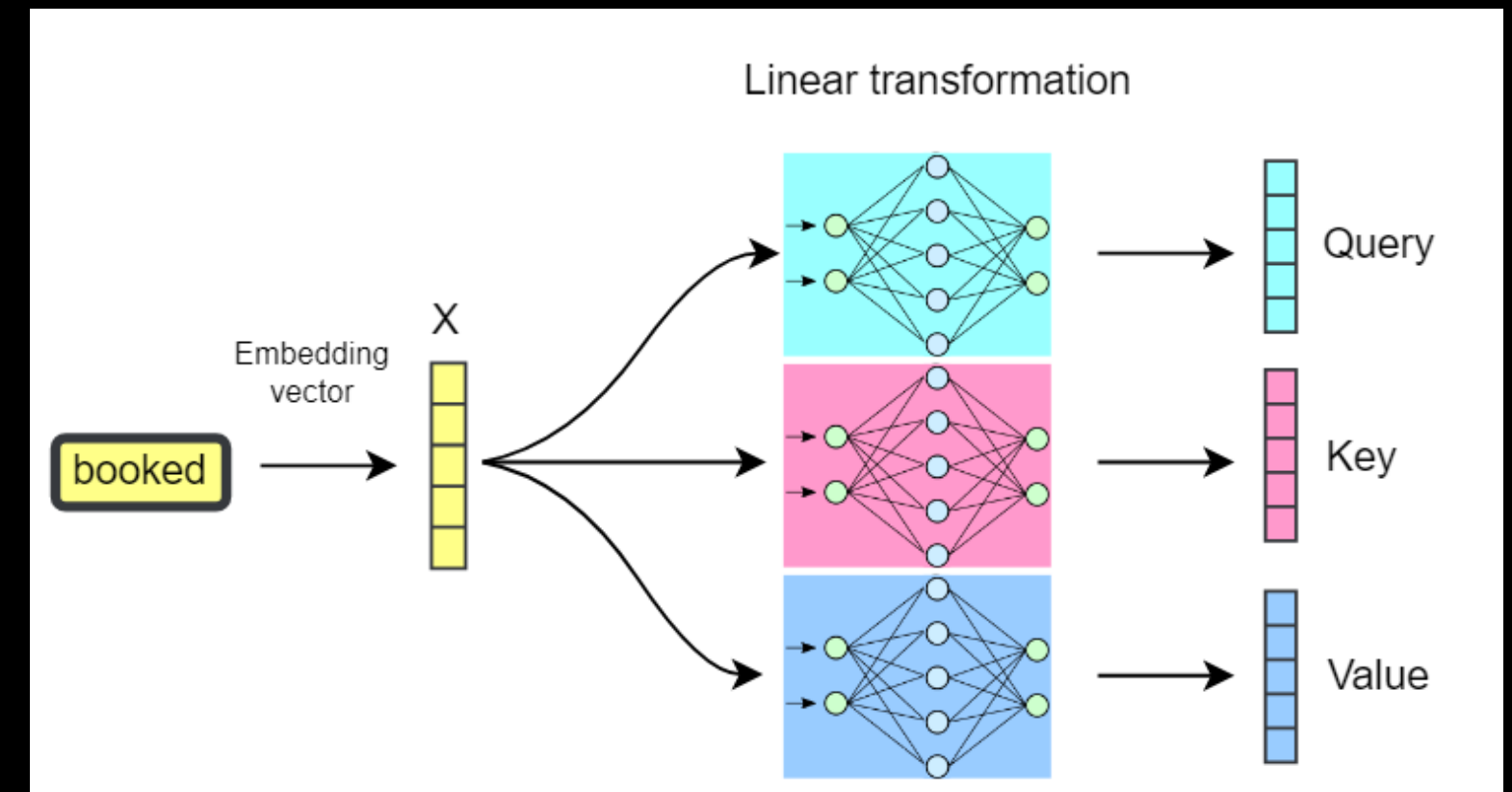
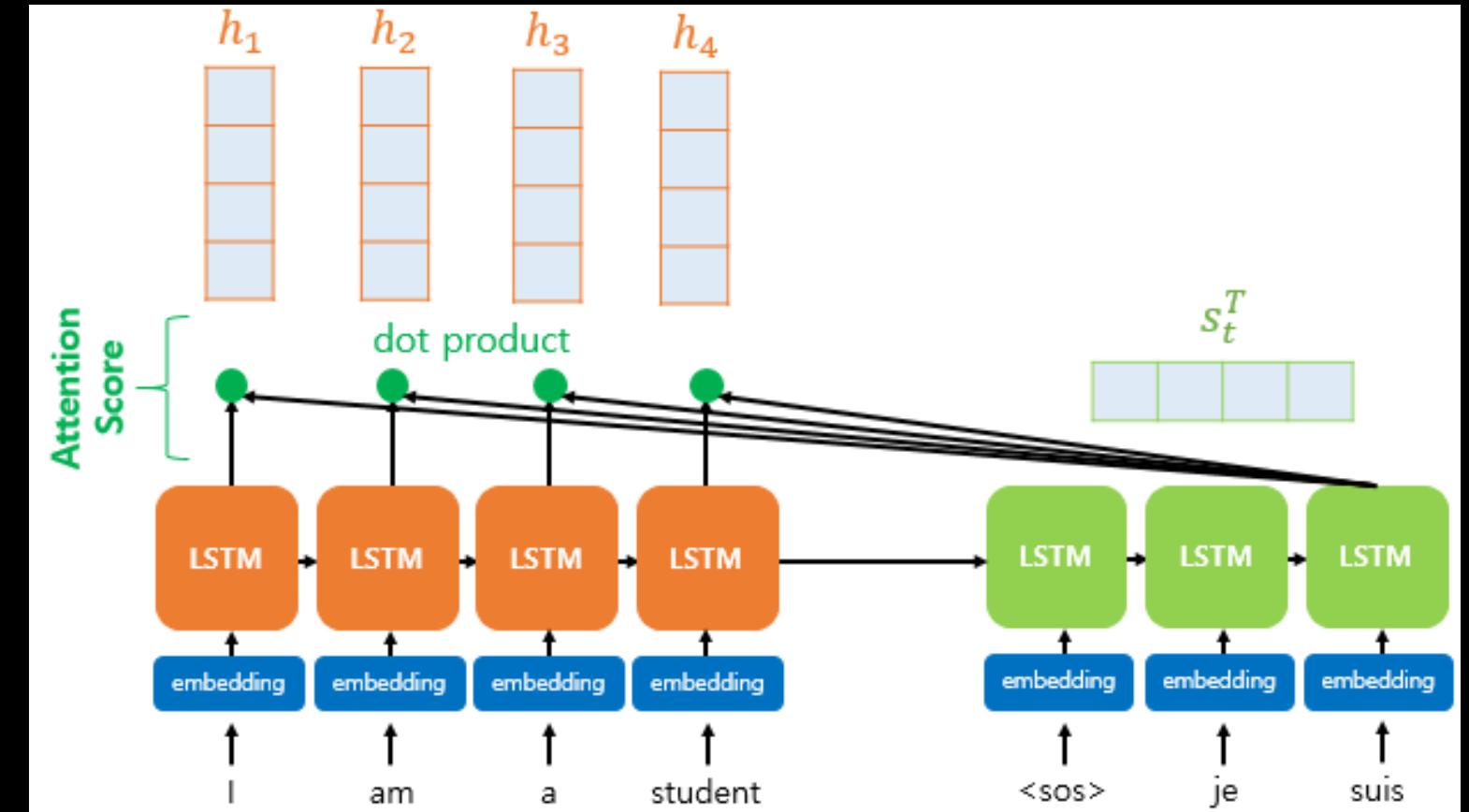
Self-Attention v.s. Attention

기존 Attention

- Key와 Value는 인코더(입력 문장)의 Hidden state값과
부속 연산 결과
- Query는 디코더(출력 문장)의 Hidden state

Self-Attention

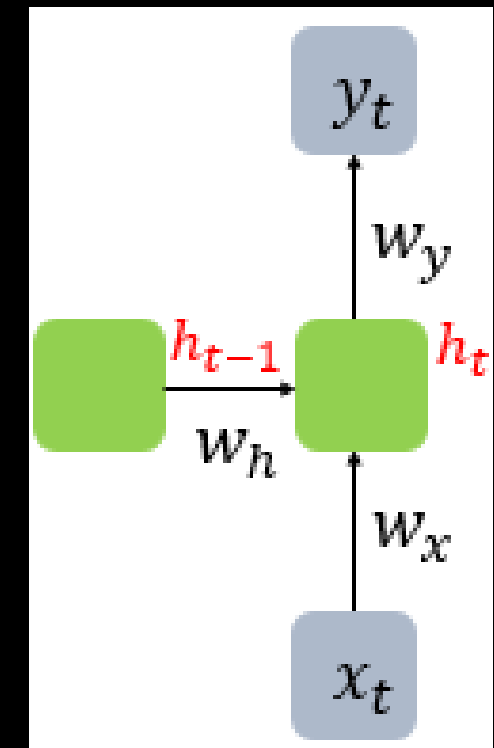
- Q, K, V 모두 동일한 문장에서 유래
 - 인코더의 경우 동일한 영어 문장에서 Q, K, V 모두 계산
 - 디코더에서도 프랑스어 문장에서 Q, K, V 모두 계산



Self-Attention의 가중치

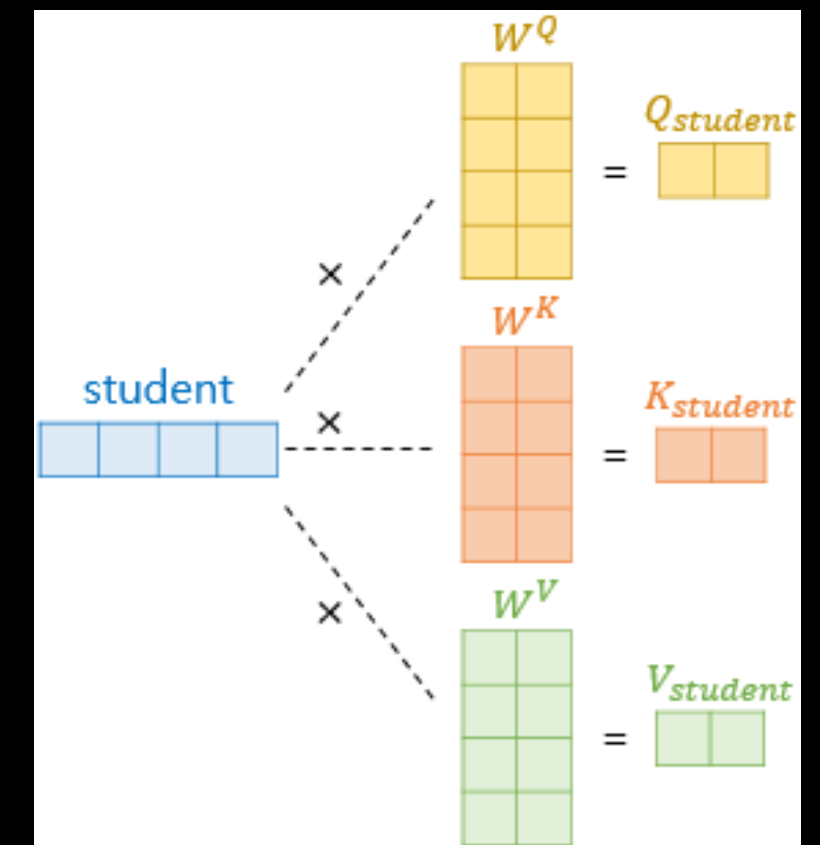
RNN에서의 가중치

- 입력값을 읽어오는 가중치 W_x
- 이전 시점을 받아 다음 시점으로 넘겨줄 때 연산되는 가중치 W_h
- 경우에 따라 출력 값을 생성할 때 사용하는 가중치 W_y



Self-Attention에서의 가중치

- 임베딩된 입력 문장에서 Q, K, V 를 생성하기 위한 행렬
- W^Q, W^K, W^V
- 위 세 가중치는 동일한 Shape의 Linear layer



Self-Attention

기존 Attention과 연산 방식에도 차이가 있음

Attention score

- Q와 K의 내적은 동일하지만
- K의 차원 수 제곱근으로 나누어 Scaling 진행
- 내적 연산 시 Q, K의 벡터 차원 수에 비례하여 값이 커지므로 Scaling 진행

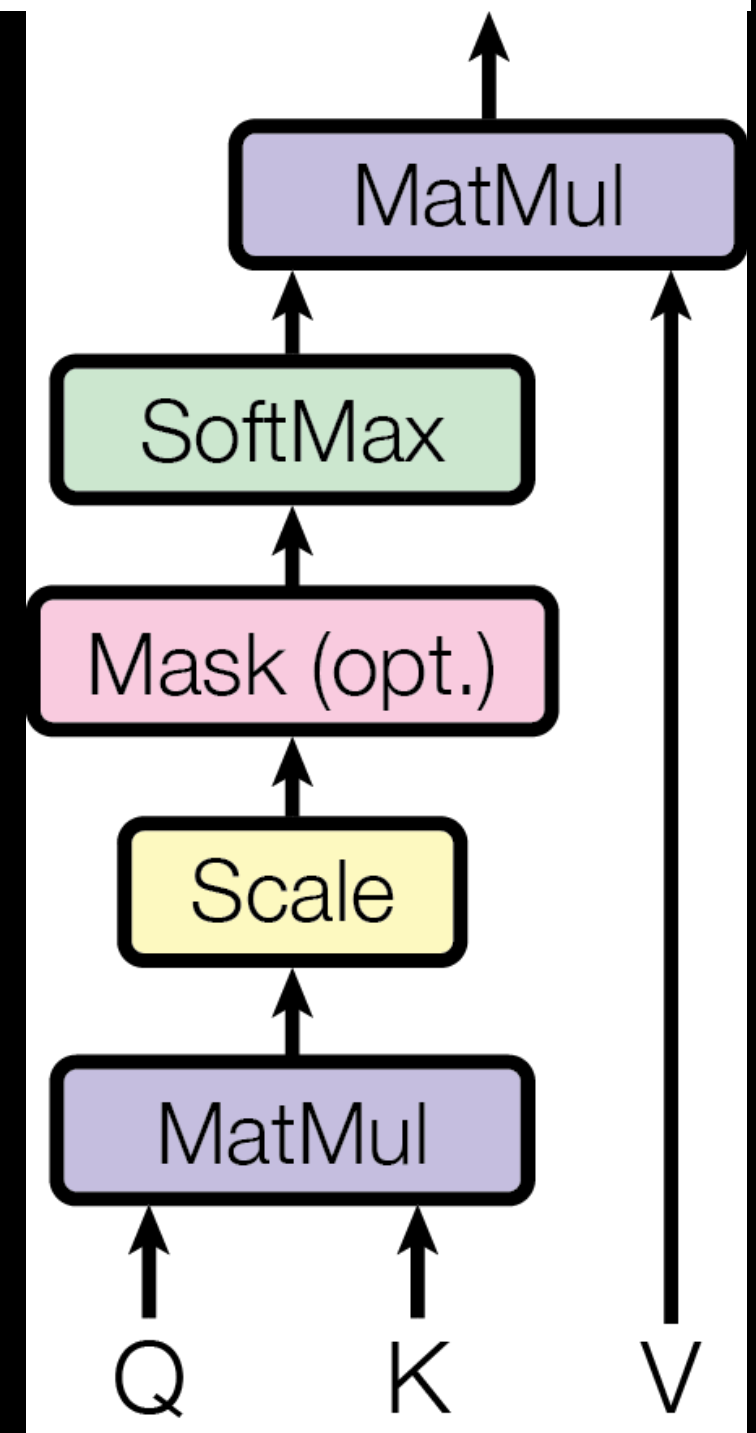
Attention value

- 기존 Attention은 Softmax의 결과와 V를 합했지만
- Self-attention에서는 이 둘을 내적(Dot product)함

그런 이유로 Self-attention을

Scaled dot-product attention 이라고도 함

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



BERT 과정

Transformer 아키텍처

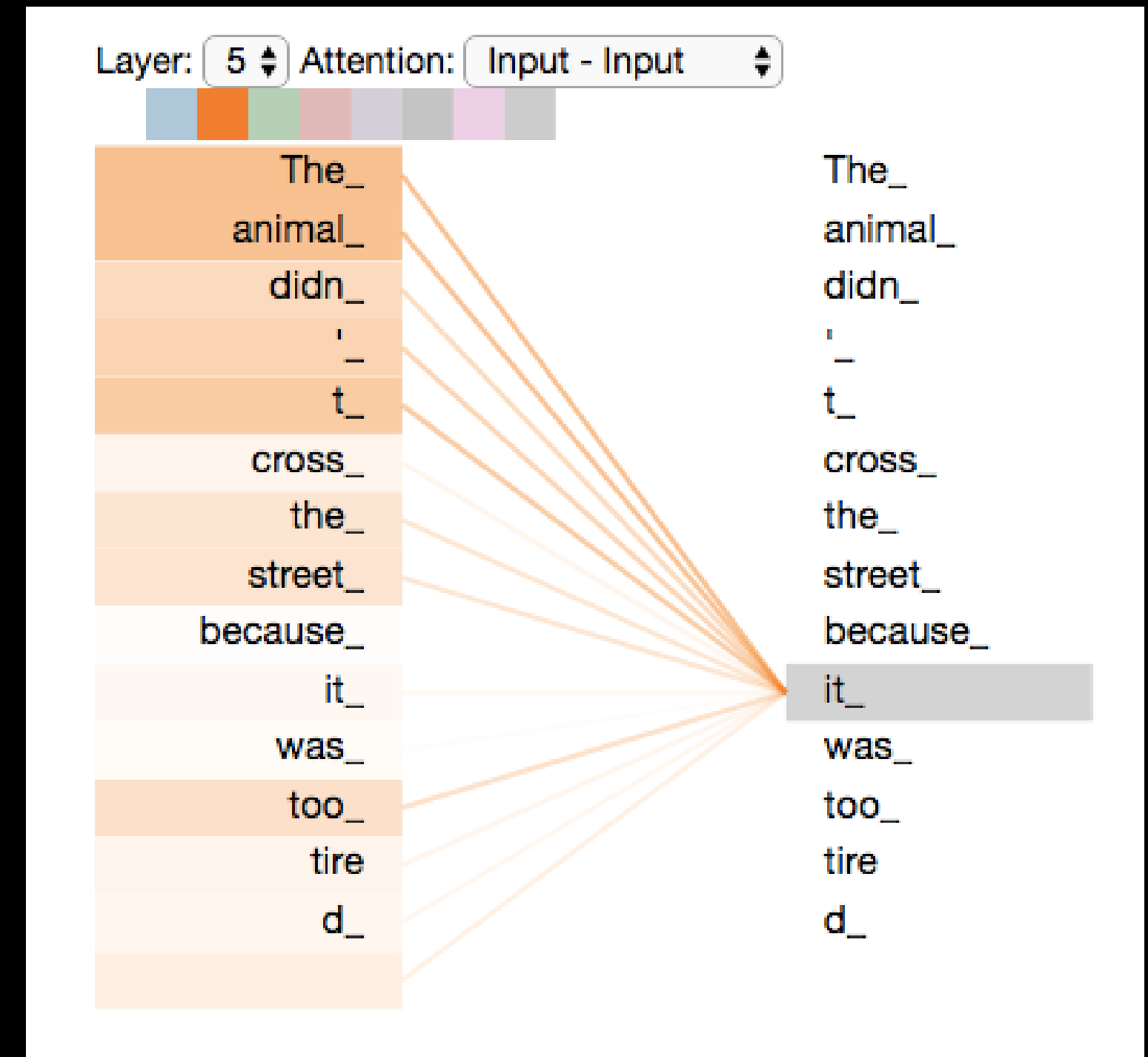
목차 Transformer 아키텍처

1. Transformer 아키텍처

4. Transformer 아키텍처

Self-Attention

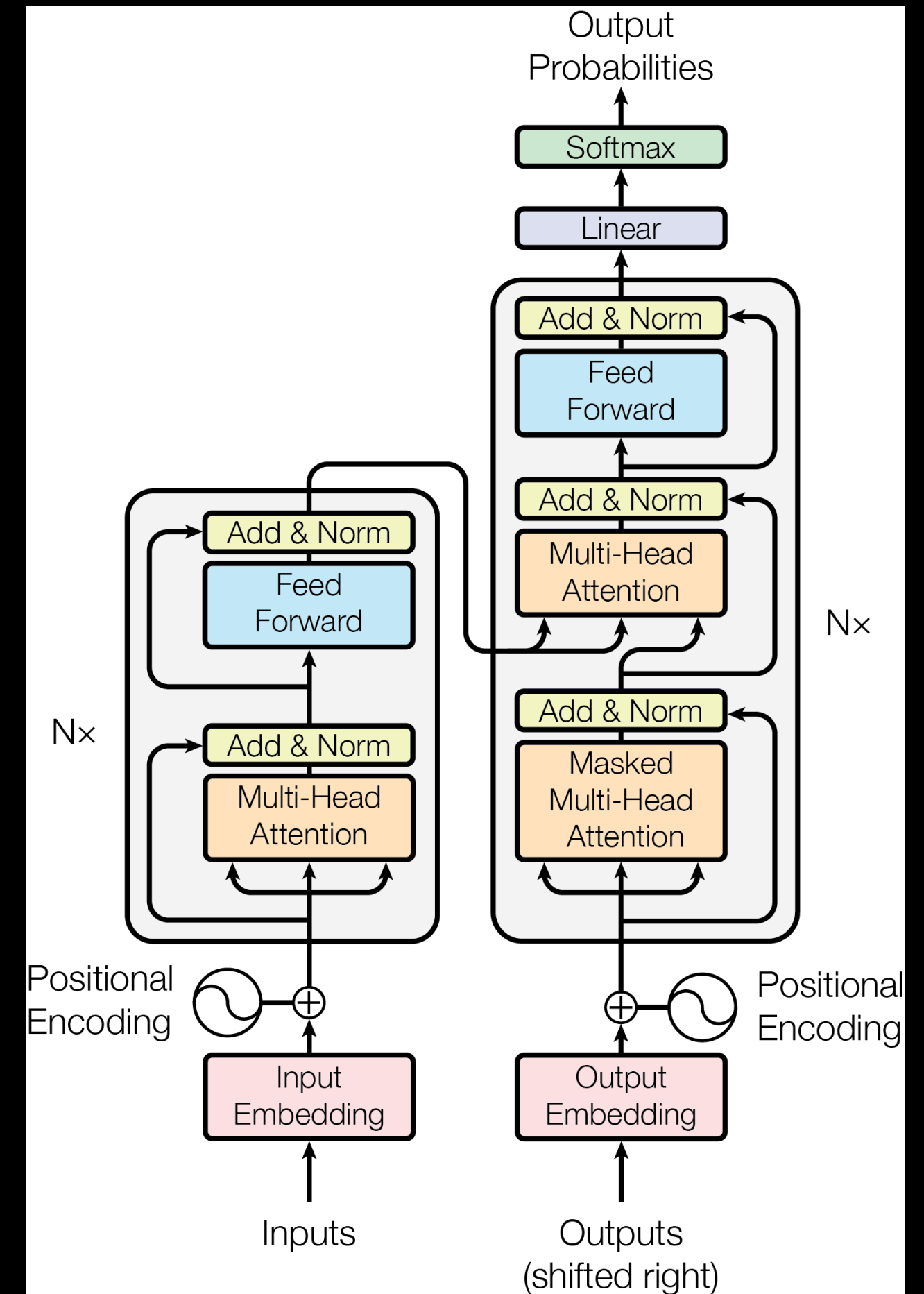
- Transformer의 핵심 원리
 - 시퀀스 내 모든 토큰 간의 관계 파악
 - Query, Key, Value의 개념을 사용하여 Attention 점수 계산
 - 앞서 배운 Attention 개념과 유사
 - Attention 점수를 병렬적으로 계산
- 각 토큰에 대한 가중치를 계산하여 새로운 표현 생성





Transformer의 구조

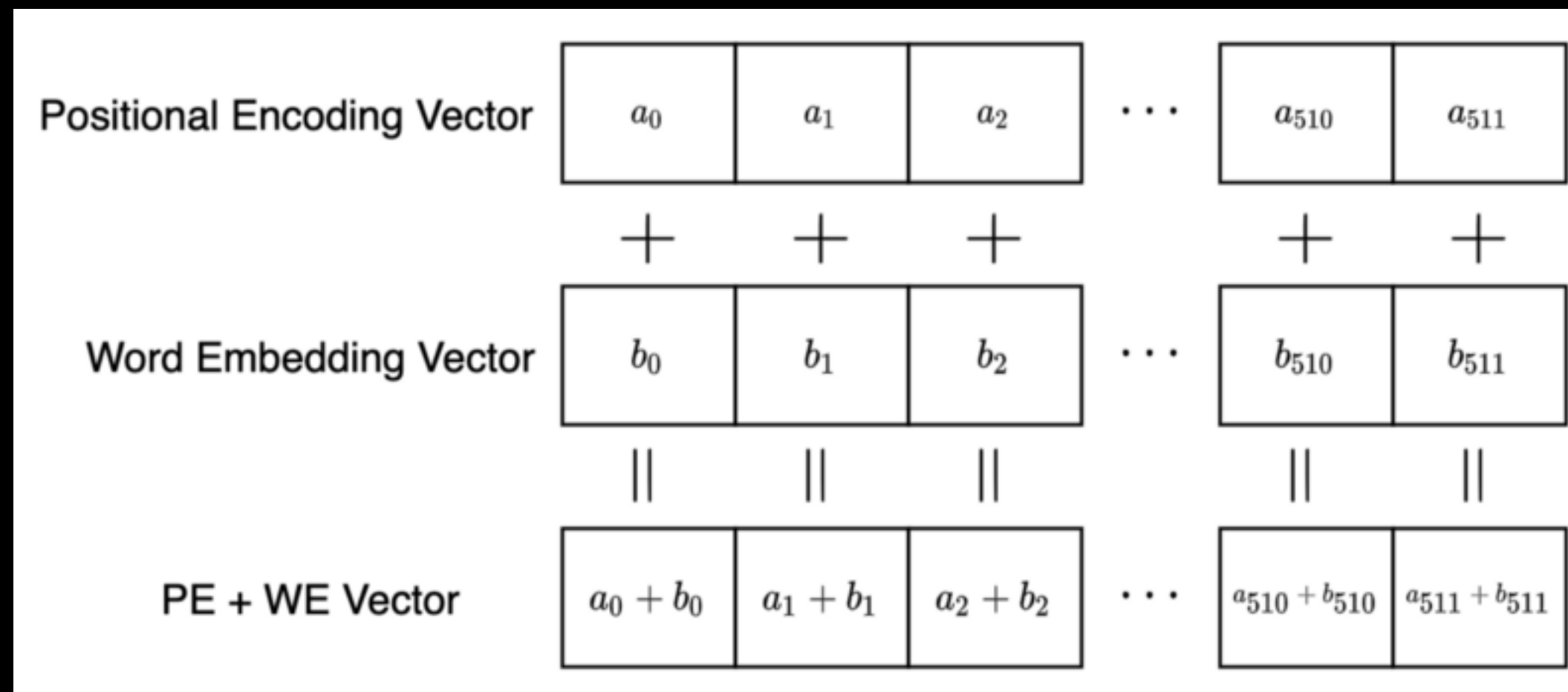
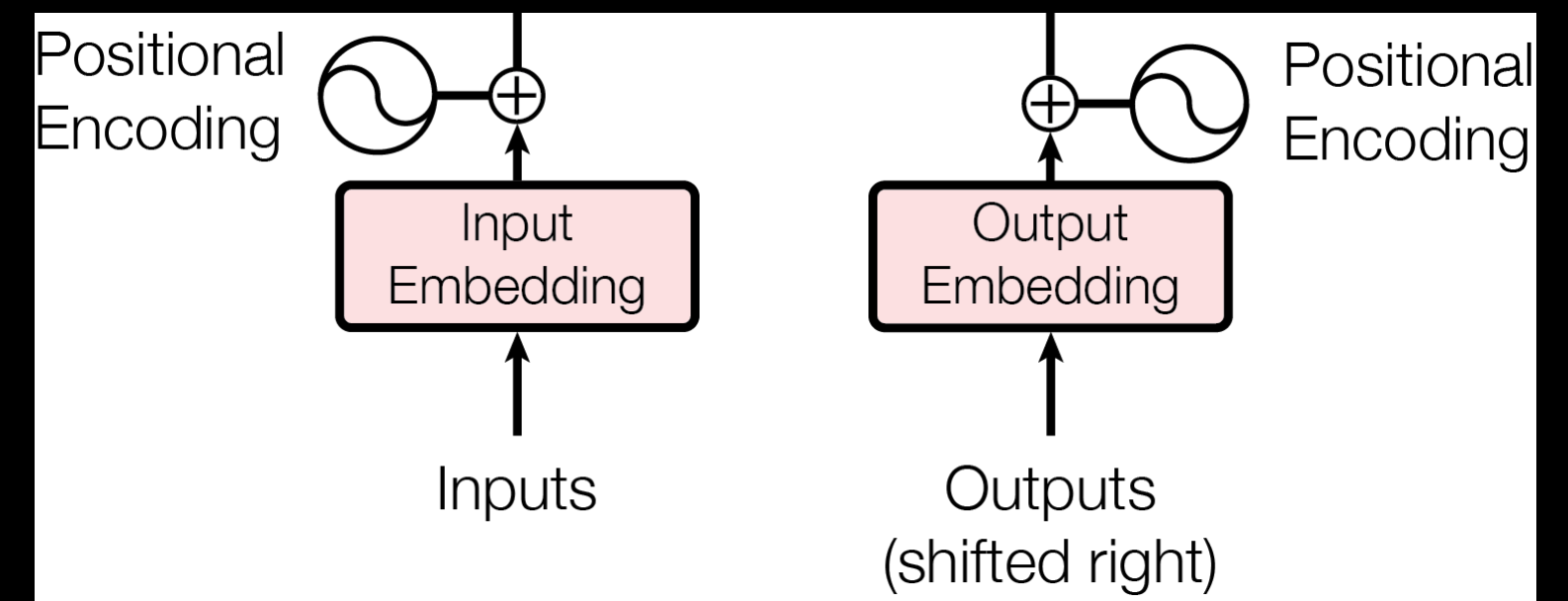
- Positional encoding
- Encoder-decoder 구조
- Attention block
 - Multi-head
 - Masked attention
- Add & Norm
- FFNN(Feed Forward Neural Network)
- Output layer





Positional Encoding

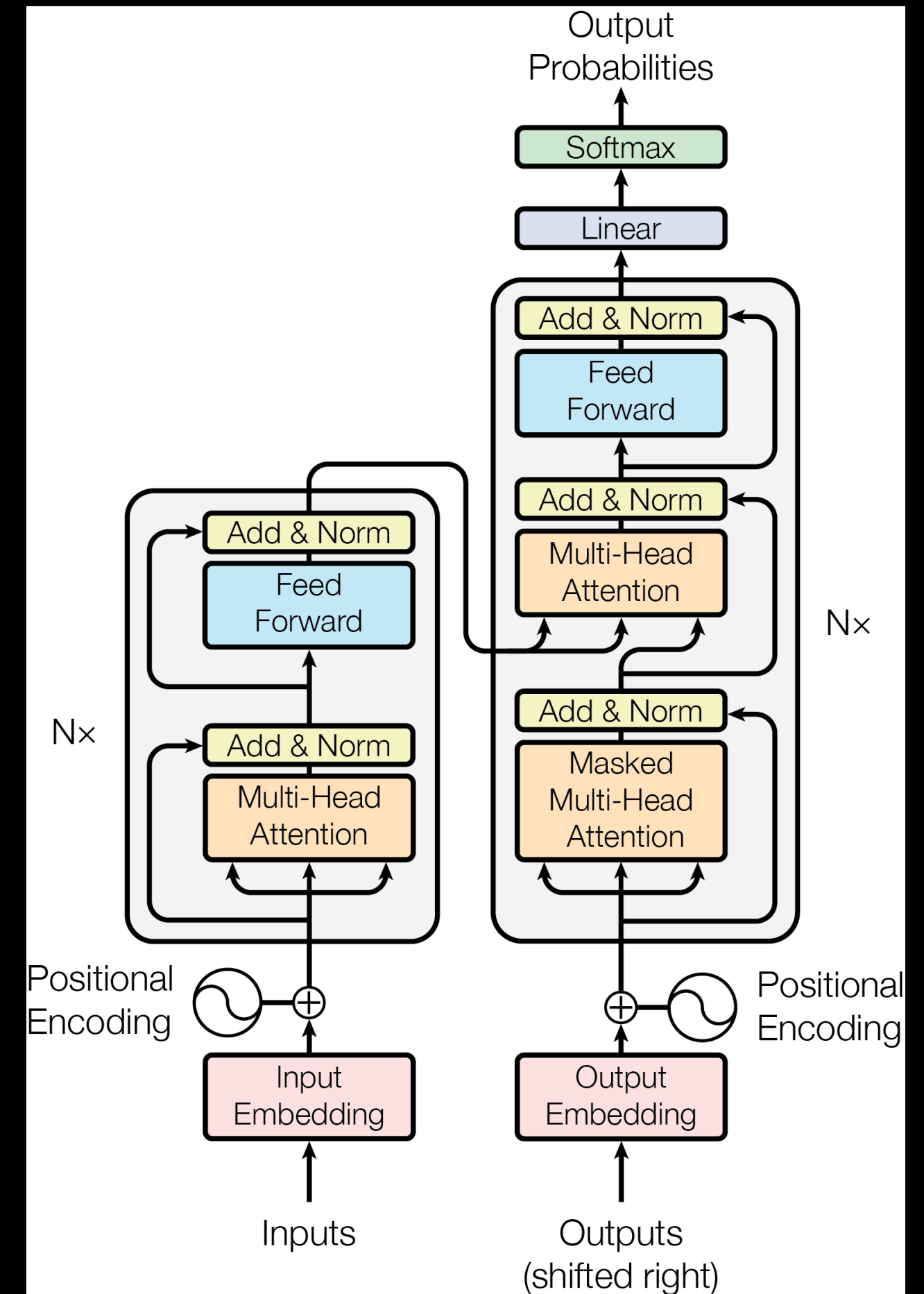
- Transformer는 순서 정보가 없는 구조
 - Token이 순차적으로 입력되지 않고, 한 번에 들어감
- 그러나 문자열 처리 시 순서 정보가 중요함
- 시퀀스 내 토큰의 위치 정보를 주입하기 위한 방법
- 고정된 벡터를 사용하여 각 토큰의 위치 정보를 인코딩





Transformer의 구조

- Positional encoding
- Encoder-decoder 구조
- Attention block
 - Multi-head
 - Masked attention
- Add & Norm
- FFNN(Feed Forward Neural Network)
- Output layer

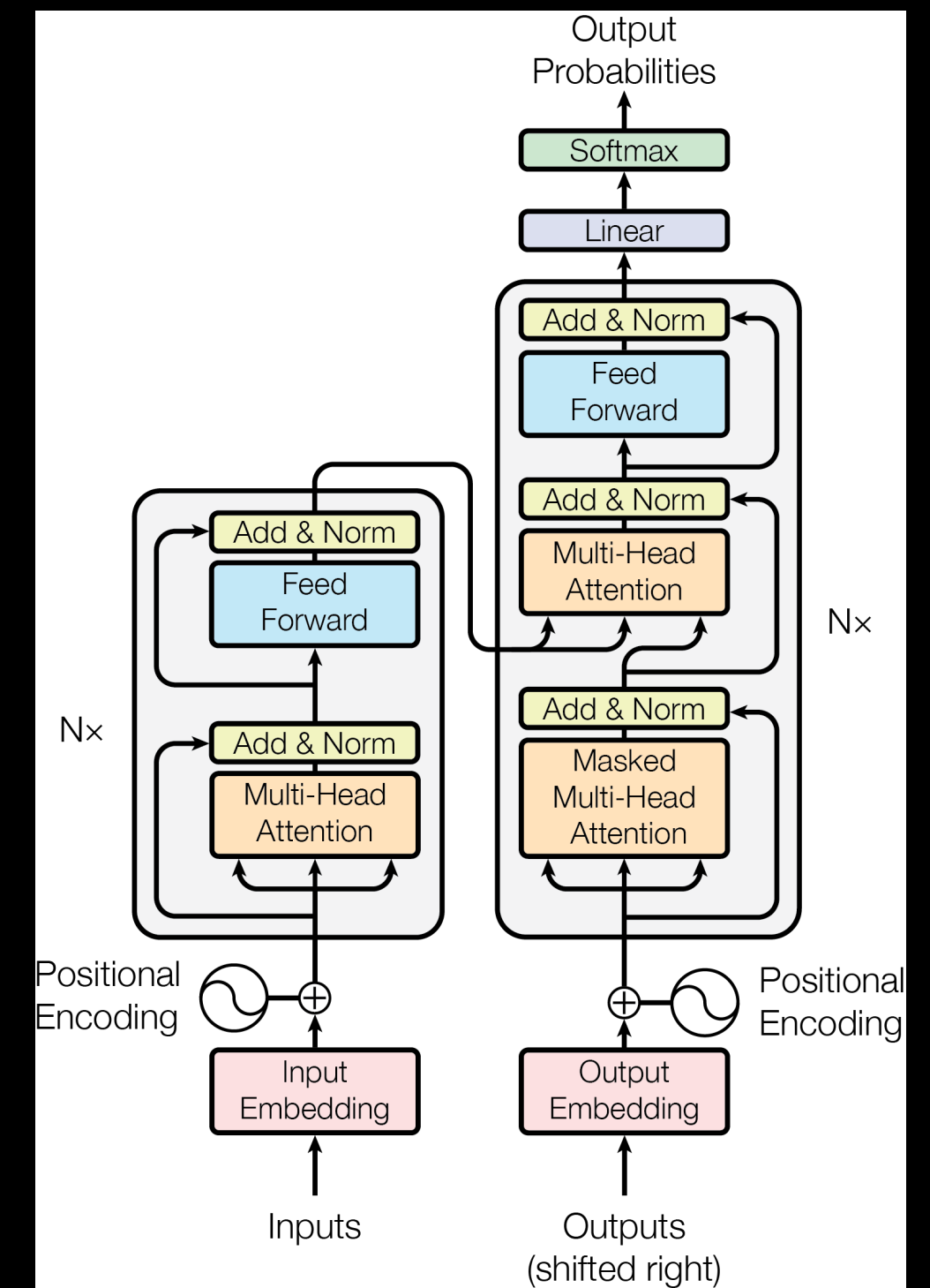




Encoder-Decoder 구조

논문에서 공개된 Transformer 모델은 번역기(Machine translation)

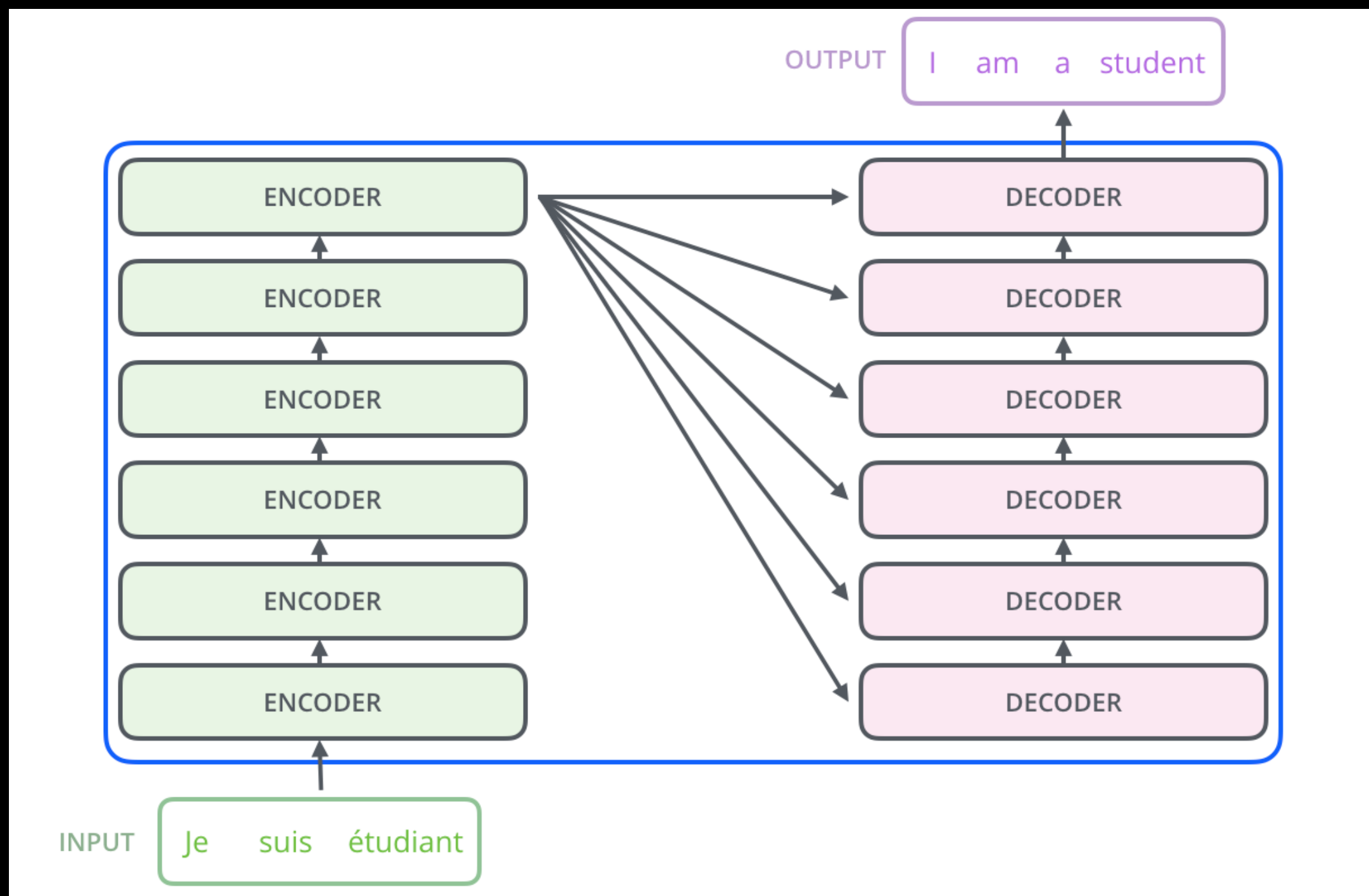
- Eng – Fra, Eng - Ger(WMT2014)
- 인코더
 - Source sentence(영어)을 이해하도록 학습
- 디코더
 - Target sentence(프랑스어/독일어)를 이해하도록 학습
 - Source sentence의 문맥을 참조하여 Target sentence를 생성





Encoder-Decoder 구조

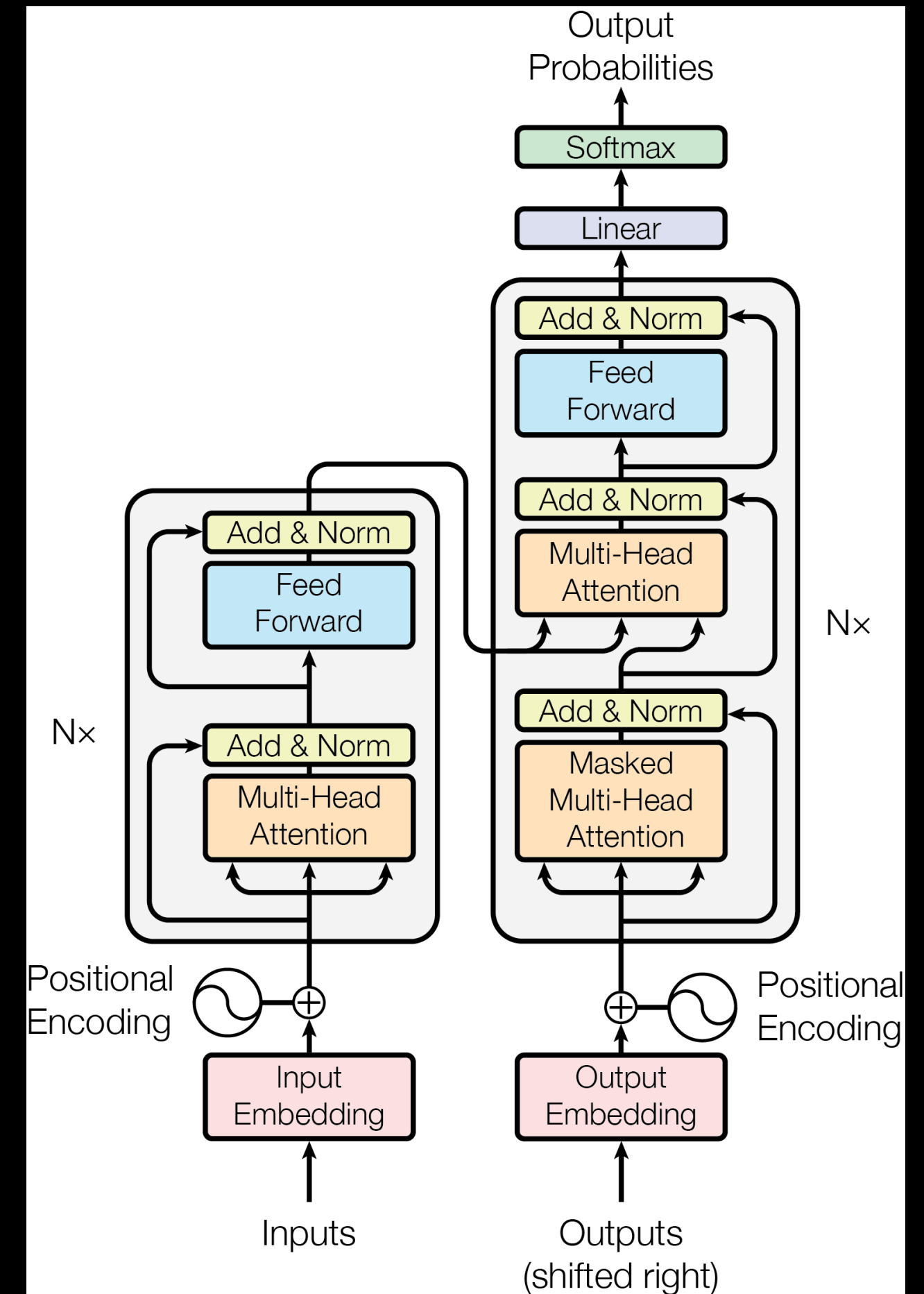
- 한 모델에서 인코더와 디코더를 여러 층으로 쌓음
 - 논문 기준
 - Base 모델: 6층
 - Large 모델: 12층
- 모델의 크기를 원하는 대로 조정 가능
 - 학습 데이터의 크기에 따라
 - 데이터의 복잡도에 따라
 - 스케일링 법칙 적용 가능





Transformer의 구조

- Positional encoding
- Encoder-decoder 구조
- Attention block
 - Multi-head
 - Masked attention
- Add & Norm
- FFNN(Feed Forward Neural Network)
- Output layer

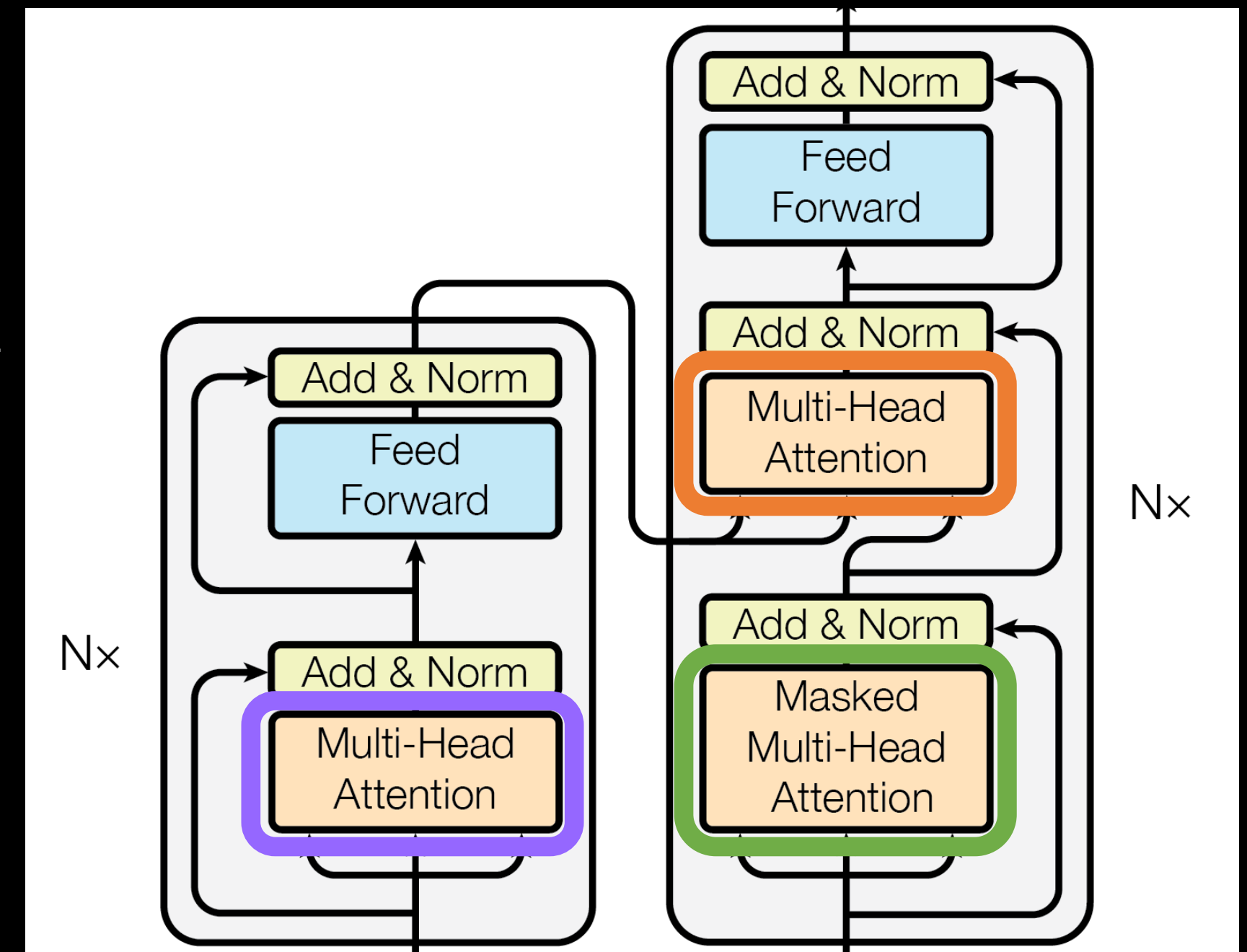




Attention Block

모델 내 번역 Task 수행을 위하여 세 가지 Attention block이 활용

- 인코더
 - Multi-head attention: Source sentence 내 문맥 이해 학습
- 디코더
 - Masked multi-head attention: Target sentence 내 문맥 이해 학습
 - Multi-head (cross) attention(위): Target sentence 생성 학습





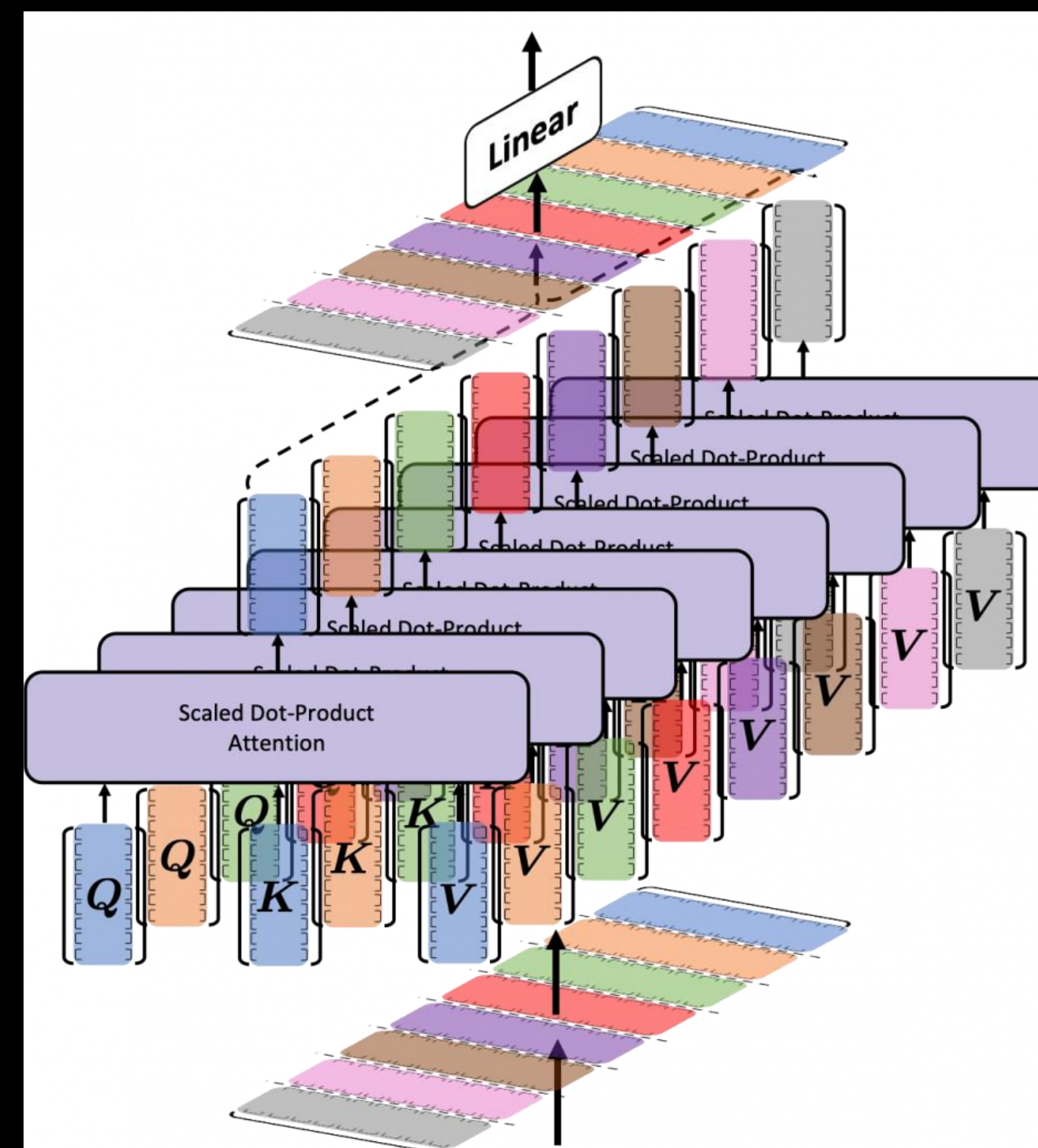
Multi-Head Attention

여러 개의 Attention 메커니즘 동시 사용

- Embedding dimension을 Head 수로 나눠 처리
- 병렬 연산 가능
- Head 별 분산 학습

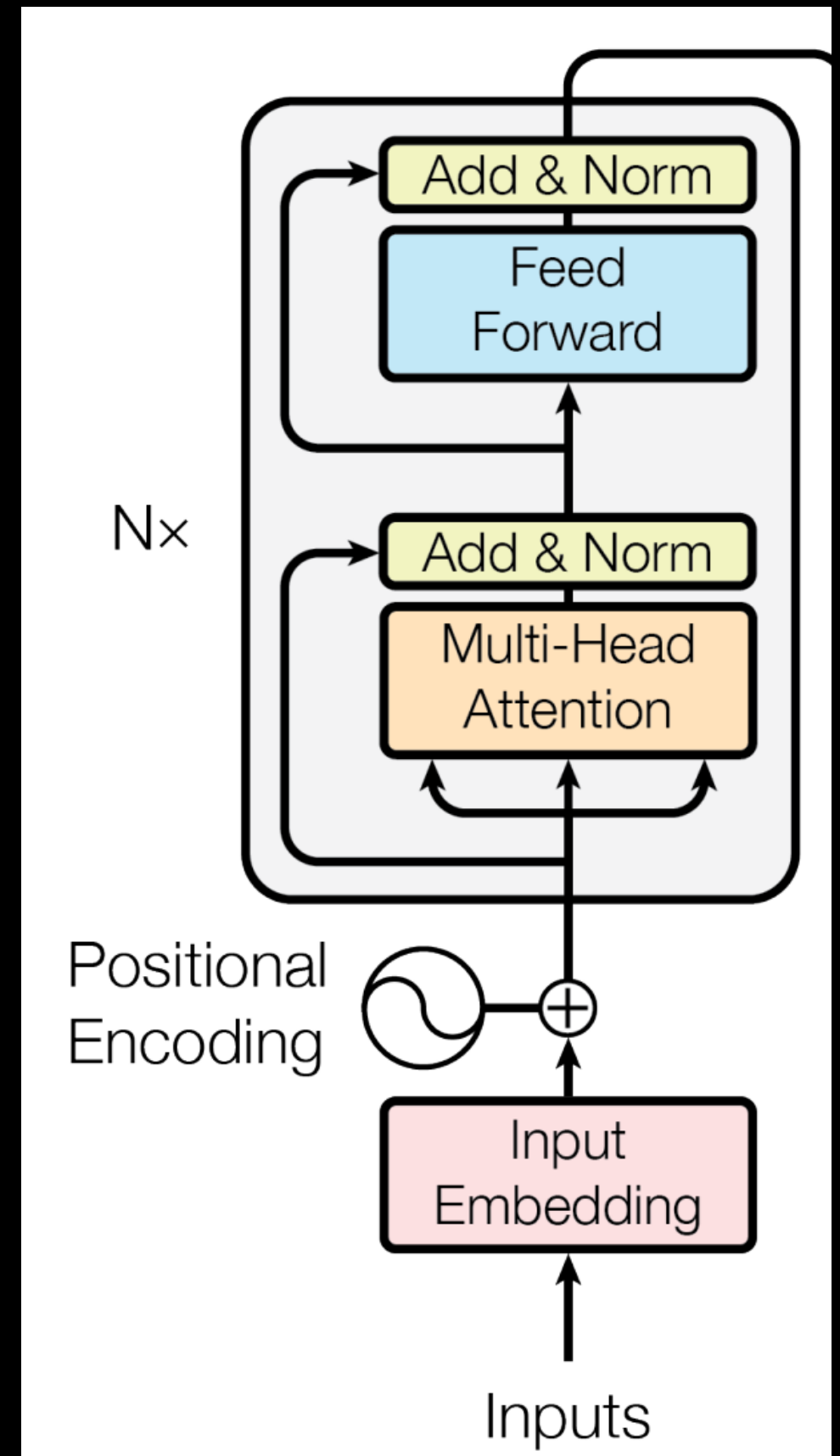
각 Head는 다른 표현 공간에서의 Attention 정보를 캡처

- 여러 모델을 동시 학습시키는 것과 유사(Ensemble)
- 데이터에서 다양한 Feature를 추출할 수 있음
- 모든 Head의 결과를 결합하여 더 풍부한 정보 표현 생성



Multi-Head Attention(Encoder)

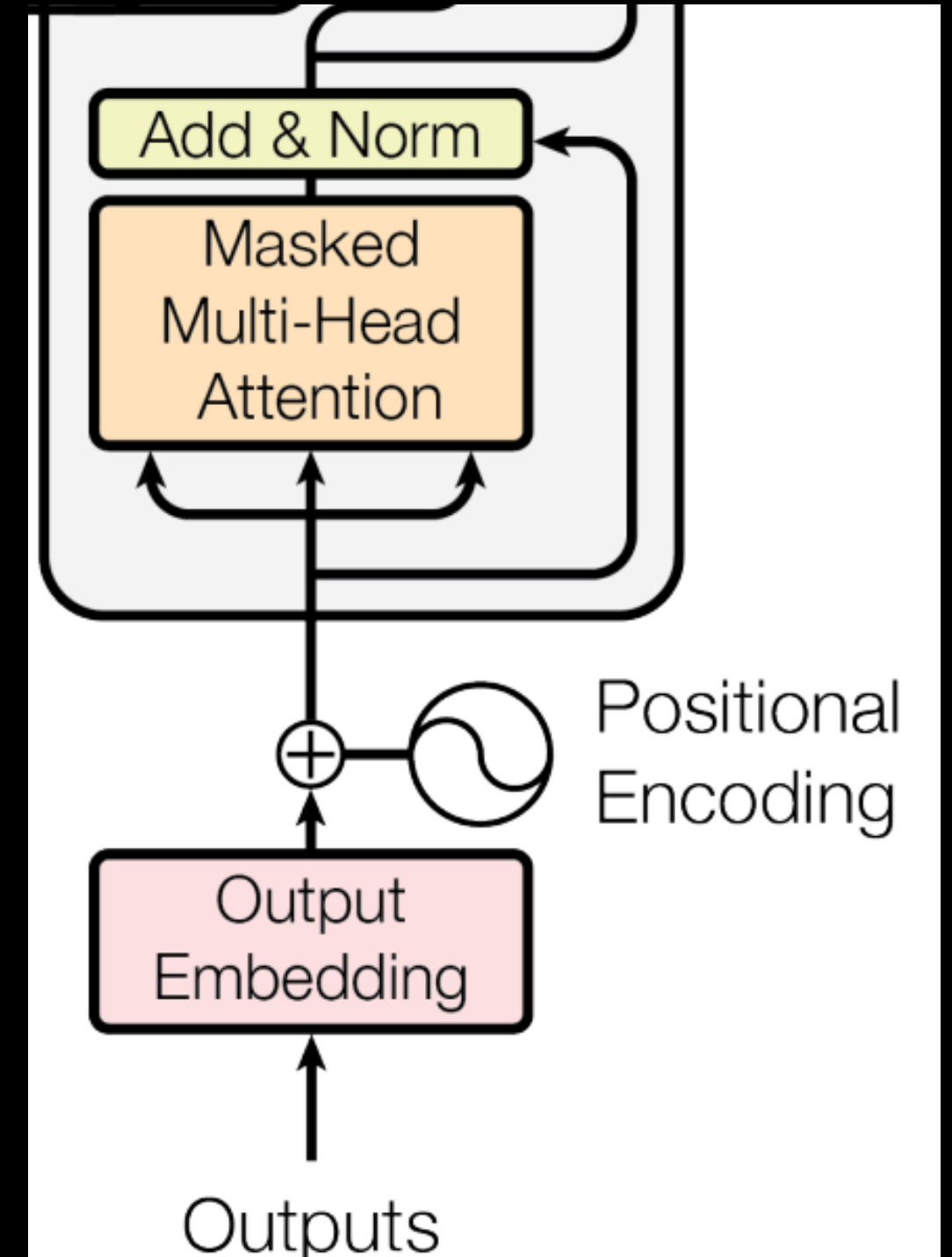
- 논문 기준 8개의 Head로 구성
- Self-Attention 연산을 통해 입력 문장의 정보를 학습
- 학습된 정보는 디코더의 Multi-Head Attention에 전달됨





Masked Multi-Head Attention(Decoder)

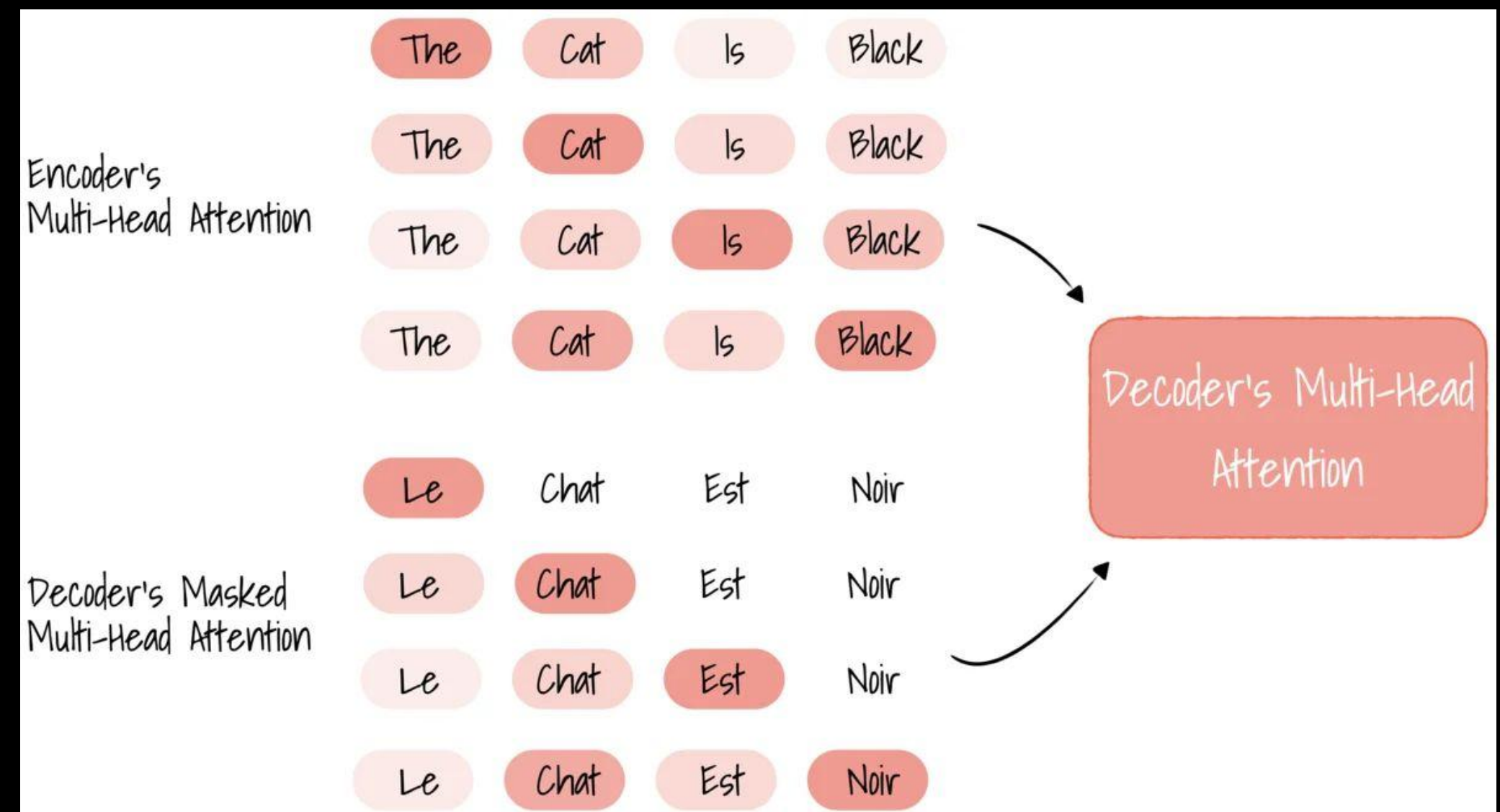
- 디코더에만 존재하는 유일한 구조
- 디코더는 LM(Language Model)의 역할을 수행해야 함
- 즉 앞 단어를 바탕으로 뒷단어를 예측할 수 있어야 함
- 해당 기능을 구현할 수 있는 장치 필요





Masked Multi-Head Attention(Decoder)

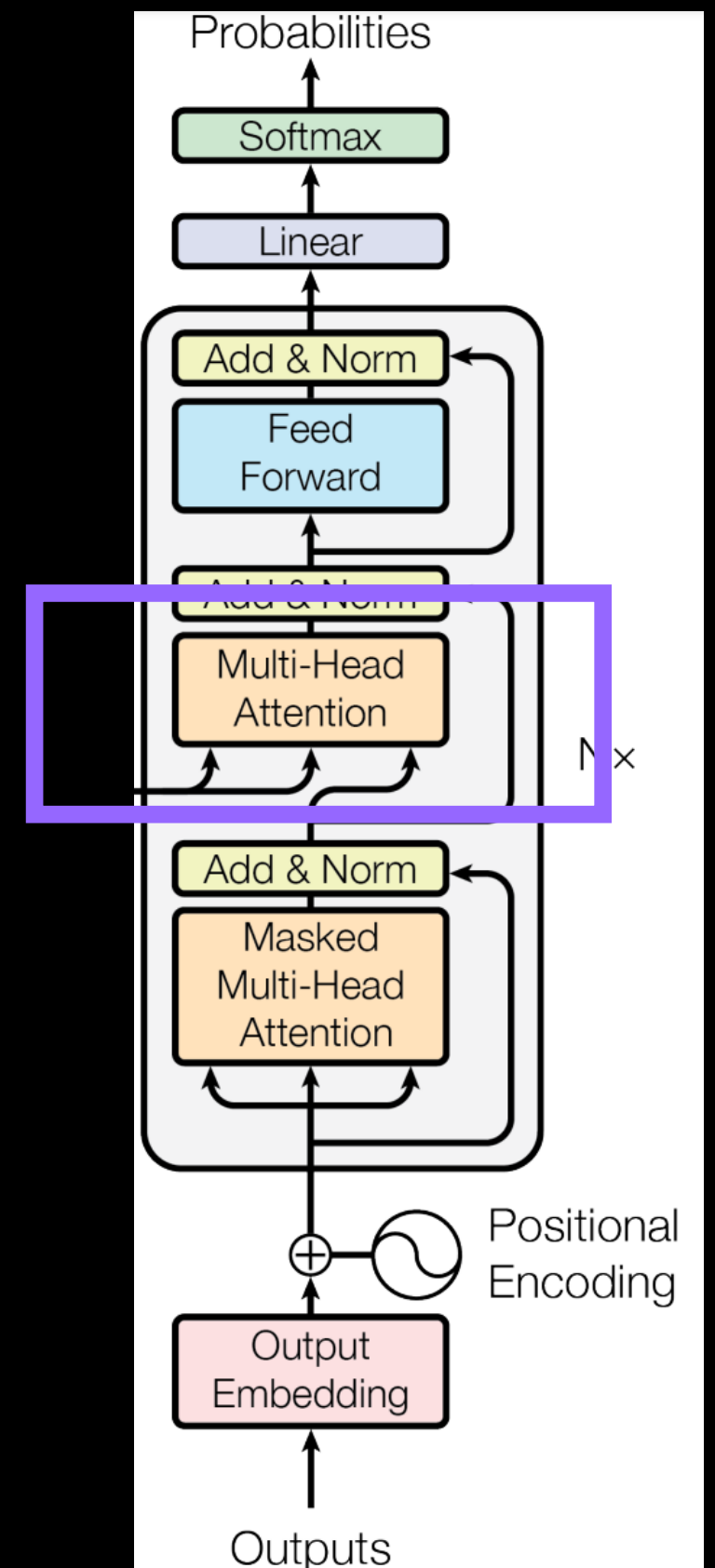
- 디코더에서 문장을 입력받을 경우, Self-attention 계산 시 **문장의 뒷부분을** 의도적으로 가림
 - Masking 된 영역은 연산에서 배제됨
- 문장의 제일 **앞 단어부터 하나씩 마스크를 해제하며 Self-attention 계산**
- 결과적으로 단어 간 순서 정보를 학습할 수 있음





Multi-Head Attention(Decoder)

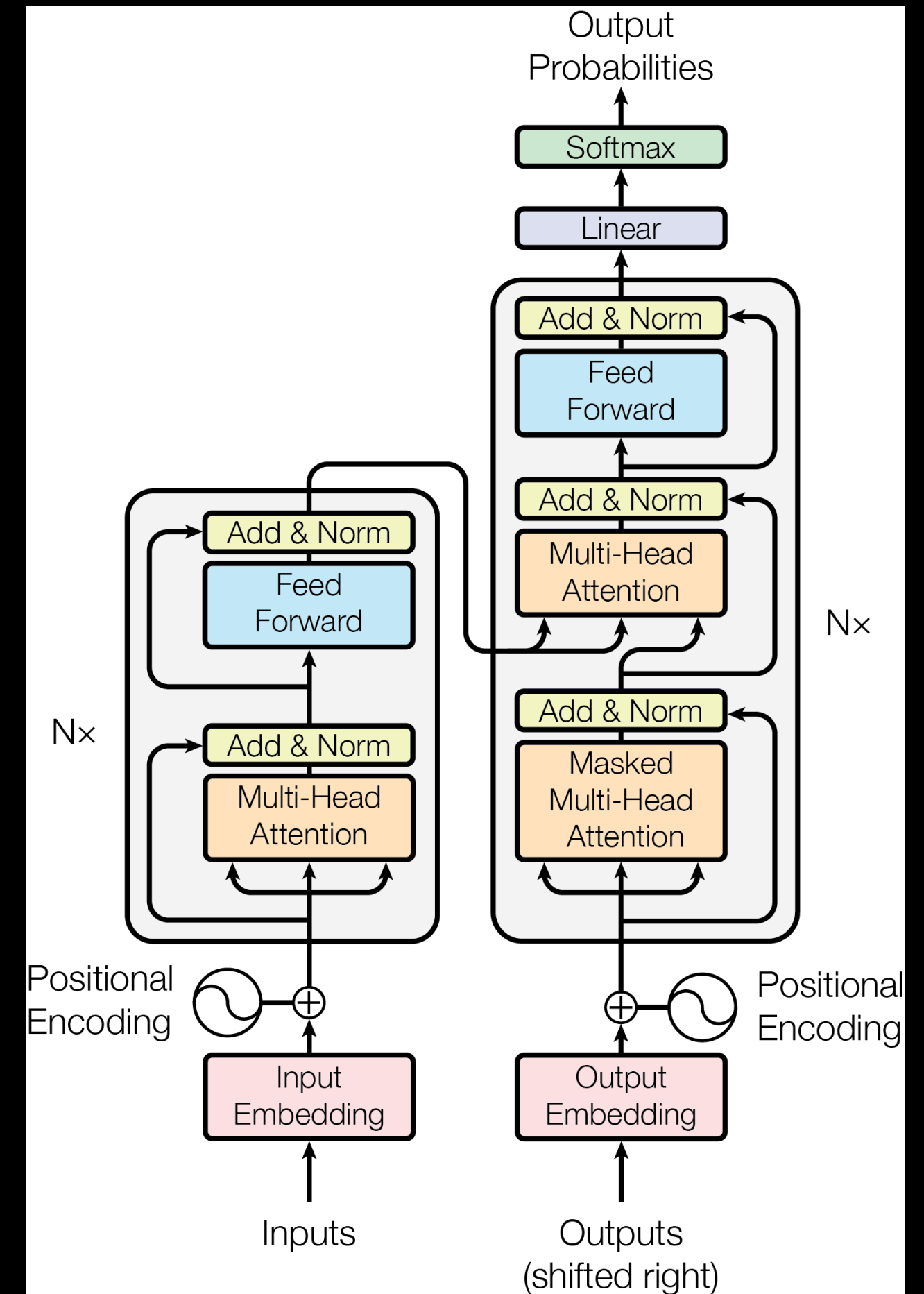
- 인코더의 Multi-head attention과 유사한 구조
- Cross-attention 구조
- 다만, 이 과정에서 K, V 는 인코더에서, Q 는 디코더에서 받음
- Seq2seq Attention 구조와 동일한 목적을 위해 연산
 - 디코더의 단어 생성(Q)을 위해
 - 인코더의 Hidden state(K, V)를 참조





Transformer의 구조

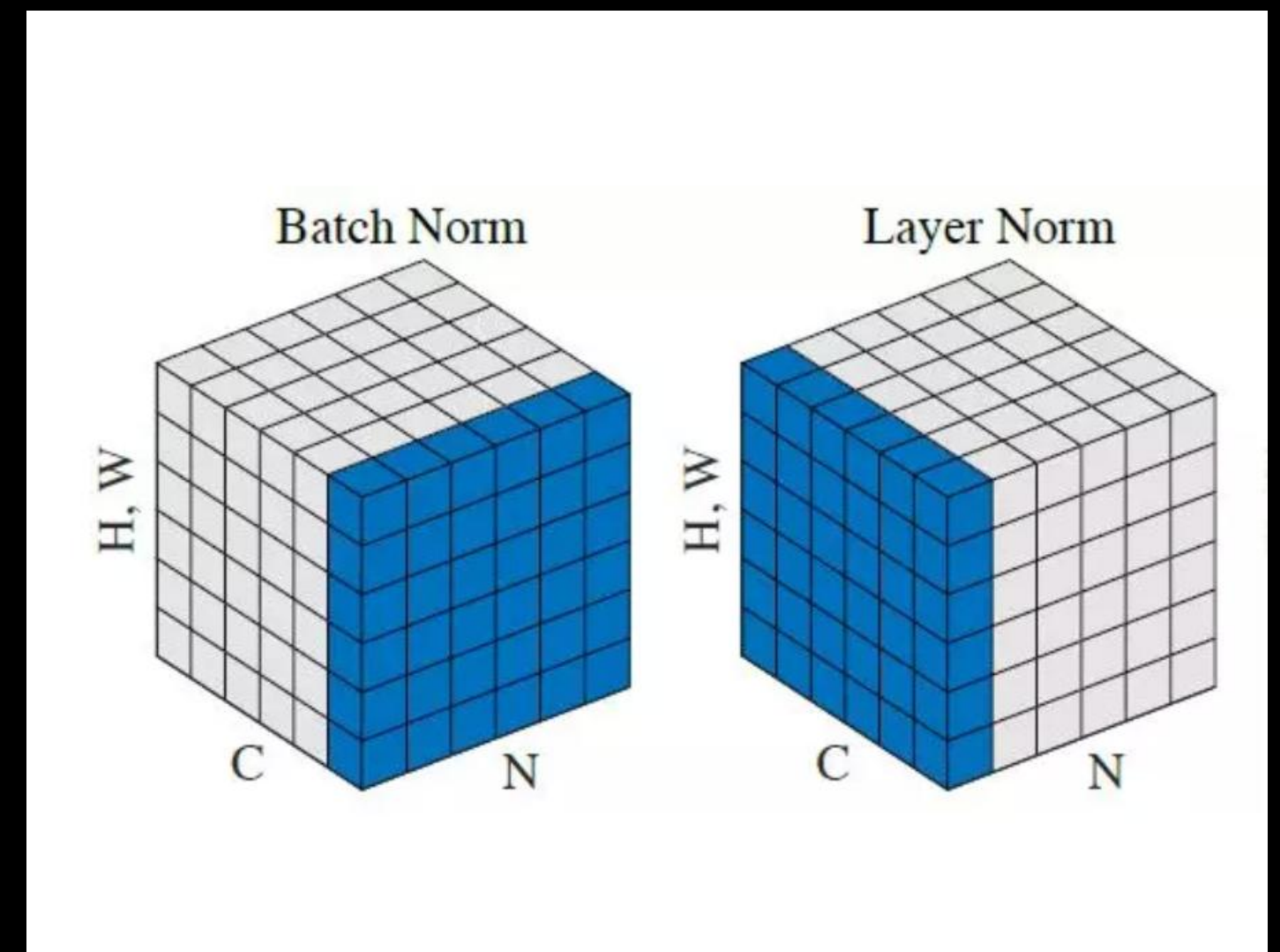
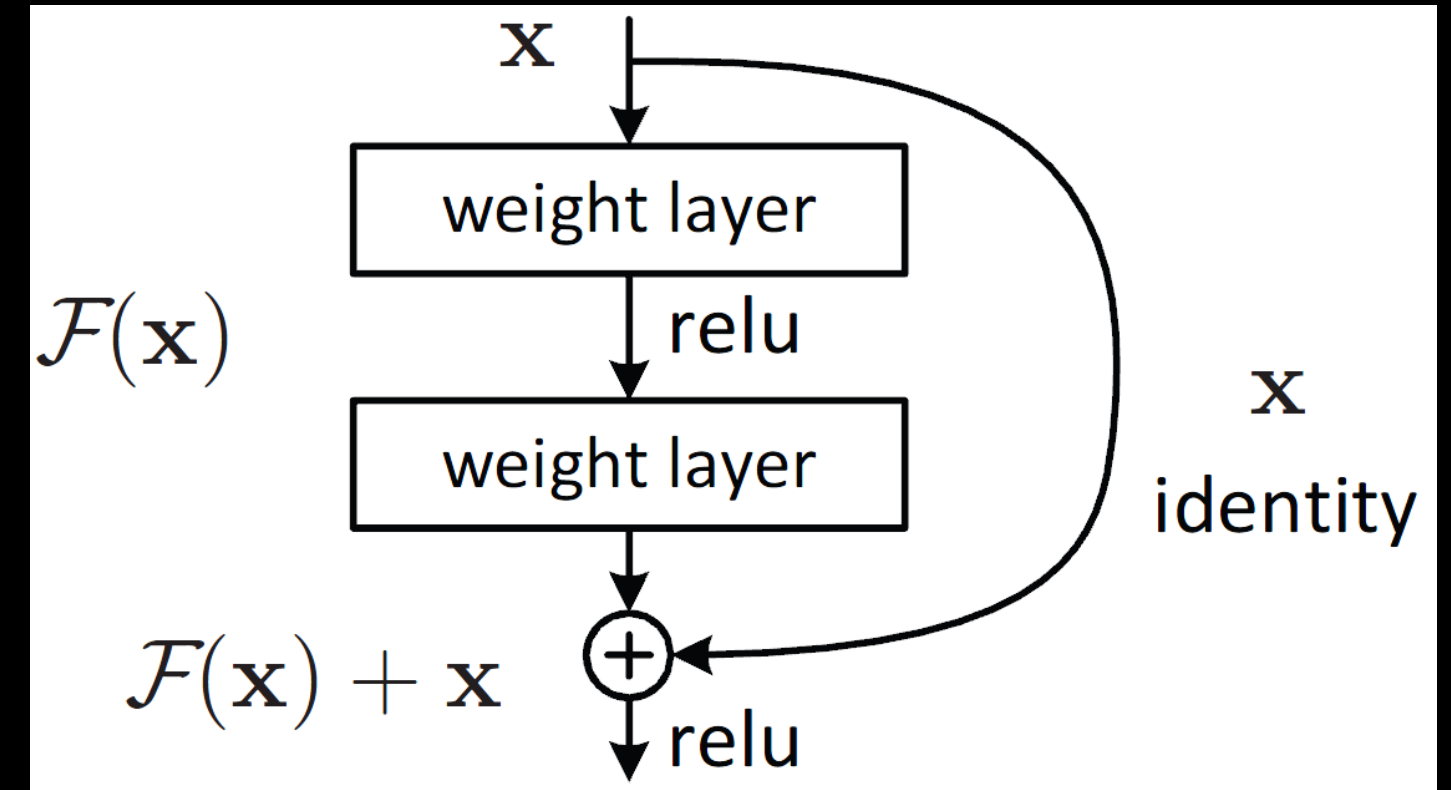
- Positional encoding
- Encoder-decoder 구조
- Attention block
 - Multi-head
 - Masked attention
- Add & Norm
- FFNN(Feed Forward Neural Network)
- Output layer





Add & Norm

- Add(Residual connection)
 - ResNet(2015)에서 제시된 아이디어
 - 신경망을 통과한 정보와 Original 입력값을 더함
- Layer normalization
 - 데이터 샘플의 Feature를 기준으로 정규화
 - 평균 0, 표준편차 1
 - Batch norm 대신 Sequence 모델에 사용하는 정규화 기법



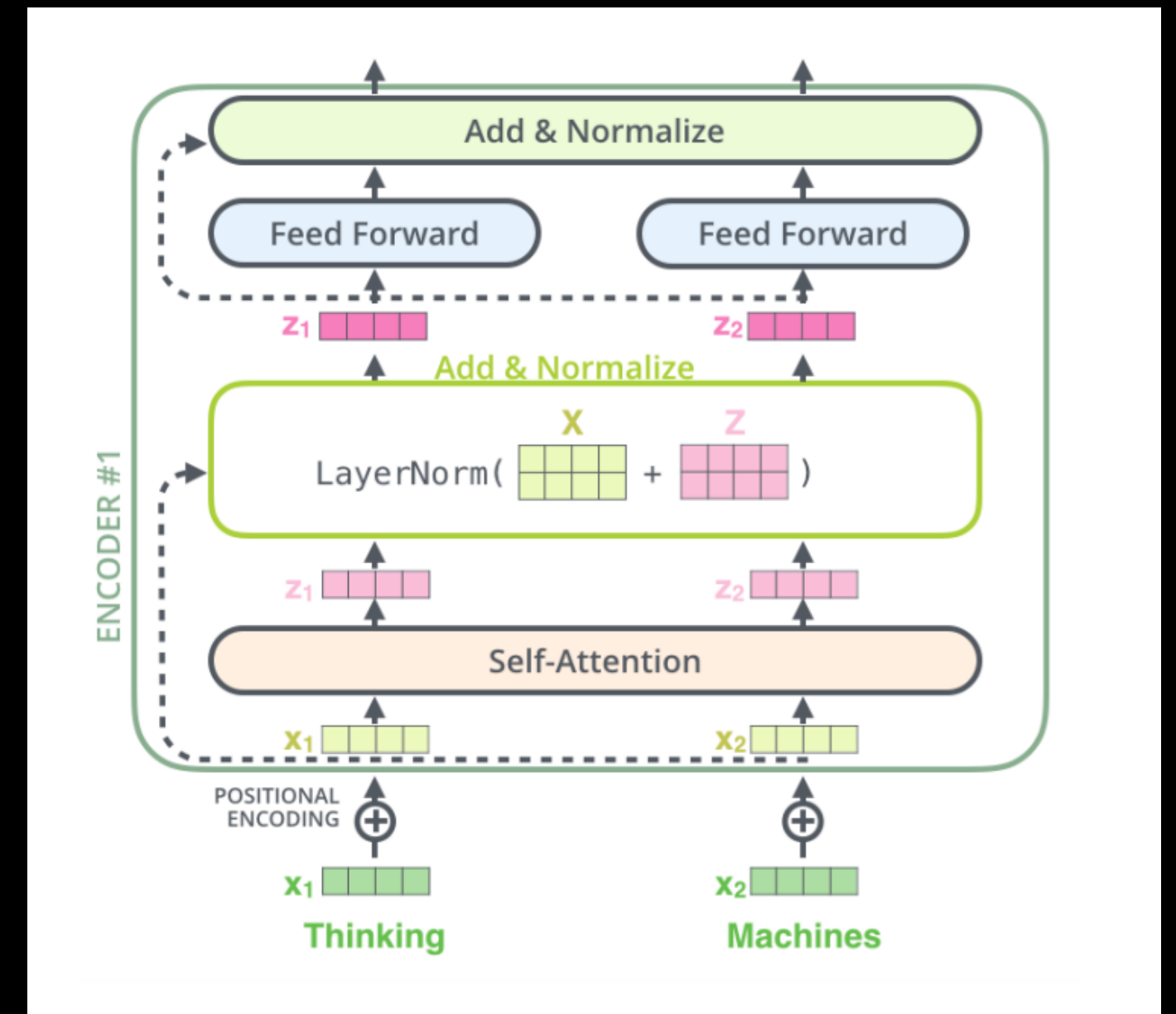
기대효과

Gradient 흐름을 원활하게 하여 학습 안정

- Transformer 등 복잡하고 깊은 신경망에서는 Vanishing/exploding gradient 문제가 자주 발생
- Residual connection의 경로가 미분 시 Gradient 경로를 하나 추가해줌

데이터 분포를 안정화

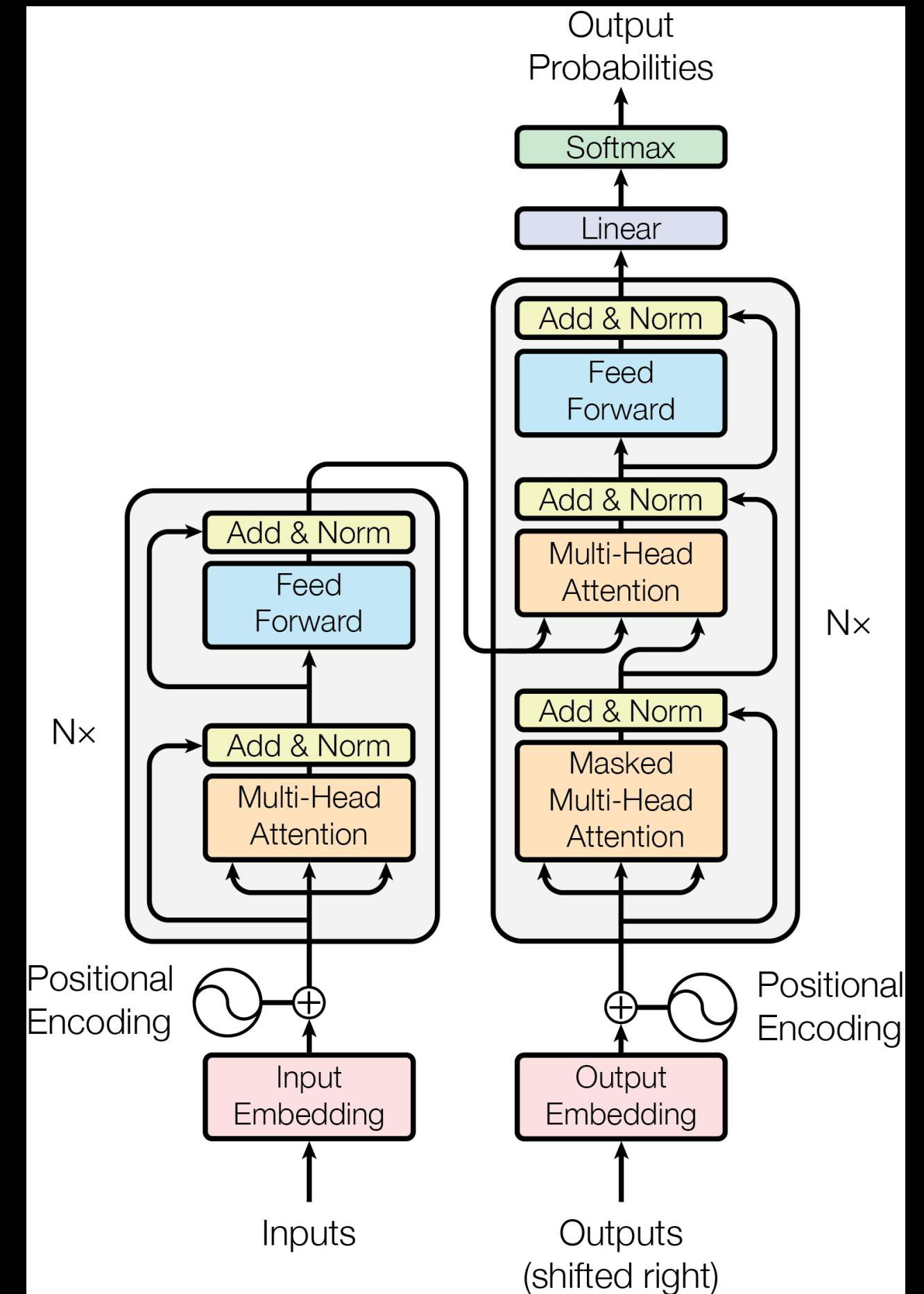
- 신경망을 비롯한 ML 알고리즘은 정규화된 값에서 특징을 더 잘 추출
- Residual connection은 데이터 분포를 지나치게 키우거나 이상하게 만들 수 있음





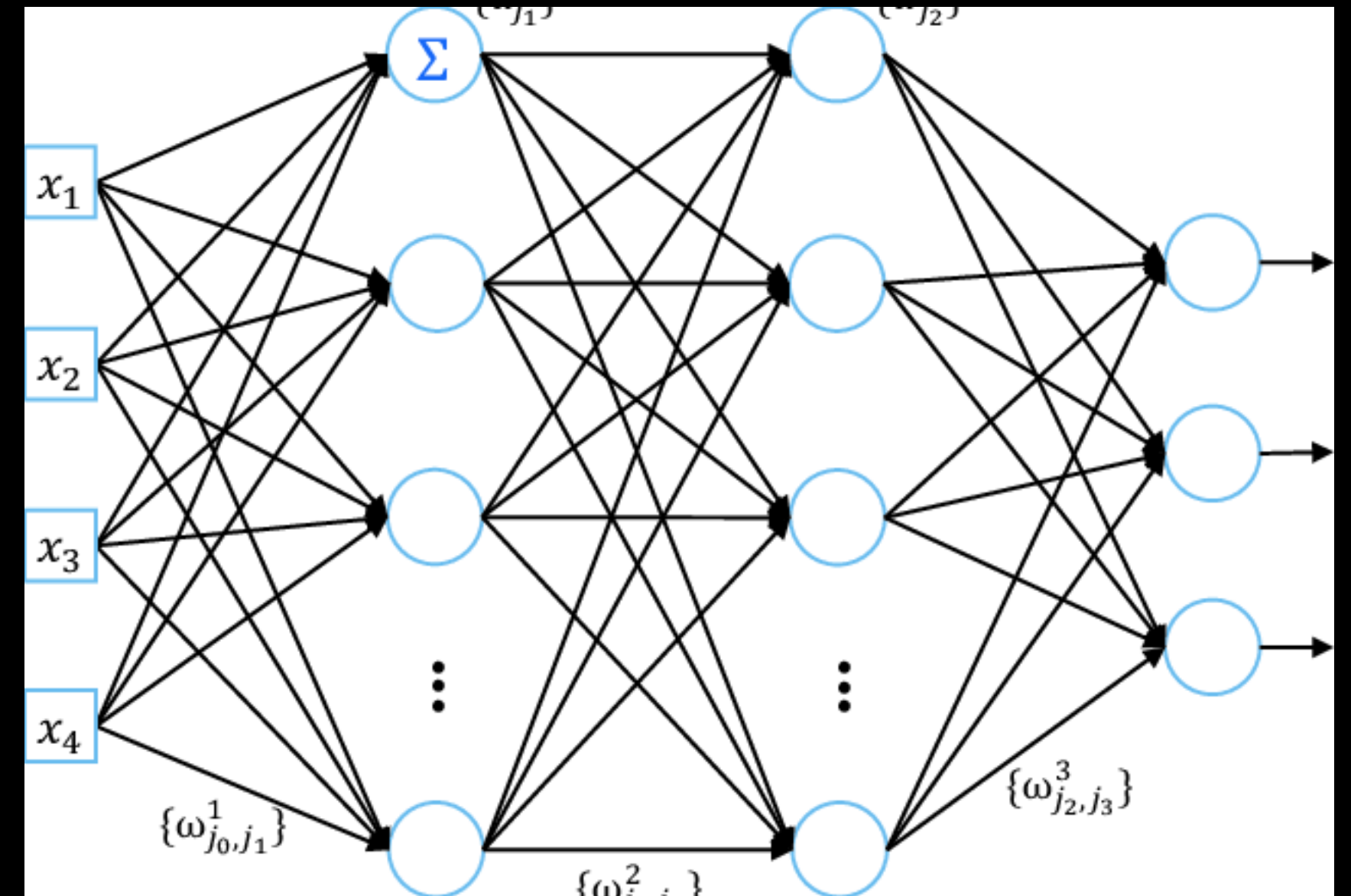
Transformer의 구조

- Positional encoding
- Encoder-decoder 구조
- Attention block
 - Multi-head
 - Masked attention
- Add & Norm
- FFNN(Feed Forward Neural Network)
- Output layer



FFNN(Feed Forward Neural Network)

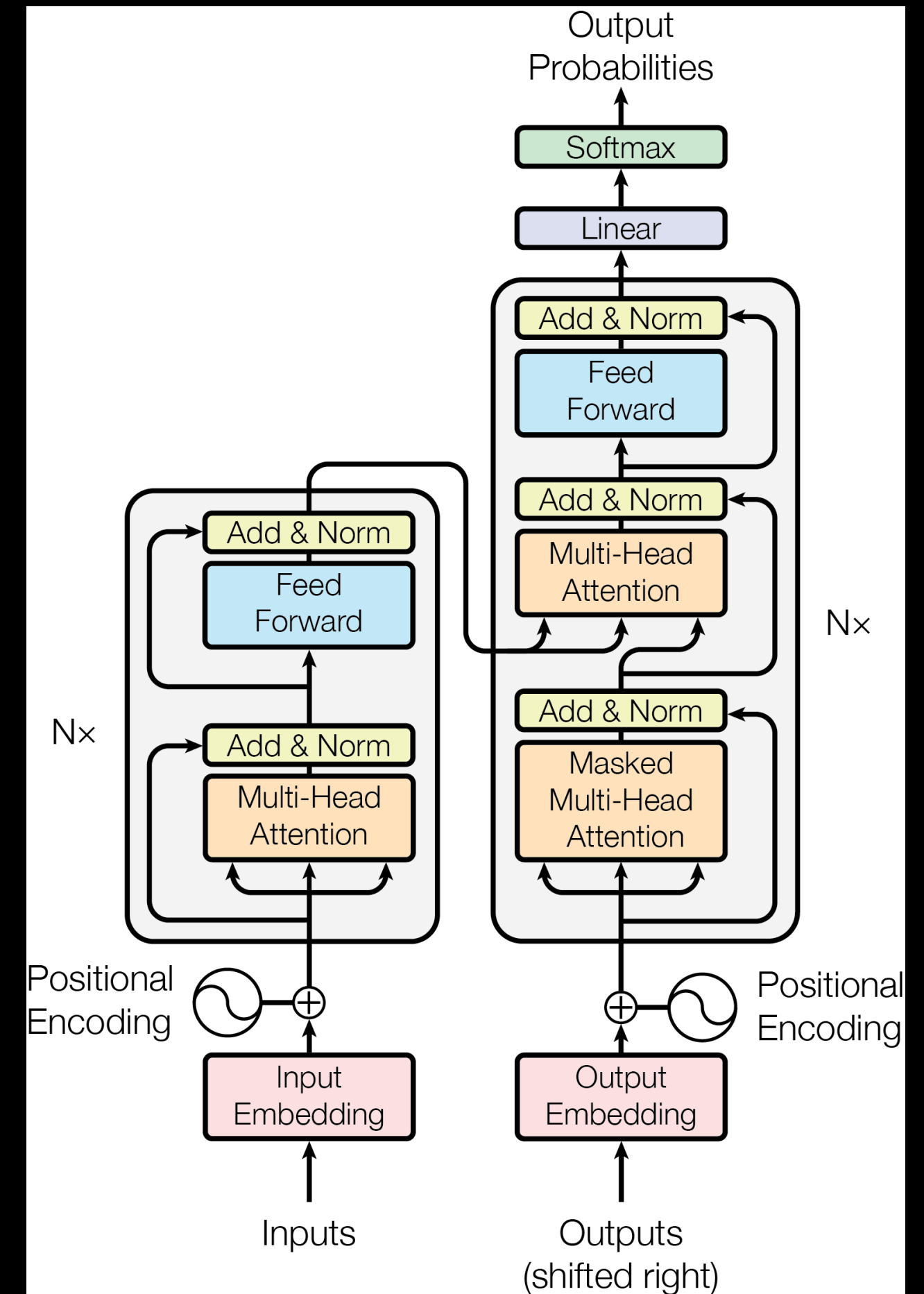
- 순방향으로 흐르는 신경망
- Linear layer로 구성
 - MLP, FCL 등 다양한 이름
- 논문 상에서는 아래와 같이 표현
 - $\text{FFNN}(x) = W_2 \text{ReLU}(W_1 x + b_1) + b_2$
- Attention과 더불어 토큰의 차원 간 의미 연산 수행





Transformer의 구조

- Positional encoding
- Encoder-decoder 구조
- Attention block
 - Multi-head
 - Masked attention
- Add & Norm
- FFNN(Feed Forward Neural Network)
- Output layer

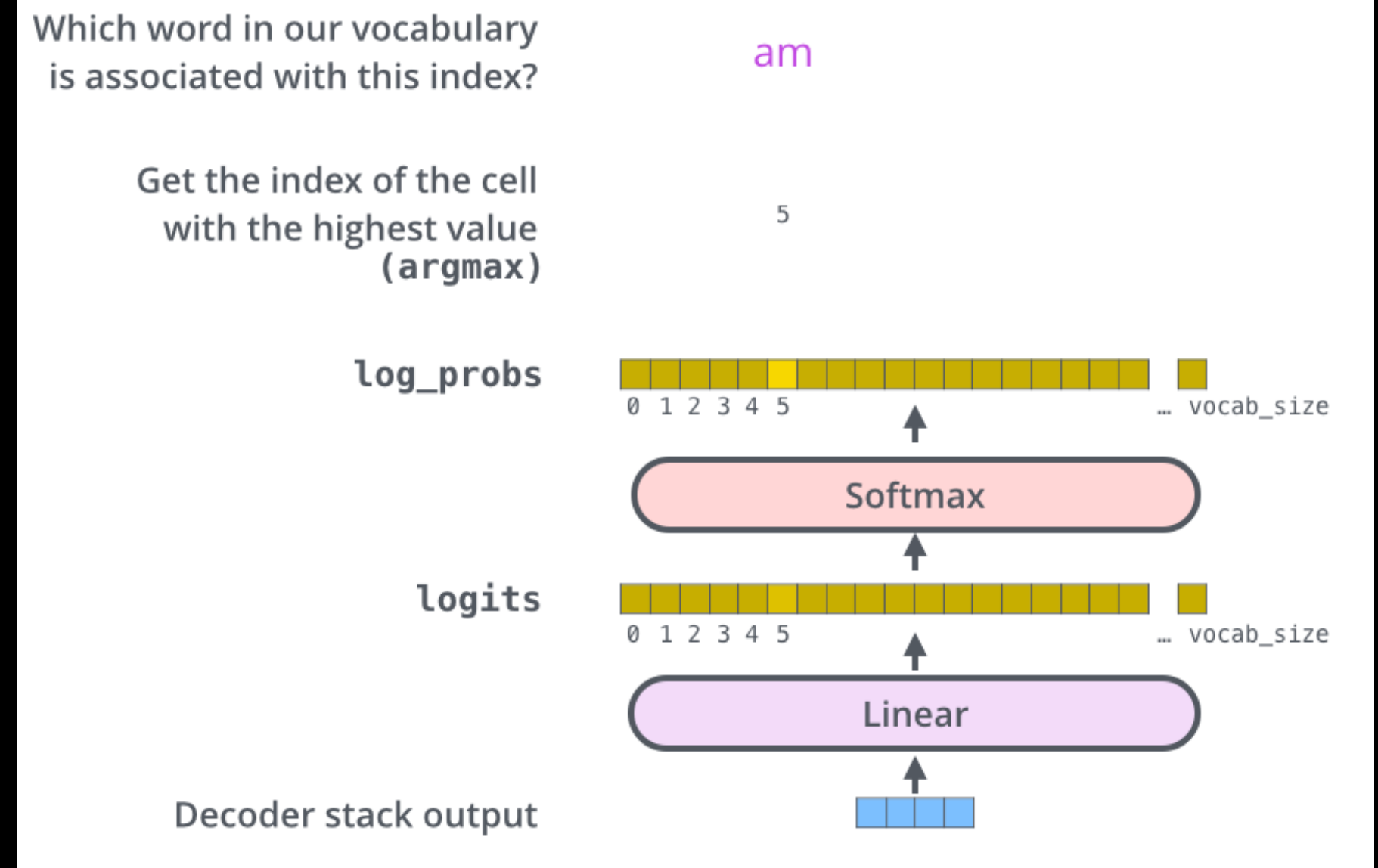


Linear layer

- Decoder block에서 출력된 Matrix 차원 축소
- Feature의 내용을 종합하여 단일 Output으로 정제
- Vocab 별 Logits 값을 계산

Softmax

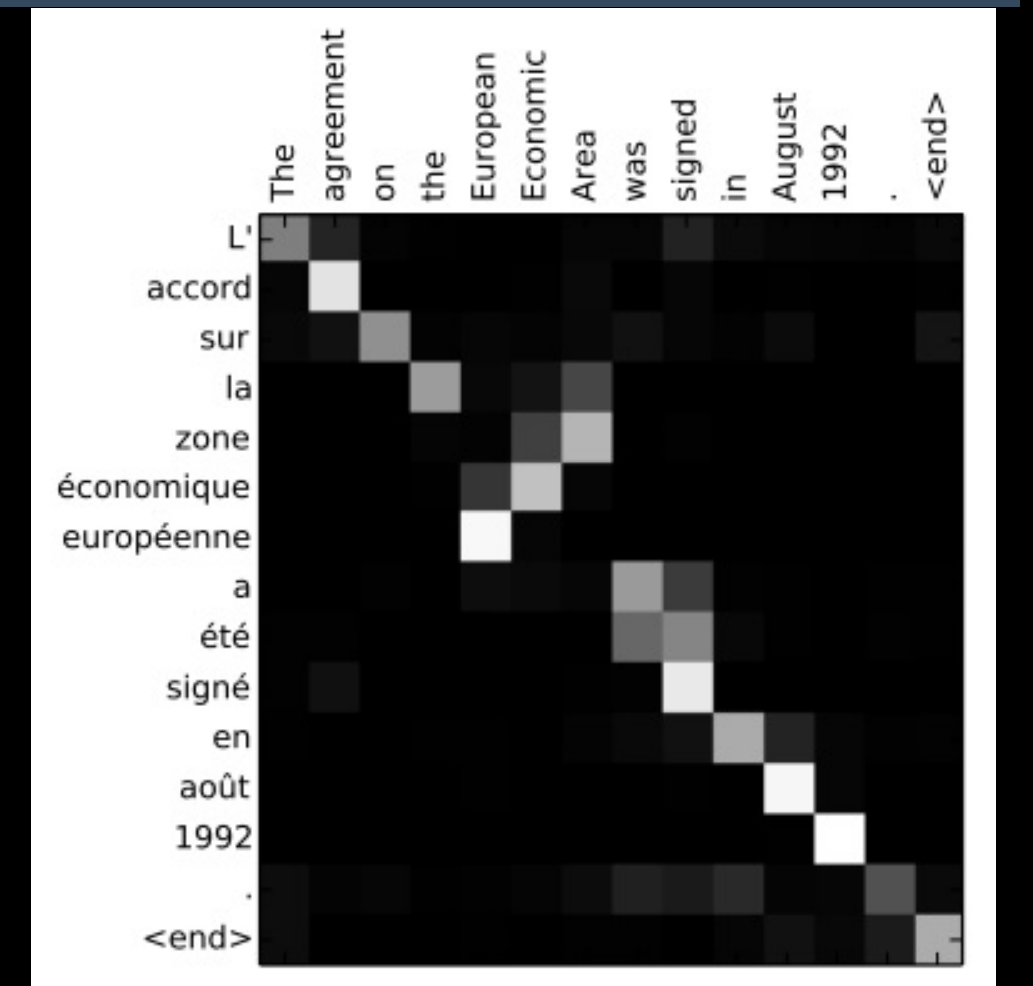
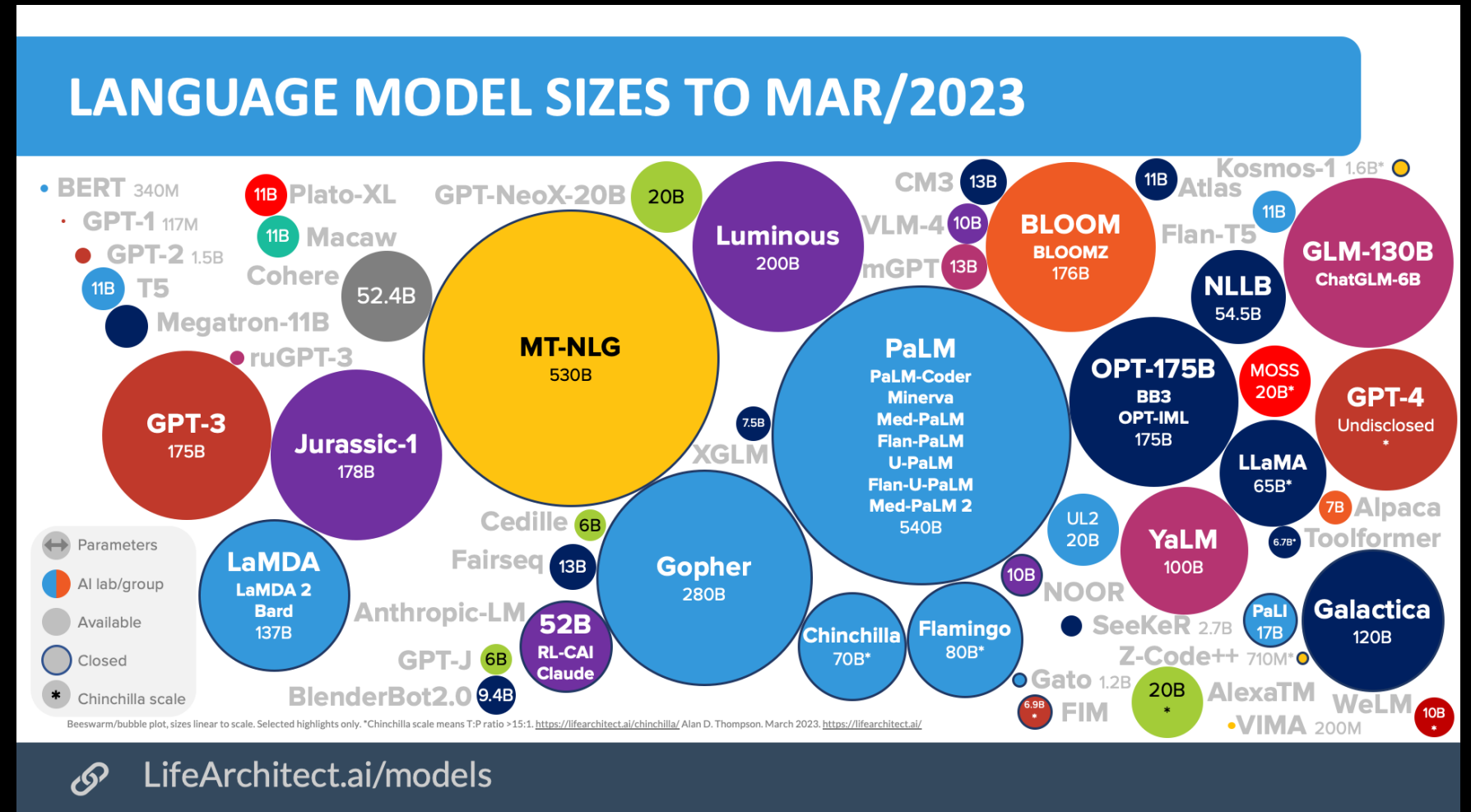
- Logit 값을 Vocab 별 확률로 변환
- 입력된 문장과 과거 생성된 토큰을 바탕으로
- 모델이 알고 있는 단어 중 하나를 선택





Transformer의 영향

- 대규모 데이터 병렬 학습을 통한 속도 향상
- 모델 크기에 비례하여 성능 향상
- 길이가 긴 문장도 잘 이해
- 하나의 모델이 여러 기능 수행
 - 다양한 문제를 해결하는 문장 생성
 - 다개국어 학습 가능
- 모델의 내부 연산 확인 가능





Transformer의 영향

거대 언어 모델(LLM)의 등장

- 대규모 데이터와 컴퓨팅 파워의 결합
- Ex) GPT, LLaMA, Claude
- 일반적인 태스크에서 높은 성능, 다양한 응용 가능

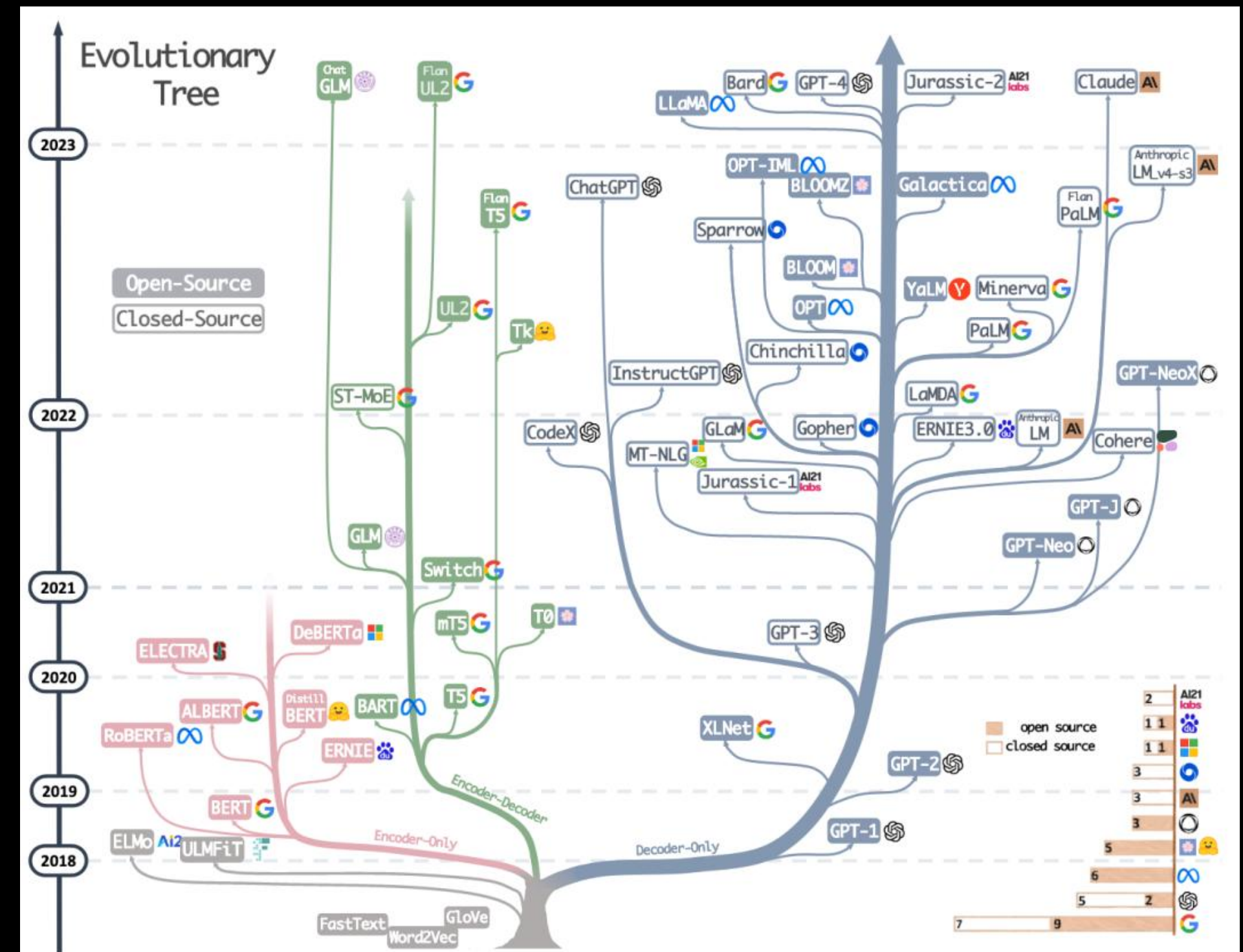
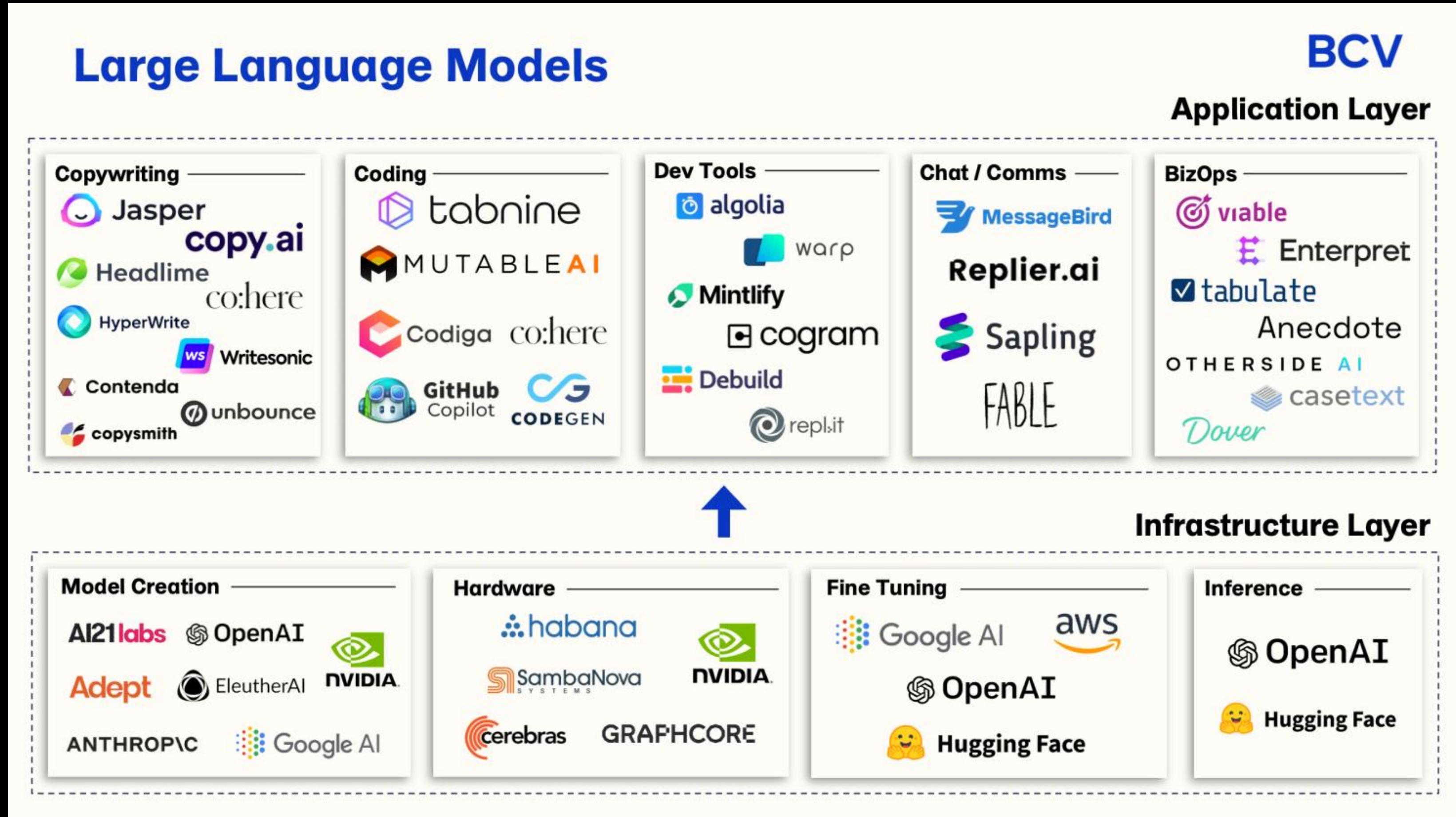


Fig. 1. The evolutionary tree of modern LLMs traces the development of language models in recent years and highlights some of the most well-known models. Models on the same branch have closer relationships. Transformer-based models are shown in non-grey colors: decoder-only models in the blue branch, encoder-only models in the pink branch, and encoder-decoder models in the green branch. The vertical position of the models on the timeline represents their release dates. Open-source models are represented by solid squares, while closed-source models are represented by hollow ones. The stacked bar plot in the bottom right corner shows the number of models from various companies and institutions.

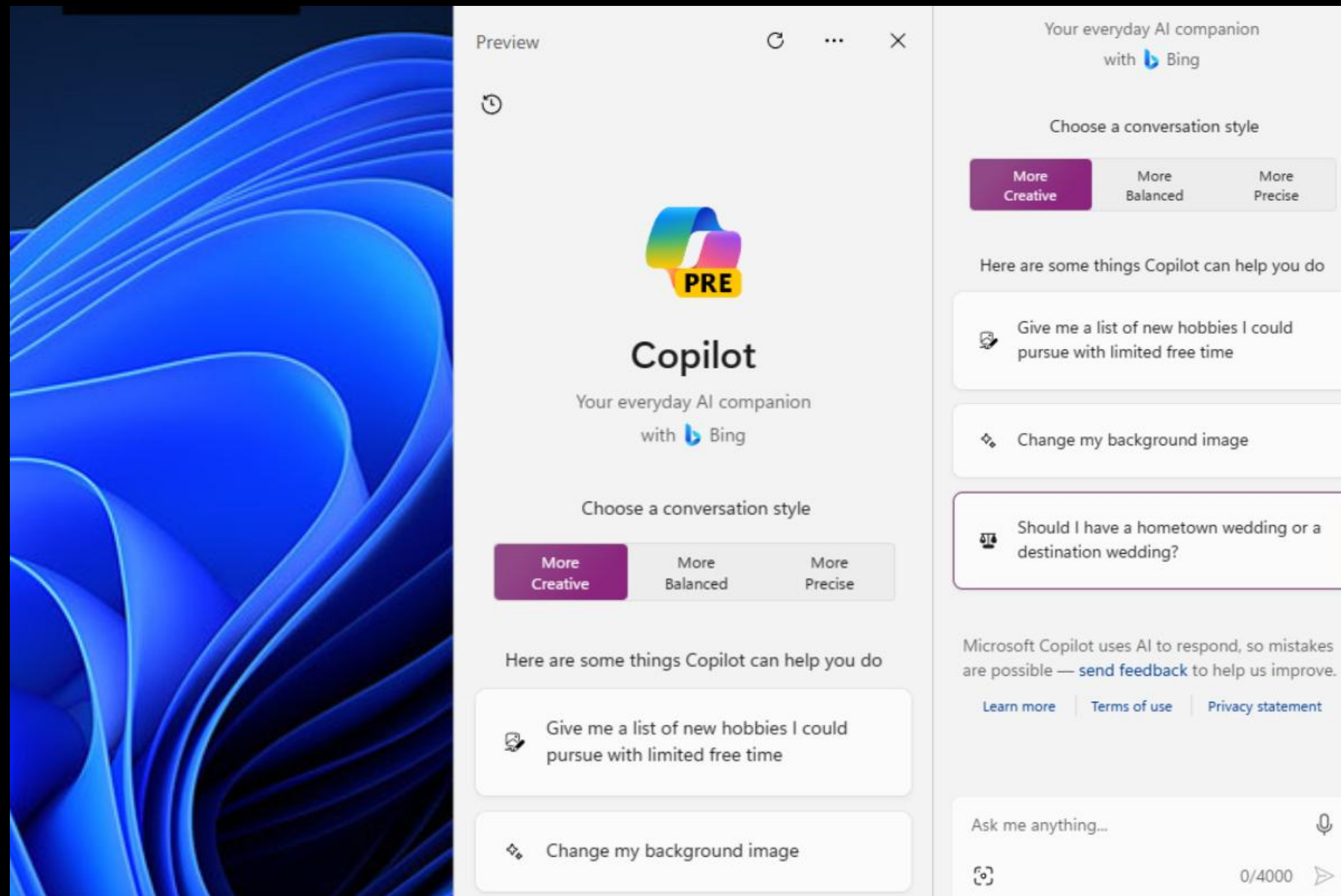


LLM 기반 다양한 서비스



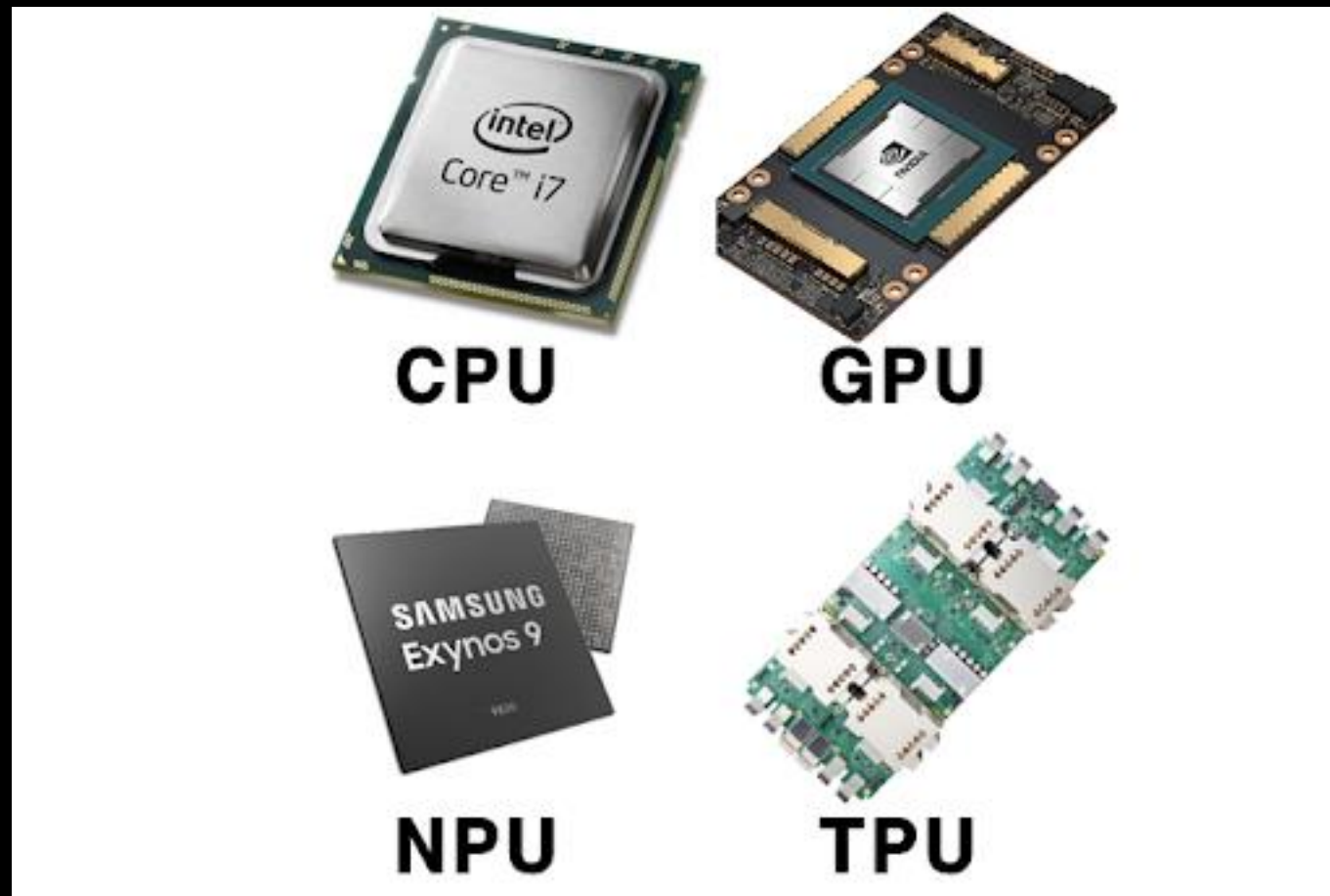


LLM 응용 소프트웨어





LLM으로 인한 하드웨어의 변환



연합뉴스

FT "위기의 삼성전자, SK하이닉스에 엔지니어 뺏길 위험"

FT "위기의 삼성전자, SK하이닉스에 엔지니어 뺏길 위험" · 엔지니어 "HBM은 SK하이닉스, 파운드리만 TSMC에 밀려 분위기 어둡다" · 전문가 "여러...

1개월 전



중앙일보

[단독] '마누라·자식 빼고 바꿔' 31살 삼성전자 위기...주 64시간 근무

"마누라와 자식 빼고 다 바꿔라"며 고(故) 이근희 삼성전자 전대회장이 '신경영'을 선언한 지 31주년이 되는 가운데, 삼성전자 내부에선 위...

2024. 6. 6.



한국경제

"위기 상황인 것 같다"...경고 쏟아진 삼성 반도체[황정수의 반도체 이슈 짚어보기]

위기 상황인 것 같다...경고 쏟아진 삼성 반도체, 기रो에 선 삼성전자 반도체 (1) 흔들리는 30년 1위 신화.

2024. 2. 3.

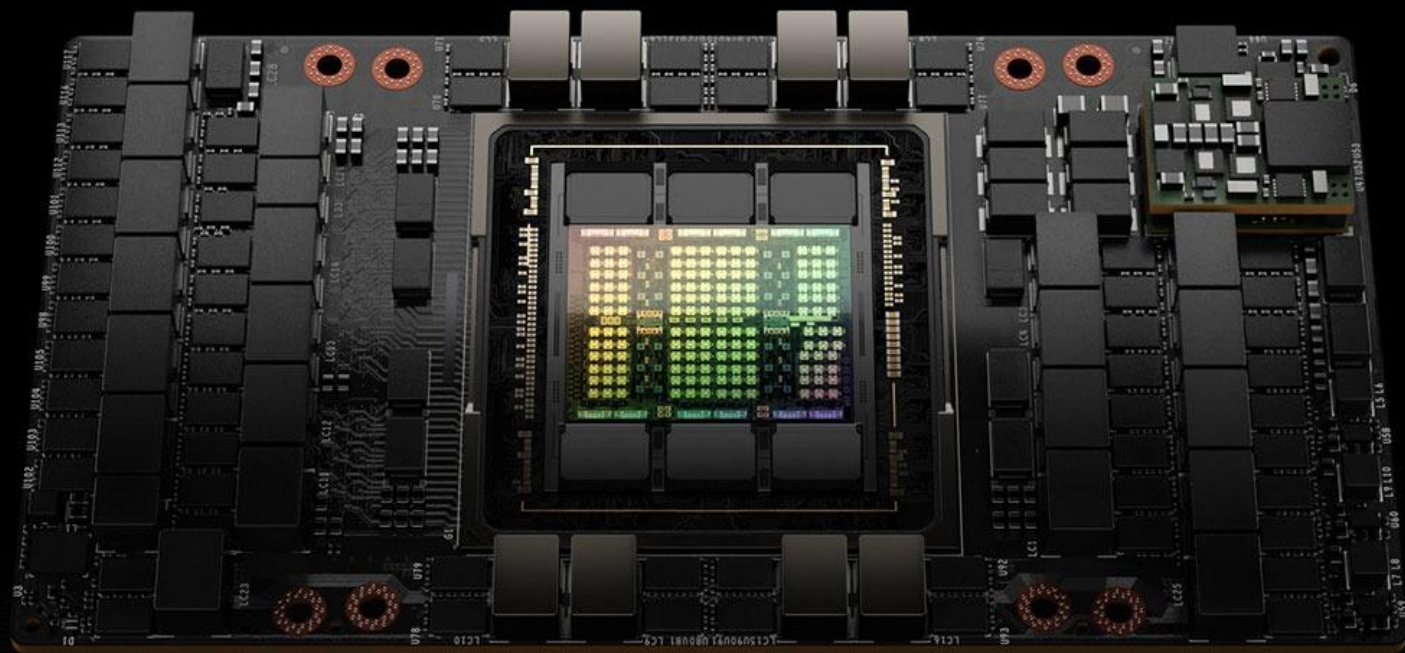


동아일보

[사설]삼성전자 창사 이래 첫 파업... '반도체 위기 탈출' 발목 잡나

삼성전자 최대 노동조합인 전국삼성전자노동조합(전삼노)이 어제부터 사흘간 총파업에 들어갔다. 삼성전자에서 파업이 발생한 것은 1969년 창사 이래...

1개월 전



BERT 과정

PyTorch로 구현한 Transformer 아키텍처

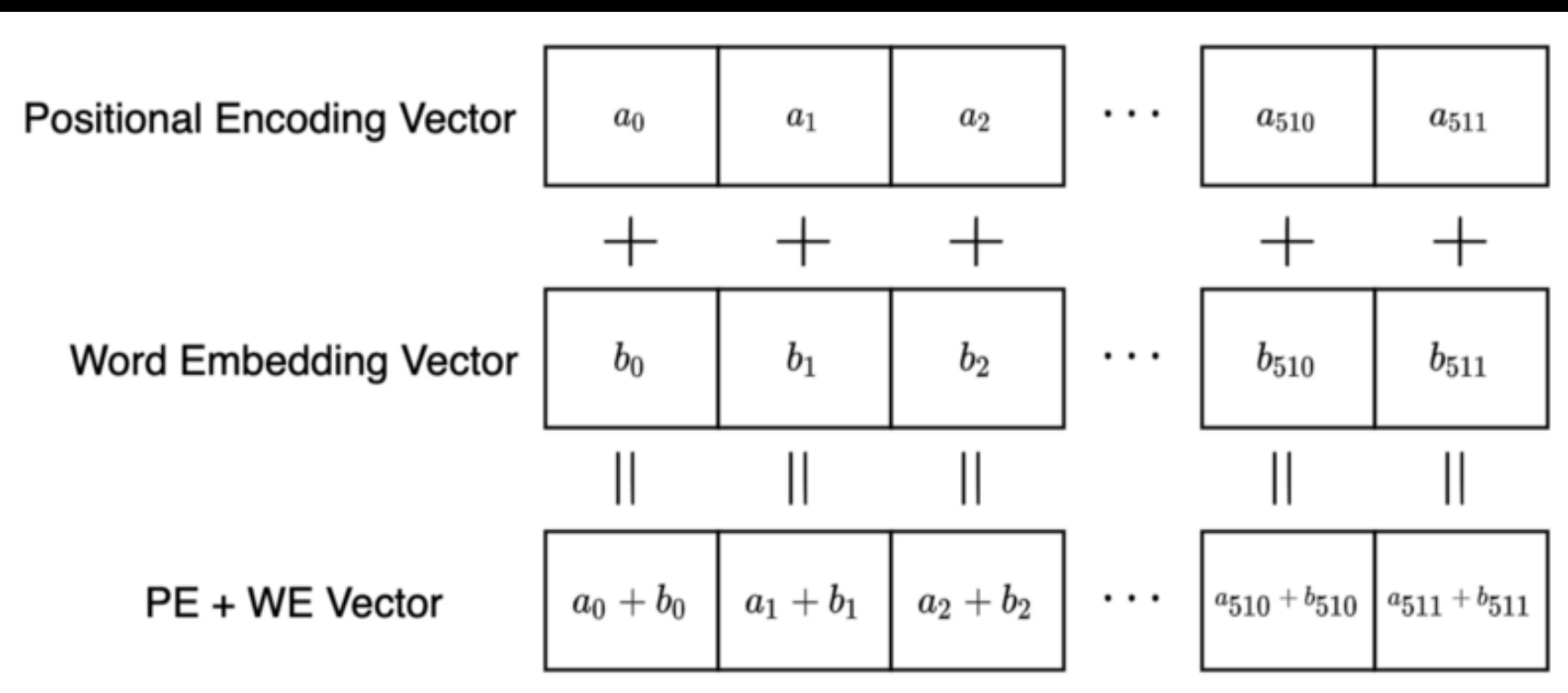
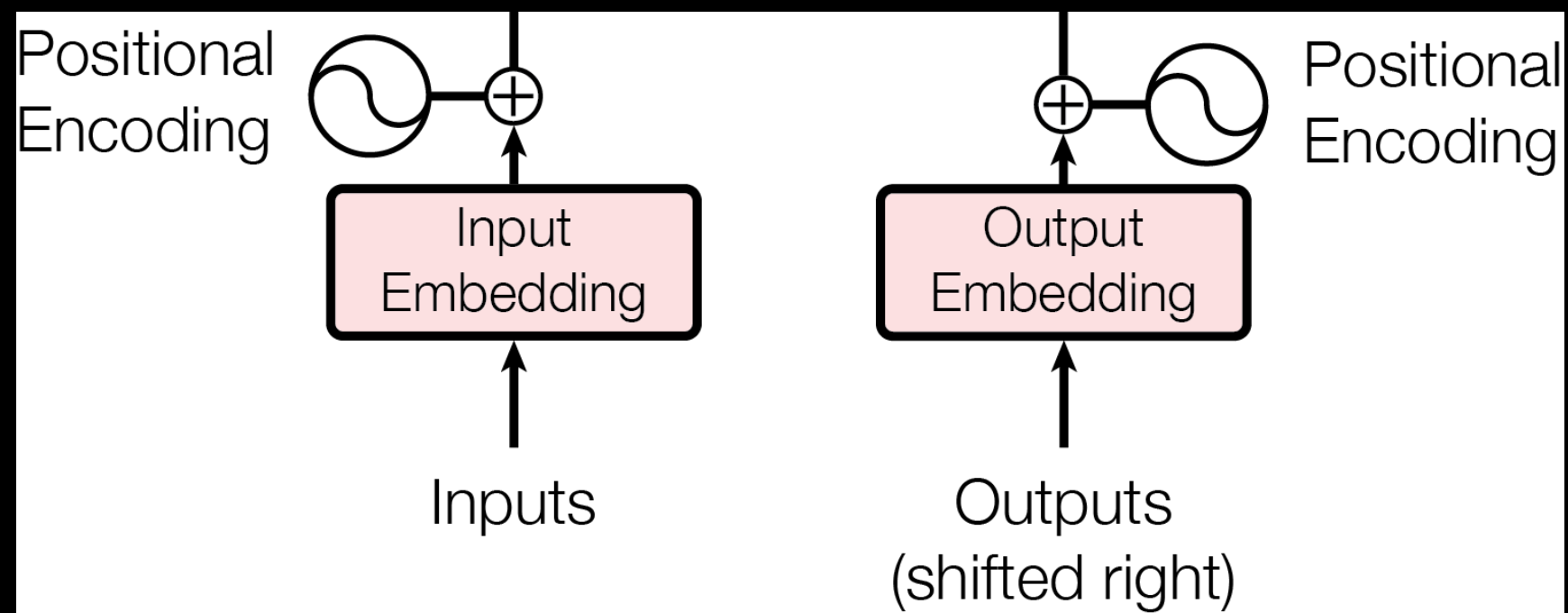
목차 PyTorch로 구현한 Transformer 아키텍처

1. PyTorch로 구현한 Transformer 모델

5. PyTorch로 구현한 Transformer



Positional Encoding



```
class PositionalEncoding(nn.Module):
    """
    위치 인코딩 모듈
    입력 임베딩에 시퀀스 내 각 토큰의 상대적 위치 정보를 추가
    """
    def __init__(self, d_model, max_seq_length=5000):
        super(PositionalEncoding, self).__init__()

        # 위치 인코딩 행렬 생성 (max_seq_length x d_model)
        pe = torch.zeros(max_seq_length, d_model)

        # 각 위치에 대한 벡터 생성
        position = torch.arange(0, max_seq_length, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))

        # sin 함수는 짝수 인덱스에, cos 함수는 홀수 인덱스에 적용
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

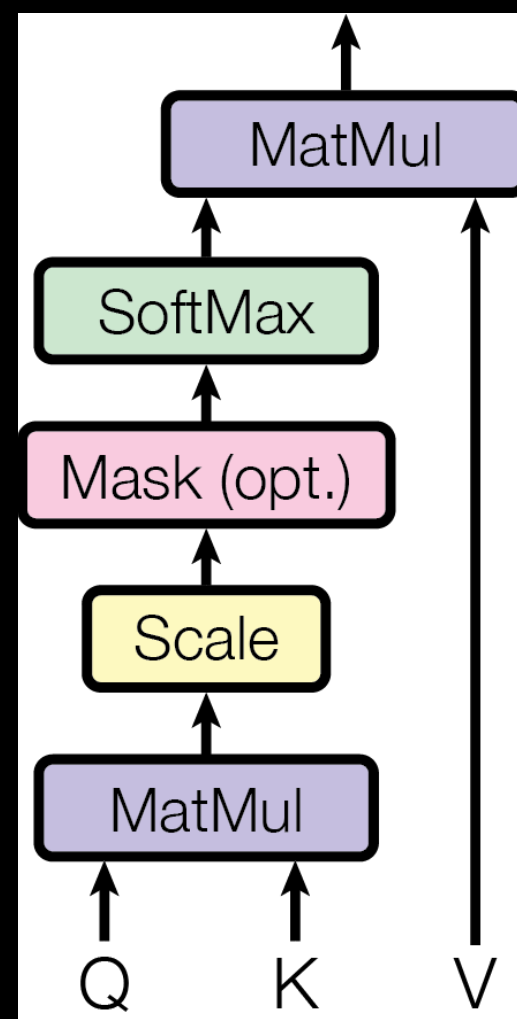
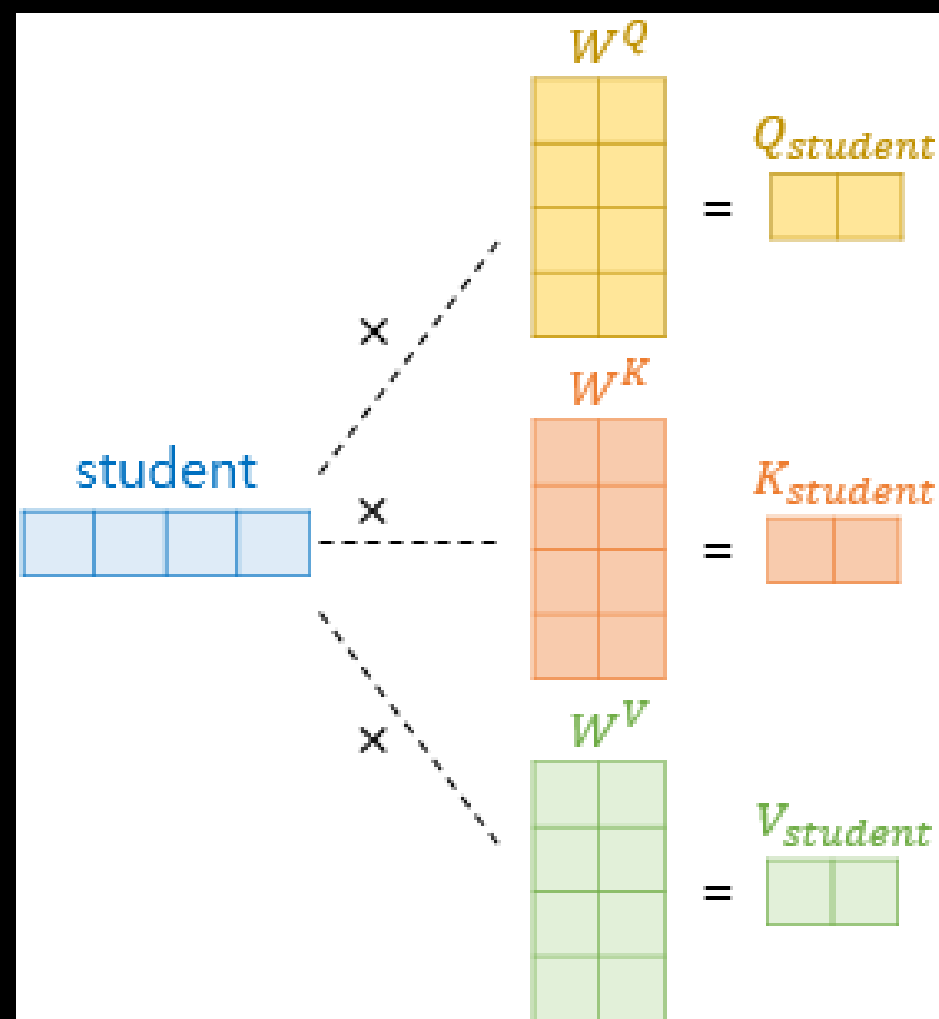
        # 배치 차원 추가 (1 x max_seq_length x d_model)
        pe = pe.unsqueeze(0)

        # 학습되지 않는 버퍼로 등록
        self.register_buffer('pe', pe)

    def forward(self, x):
        """
        Args:
            x: 입력 임베딩 텐서 [batch_size, seq_length, d_model]
        Returns:
            위치 정보가 더해진 텐서 [batch_size, seq_length, d_model]
        """
        # 입력 텐서에 위치 인코딩 더하기 (입력 시퀀스 길이에 맞게 슬라이싱)
        x = x + self.pe[:, :x.size(1), :]
        return x
```

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



```
class ScaledDotProductAttention(nn.Module):
    """
    스케일드 닷-프로덕트 어텐션 메커니즘
    """
    def __init__(self, dropout=0.1):
        super(ScaledDotProductAttention, self).__init__()
        self.dropout = nn.Dropout(p=dropout)

    def forward(self, Q, K, V, mask=None):
        """
        Args:
            Q: 쿼리 [batch_size, n_heads, seq_length, d_k]
            K: 키 [batch_size, n_heads, seq_length, d_k]
            V: 값 [batch_size, n_heads, seq_length, d_v]
            mask: 어텐션 마스크 [batch_size, 1, seq_length, seq_length]

        Returns:
            어텐션 출력과 어텐션 가중치
        """
        # 어텐션 스코어 계산:  $Q \times K^T / \sqrt{d_k}$ 
        d_k = Q.size(-1)
        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)

        # 마스크 적용 (필요한 경우)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)

        # 소프트맥스 적용하여 어텐션 가중치 계산
        attn_weights = F.softmax(scores, dim=-1)
        attn_weights = self.dropout(attn_weights)

        # 가중치와 값을 곱하여 컨텍스트 벡터 계산
        output = torch.matmul(attn_weights, V)

        return output, attn_weights
```



Multi-Head Attention

```
class MultiHeadAttention(nn.Module):
    """
    멀티헤드 어텐션 모듈
    여러 개의 어텐션 헤드를 병렬로 수행
    """
    def __init__(self, d_model, n_heads, dropout=0.1):
        super(MultiHeadAttention, self).__init__()

        # 헤드 수와 차원 설정
        self.n_heads = n_heads
        self.d_model = d_model

        # 각 헤드의 차원 (d_model을 n_heads로 나눔)
        self.d_k = d_model // n_heads
        self.d_v = d_model // n_heads

        # Q, K, V를 위한 선형 변환 레이어
        self.W_Q = nn.Linear(d_model, d_model)
        self.W_K = nn.Linear(d_model, d_model)
        self.W_V = nn.Linear(d_model, d_model)

        # 어텐션 메커니즘
        self.attention = ScaledDotProductAttention(dropout)

        # 출력 선형 변환 레이어
        self.W_O = nn.Linear(d_model, d_model)

        self.dropout = nn.Dropout(p=dropout)
        self.layer_norm = nn.LayerNorm(d_model)
```

```
def forward(self, Q, K, V, mask=None):
    """
    Args:
        Q: 쿼리 [batch_size, seq_length, d_model]
        K: 키 [batch_size, seq_length, d_model]
        V: 값 [batch_size, seq_length, d_model]
        mask: 어텐션 마스크 [batch_size, 1, seq_length, seq_length]
    Returns:
        멀티헤드 어텐션 출력
    """
    batch_size = Q.size(0)
    residual = Q # 잔차 연결을 위해 원본 저장

    # 선형 투영 및 헤드 분할
    # [batch_size, seq_length, d_model] -> [batch_size, seq_length, n_heads, d_k] -> [batch_size, n_heads, seq_length, d_k]
    Q = self.W_Q(Q).view(batch_size, -1, self.n_heads, self.d_k).transpose(1, 2)
    K = self.W_K(K).view(batch_size, -1, self.n_heads, self.d_k).transpose(1, 2)
    V = self.W_V(V).view(batch_size, -1, self.n_heads, self.d_v).transpose(1, 2)

    # 스케일드 닷-프로덕트 어텐션 적용
    attn_output, attn_weights = self.attention(Q, K, V, mask)

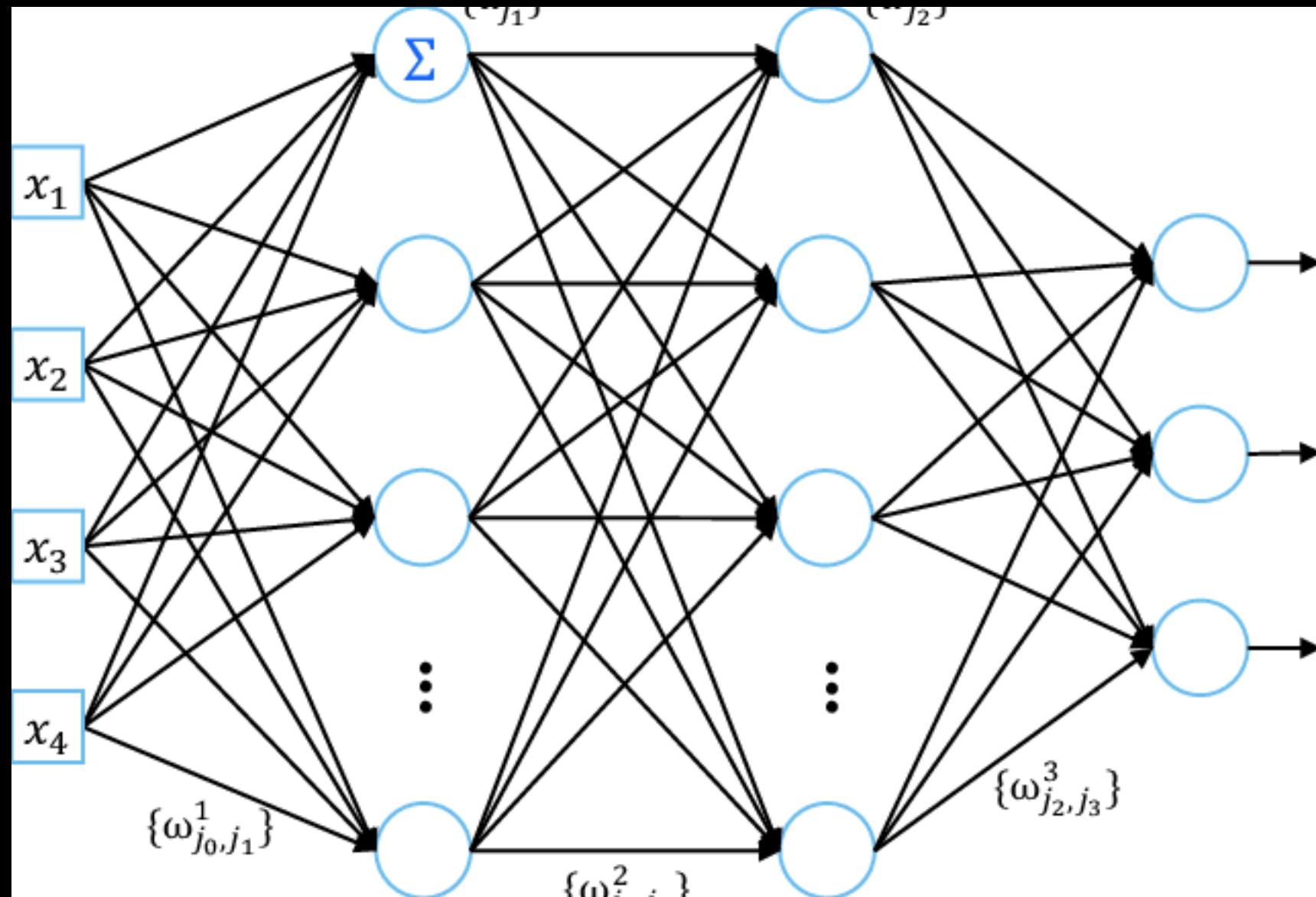
    # 헤드 결합
    # [batch_size, n_heads, seq_length, d_v] -> [batch_size, seq_length, n_heads, d_v] -> [batch_size, seq_length, d_model]
    attn_output = attn_output.transpose(1, 2).contiguous().view(batch_size, -1, self.d_model)

    # 최종 선형 변환, 드롭아웃, 잔차 연결, 레이어 정규화
    output = self.W_O(attn_output)
    output = self.dropout(output)
    output = self.layer_norm(output + residual) # Add & Norm

    return output, attn_weights
```




FFNN



```
class PositionwiseFeedForward(nn.Module):
    """
    포지션와이즈 피드포워드 네트워크 모듈
    두 개의 선형 변환과 ReLU 활성화 함수로 구성
    """

    def __init__(self, d_model, d_ff, dropout=0.1):
        super(PositionwiseFeedForward, self).__init__()
        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)
        self.dropout = nn.Dropout(p=dropout)
        self.layer_norm = nn.LayerNorm(d_model)

    def forward(self, x):
        """
        Args:
            x: 입력 텐서 [batch_size, seq_length, d_model]
        Returns:
            변환된 텐서 [batch_size, seq_length, d_model]
        """
        residual = x # 잔차 연결을 위해 원본 저장

        # 첫 번째 선형 변환 및 ReLU 활성화
        x = F.relu(self.fc1(x))

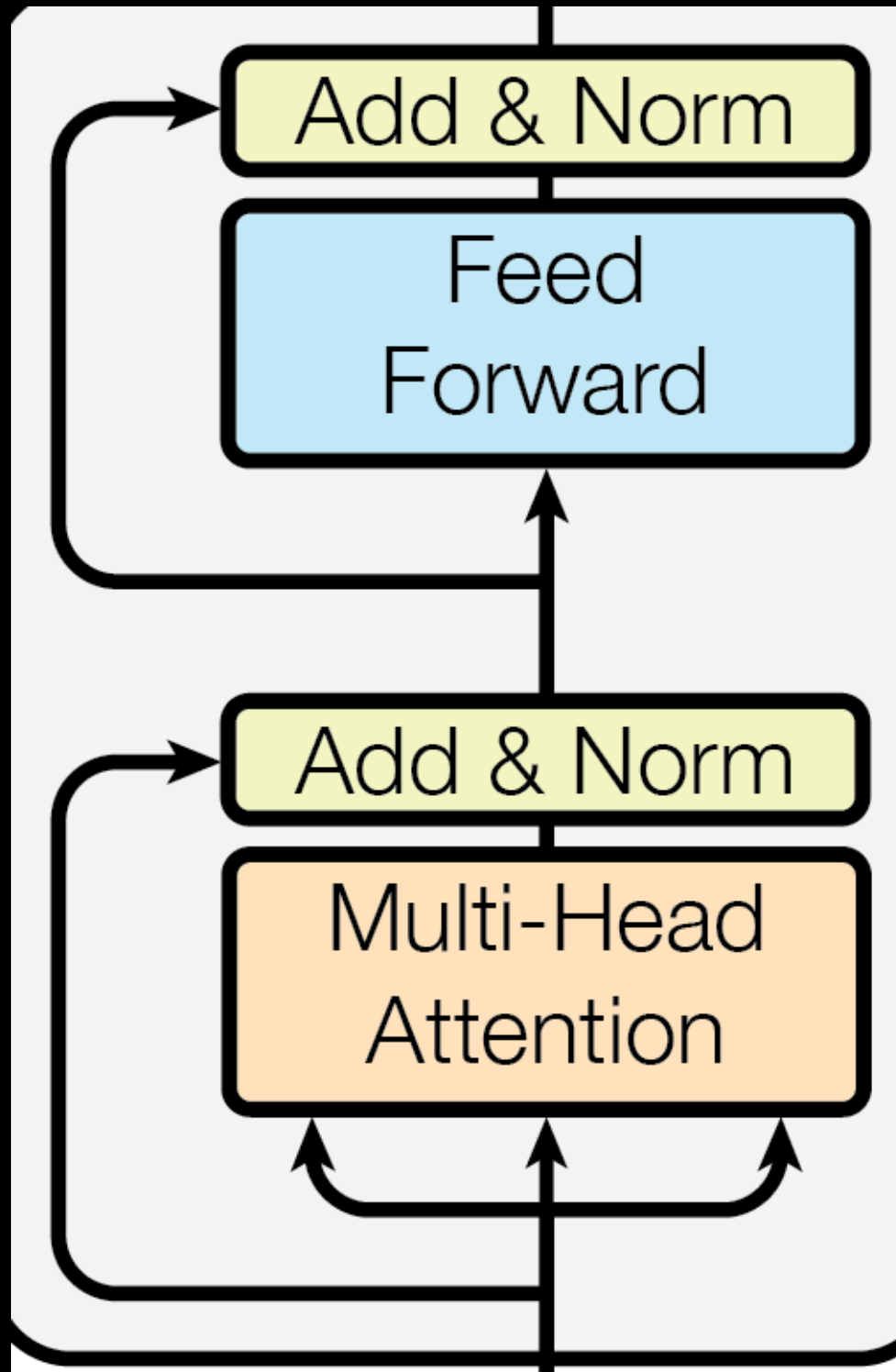
        # 두 번째 선형 변환 및 드롭아웃
        x = self.fc2(x)
        x = self.dropout(x)

        # 잔차 연결 및 레이어 정규화 (Add & Norm)
        x = self.layer_norm(x + residual)

        return x
```



Encoder Layer



```
class EncoderLayer(nn.Module):
    """
    트랜스포머 인코더 레이어
    멀티헤드 어텐션과 포지션와이즈 피드포워드 네트워크로 구성
    """

    def __init__(self, d_model, n_heads, d_ff, dropout=0.1):
        super(EncoderLayer, self).__init__()
        self.self_attention = MultiHeadAttention(d_model, n_heads, dropout)
        self.feed_forward = PositionwiseFeedForward(d_model, d_ff, dropout)

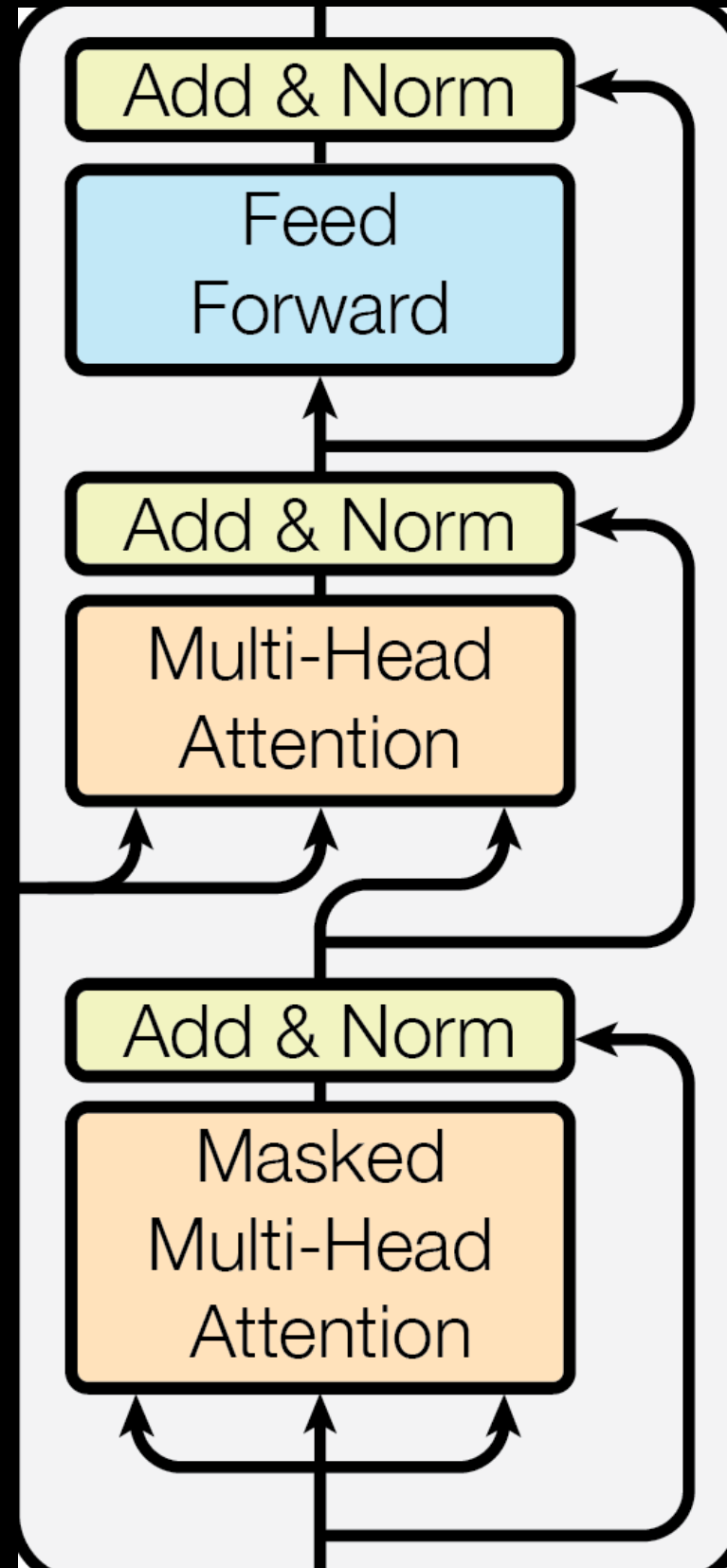
    def forward(self, x, src_mask=None):
        """
        Args:
            x: 입력 텐서 [batch_size, src_seq_length, d_model]
            src_mask: 소스 시퀀스 마스크 [batch_size, 1, src_seq_length, src_seq_length]
        Returns:
            인코더 레이어 출력 [batch_size, src_seq_length, d_model]
        """
        # 셀프 어텐션 적용
        attn_output, _ = self.self_attention(x, x, x, src_mask)

        # 피드포워드 네트워크 적용
        output = self.feed_forward(attn_output)

        return output
```



Decoder Layer



```
class DecoderLayer(nn.Module):
    """
    트랜스포머 디코더 레이어
    마스크드 멀티헤드 셀프 어텐션, 인코더-디코더 어텐션, 피드포워드 네트워크로 구성
    """

    def __init__(self, d_model, n_heads, d_ff, dropout=0.1):
        super(DecoderLayer, self).__init__()
        self.self_attention = MultiHeadAttention(d_model, n_heads, dropout)
        self.enc_dec_attention = MultiHeadAttention(d_model, n_heads, dropout)
        self.feed_forward = PositionwiseFeedForward(d_model, d_ff, dropout)

    def forward(self, x, enc_output, tgt_mask=None, src_tgt_mask=None):
        """
        Args:
            x: 입력 텐서 [batch_size, tgt_seq_length, d_model]
            enc_output: 인코더 출력 [batch_size, src_seq_length, d_model]
            tgt_mask: 타겟 시퀀스 마스크 (미래 정보 차단) [batch_size, 1, tgt_seq_length, tgt_seq_length]
            src_tgt_mask: 소스-타겟 어텐션 마스크 [batch_size, 1, tgt_seq_length, src_seq_length]

        Returns:
            디코더 레이어 출력 [batch_size, tgt_seq_length, d_model]
        """
        # 마스크드 셀프 어텐션 적용 (미래 토큰 참조 방지)
        attn_output, _ = self.self_attention(x, x, x, tgt_mask)

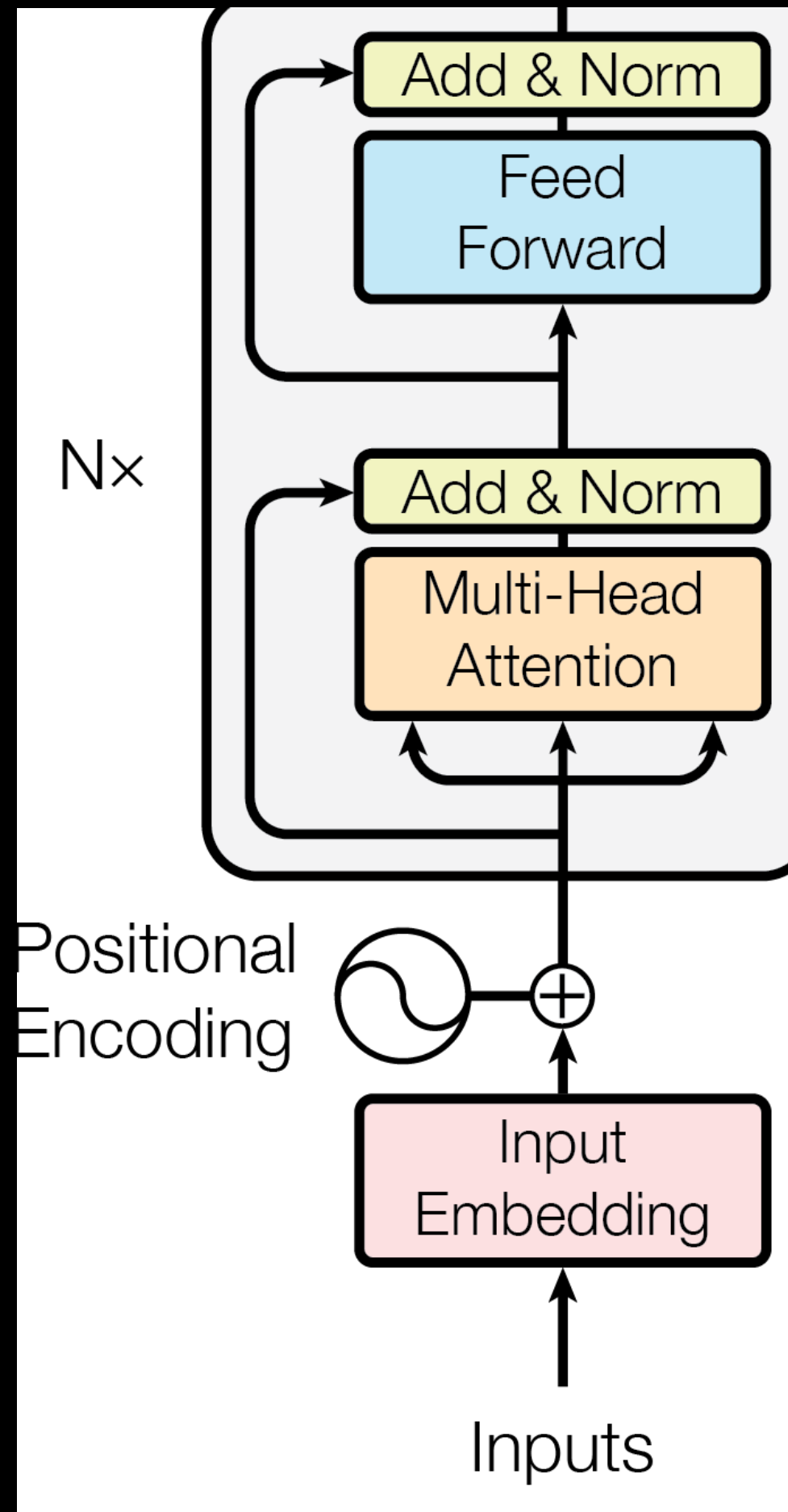
        # 인코더-디코더 어텐션 적용 (인코더 출력에 대한 어텐션)
        enc_attn_output, _ = self.enc_dec_attention(attn_output, enc_output, enc_output, src_tgt_mask)

        # 피드포워드 네트워크 적용
        output = self.feed_forward(enc_attn_output)

        return output
```




Encoder



```
class Encoder(nn.Module):
```

```
    """
```

```
    트랜스포머 인코더
```

```
    임베딩, 위치 인코딩, 다수의 인코더 레이어로 구성
```

```
    """
```

```
    def __init__(self, vocab_size, d_model, n_layers, n_heads, d_ff, dropout=0.1, max_seq_length=5000):
```

```
        super(Encoder, self).__init__()
```

```
        self.embedding = nn.Embedding(vocab_size, d_model)
```

```
        self.positional_encoding = PositionalEncoding(d_model, max_seq_length)
```

```
        self.dropout = nn.Dropout(p=dropout)
```

```
        # 여러 개의 인코더 레이어 스택
```

```
        self.layers = nn.ModuleList([
```

```
            EncoderLayer(d_model, n_heads, d_ff, dropout)
```

```
            for _ in range(n_layers)
```

```
        ])
```

```
    def forward(self, x, src_mask=None):
```

```
        """
```

```
        Args:
```

```
            x: 입력 시퀀스 [batch_size, src_seq_length]
```

```
            src_mask: 소스 시퀀스 마스크 [batch_size, 1, src_seq_length, src_seq_length]
```

```
        Returns:
```

```
            인코더 출력 [batch_size, src_seq_length, d_model]
```

```
        """
```

```
        # 임베딩 및 위치 인코딩 적용
```

```
        x = self.embedding(x) * math.sqrt(self.embedding.embedding_dim)
```

```
        x = self.positional_encoding(x)
```

```
        x = self.dropout(x)
```

```
        # 모든 인코더 레이어를 통과
```

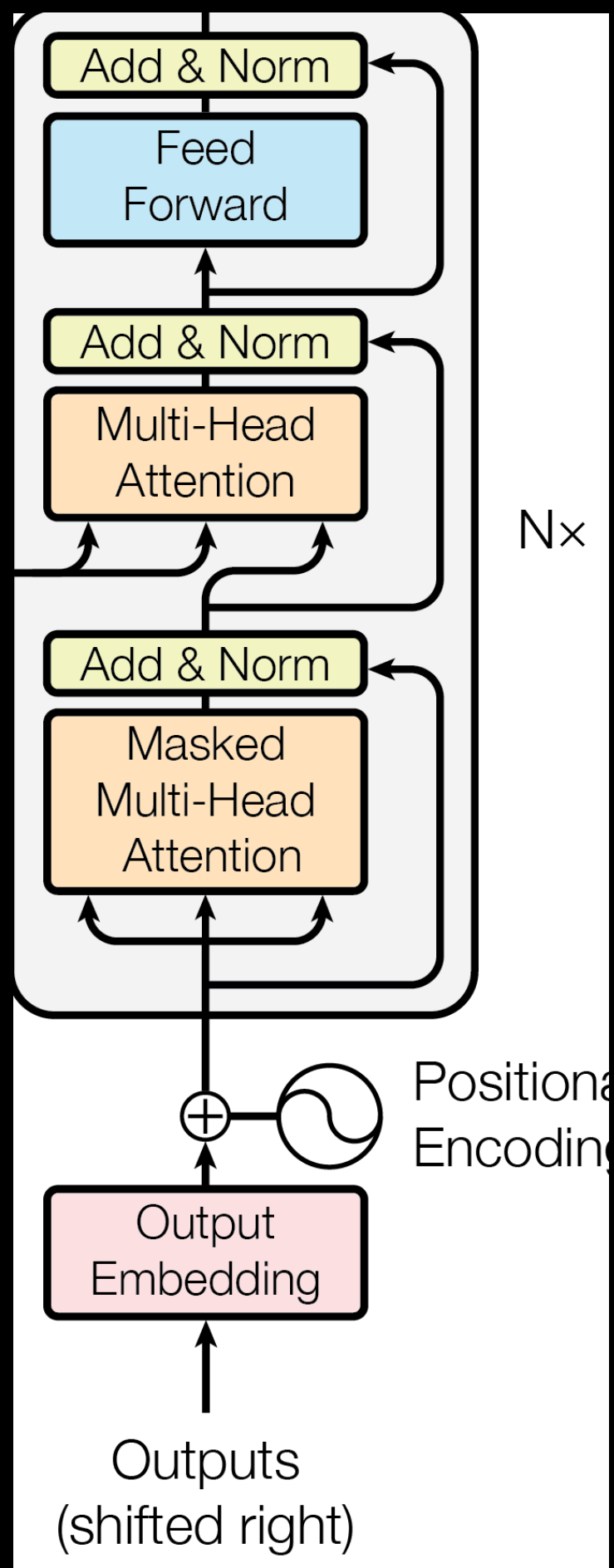
```
        for layer in self.layers:
```

```
            x = layer(x, src_mask)
```

```
        return x
```



Decoder



```
class Decoder(nn.Module):  
    """  
    트랜스포머 디코더  
    임베딩, 위치 인코딩, 다수의 디코더 레이어로 구성  
    """  
  
    def __init__(self, vocab_size, d_model, n_layers, n_heads, d_ff, dropout=0.1, max_seq_length=5000):  
        super(Decoder, self).__init__()  
        self.embedding = nn.Embedding(vocab_size, d_model)  
        self.positional_encoding = PositionalEncoding(d_model, max_seq_length)  
        self.dropout = nn.Dropout(p=dropout)  
  
        # 여러 개의 디코더 레이어 스택  
        self.layers = nn.ModuleList([  
            DecoderLayer(d_model, n_heads, d_ff, dropout)  
            for _ in range(n_layers)  
        ])  
  
    def forward(self, x, enc_output, tgt_mask=None, src_tgt_mask=None):  
        """  
        Args:  
        x: 입력 시퀀스 [batch_size, tgt_seq_length]  
        enc_output: 인코더 출력 [batch_size, src_seq_length, d_model]  
        tgt_mask: 타겟 시퀀스 마스크 [batch_size, 1, tgt_seq_length, tgt_seq_length]  
        src_tgt_mask: 소스-타겟 어텐션 마스크 [batch_size, 1, tgt_seq_length, src_seq_length]  
        Returns:  
        디코더 출력 [batch_size, tgt_seq_length, d_model]  
        """  
        # 임베딩 및 위치 인코딩 적용  
        x = self.embedding(x) * math.sqrt(self.embedding.embedding_dim)  
        x = self.positional_encoding(x)  
        x = self.dropout(x)  
  
        # 모든 디코더 레이어를 통과  
        for layer in self.layers:  
            x = layer(x, enc_output, tgt_mask, src_tgt_mask)  
  
        return x
```



Transformer

```
class Transformer(nn.Module):
    """
    트랜스포머 모델
    인코더, 디코더, 출력 레이어로 구성된 전체 모델
    """

    def __init__(self, src_vocab_size, tgt_vocab_size, d_model=512, n_layers=6,
                  n_heads=8, d_ff=2048, dropout=0.1, max_seq_length=5000):
        super(Transformer, self).__init__()

        # 모델 구성 요소 및 하이퍼파라미터
        self.d_model = d_model
        self.encoder = Encoder(src_vocab_size, d_model, n_layers, n_heads, d_ff, dropout, max_seq_length)
        self.decoder = Decoder(tgt_vocab_size, d_model, n_layers, n_heads, d_ff, dropout, max_seq_length)

        # 출력 레이어 (선형 변환 + 소프트맥스)
        self.output_layer = nn.Linear(d_model, tgt_vocab_size)

    def forward(self, src, tgt, src_mask=None, tgt_mask=None, src_tgt_mask=None):
        """
        Args:
            src: 소스 시퀀스 [batch_size, src_seq_length]
            tgt: 타겟 시퀀스 [batch_size, tgt_seq_length]
            src_mask: 소스 시퀀스 패딩 마스크
            tgt_mask: 타겟 시퀀스 마스크 (미래 정보 차단)
            src_tgt_mask: 인코더-디코더 어텐션 마스크

        Returns:
            출력 확률 분포 [batch_size, tgt_seq_length, tgt_vocab_size]
        """

        # 인코더 통과
        enc_output = self.encoder(src, src_mask)

        # 디코더 통과
        dec_output = self.decoder(tgt, enc_output, tgt_mask, src_tgt_mask)

        # 출력 레이어 통과
        output = self.output_layer(dec_output)

        return output
```

```
def generate_square_subsequent_mask(self, size):
    """
    미래 정보 참조를 방지하기 위한 마스크 생성
    Args:
        size: 시퀀스 길이
    Returns:
        하삼각행렬 마스크 (대각선 포함)
    """

    # 하삼각행렬(lower triangular matrix)에 -inf를 채우기
    mask = (torch.triu(torch.ones(size, size)) == 1).transpose(0, 1)
    mask = mask.float().masked_fill(mask == 0, float('-inf')).masked_fill(mask == 1, float(0.0))
    return mask

def create_padding_mask(self, seq, pad_idx=0):
    """
    패딩 토큰을 마스킹하기 위한 패딩 마스크 생성
    Args:
        seq: 시퀀스 [batch_size, seq_length]
        pad_idx: 패딩 토큰 인덱스
    Returns:
        패딩 마스크 [batch_size, 1, 1, seq_length]
    """

    # 패딩 토큰을 0으로, 나머지를 1로 변환
    mask = (seq != pad_idx).unsqueeze(1).unsqueeze(2)
    return mask
```